Agentic OS Audit

- Independent technical audit and research study of a production agentic operating system — by Haris Mekic, built with Claude Opus 4.8 and other models
- Executive Summary — Structured Technical Audit of a Gate-First, Bring-Your-Own-Model Agentic Operating-System Shell
  - What the system under review (SUR) is
  - The audit headline
  - The strongest VERIFIED findings
  - The honest posture
  - Why it matters
  - Navigation — the rest of this audit
- The System Under Review — Architecture of a Local-First, BYOM, Gated, Auditable Agentic Operating Layer
  - Purpose and scope of this document
  - 1. What the system is, in one paragraph
  - 2. The three-layer shape
  - 3. Components and what each one owns
  - 4. Data flow
  - 5. The gate model
  - 6. The memory model
  - 7. How the SUR's components were obtained (consumption, in brief)
  - 8. The architecture in one diagram (described)
  - 9. Auditor's architectural summary
- Consumption Audit — What Was Consumed
  - Scope and method
  - 1. The claim under audit, stated plainly
  - 2. The consumption discipline (how the take is governed)
  - 3. The consumed-versus-built boundary (the central evidence)
  - 4. What was consumed — the ledger
  - 5. The direction of flow (the one-way claim, corrected)
  - 6. Why this matters for the comparison (the defensibility question)
  - 7. Honest gaps and auditor cautions
  - 8. Verdict
- Capabilities — The Capability Catalog of a Gate-First, BYOM Agentic Operating Layer
  - Purpose and scope of this document
  - How to read this catalog
  - Catalog at a glance
  - 1. BYOM model router
  - 2. Context-discipline sandbox (context-pressure policy)
  - 3. Tool integration (the allow-listed action runner)
  - 4. Parallel agent orchestration (the workforce surface)
  - 5. Human-in-the-loop gate layer (autonomy dial + fast-verify queue)
  - 6. Mechanical safety hooks (enforcement by construction)
  - 7. File-based memory (persistent state in plain files)
  - 8. Audit surface + token meter (structured, append-only, fail-closed)
  - 9. Regression memory (the quality-gated retention loop)
  - 10. Local server + reliability posture
  - Auditor summary
- Scalability — A Quantitative Study
  - How to read this document

    - 0. Notation and stated assumptions
    - 1. Token economics — cost per task, caching savings, cost-to-serve
    - 2. Parallel multi-agent concurrency math
    - 3. Own-server + sandboxed-tool + full-stack scaling model
    - 4. Reliability engineering for scale — the availability math
    - 5. Literature anchors (the science under the numbers)
    - 6. Synthesis — where the numbers land, honestly
- Comparative Analysis — The System Under Review Against ECC, OpenClaw, Hermes, and Nous
    - How to read this comparison
    - 1. The property matrix
    - 2. Per-competitor honest sections
    - 3. The consume-vs-harness differentiator, examined
    - 4. What GPT said
    - 5. What researchers say
    - 6. Synthesis — where the comparison lands
- Methodology — Built With Claude
    - How to read this document
    - The alignment frame this methodology answers to
    - 1. The honest premise: this was built with AI, and says so
    - 2. The human-plus-AI build loop
    - 3. The discipline applied to AI-written code
    - 4. Verification practice
    - 5. Deterministic-scripted versus reasoned: the dividing line
    - 6. The cost and scale of the build itself
    - 7. Honest limits of this methodology audit
    - 8. Auditor's net assessment
- Metrics — Honest, Labeled, and Scoped
    - How to read this document
    - 1. Token volume and API-equivalent cost-to-serve
    - 2. Capability-module inventory
    - 3. Documented build-and-verify iterations
    - 4. Acceptance-test pass rates — the strongest reproducible-shape evidence
    - 5. The public reproducibility harness — the strongest freshly-publishable proof
    - 6. Small-model fine-tune deltas — where candor is the finding
    - 7. Scaling multipliers — every one scoped
    - 8. The literature-grounded upside — measured elsewhere, not here
    - 9. One-line honest summary
    - 10. What an auditor would tell a buyer about these numbers
- Limits, Honesty Ledger & Reproducibility
    - Why this document exists
    - 1. The consolidated honesty ledger
    - 2. The PARTIAL → VERIFIED unlock paths
    - 3. Reproducibility — what an outside reviewer can do now vs not
    - 4. The honest known limits
    - 5. What this ledger lets a reviewer conclude
    - 6. Strongest counterarguments — pre-empted, not hidden
- Claims & Evidence Register — Make the Audit Independently Checkable
    - Purpose
    - A. Reproducible right now (public repo only)
    - B. Reproducible with a clean checkout + a run
    - C. Not reproducible from public materials yet
    - The pattern across the register
- References

- 1. The paradigm — LLM-as-Operating-System / Software 3.0
- 2. AI safety / alignment context — AI 2027
- 3. Multi-agent orchestration — the case FOR and AGAINST
- 4. Context engineering & long-horizon reliability
- 5. Inference economics
- 6. The open-source agentic-OS ecosystem (comparables)
- 7. The open SKILL.md standard (shared, cross-vendor layer)
- 8. Pattern sources consumed (pattern-only, no vendor code)

Independent technical audit and research study of a production agentic operating system — by Haris Mekic, built with Claude Opus 4.8 and other models

Executive Summary — Structured Technical Audit of a Gate-First, Bring-Your-Own-Model Agentic Operating-System Shell

Structured technical audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

> *Methodology note: this audit was built by the same AI model (Claude Opus 4.8) that built the platform under review, with Sonnet, GPT, and Gemini for cross-checks. Evidence labeled VERIFIED rests on publicly readable code and public registry data that any reviewer can confirm independently; it does not rest on the AI's self-assessment. Evidence labeled PARTIAL or UNVERIFIED is explicitly bounded to that status. This is not a professionally-independent audit; it is a structured, labeled, multi-model review. See doc 06 for the full methodology and doc 08 §6 for the direct rebuttal to the "an audit written with the subject's own AI is not independent" objection.*

What the system under review (SUR) is

The system under review is a **verifiable, gate-first, bring-your-own-model (BYOM) agentic operating-system shell**: a React front end over a **pure-stdlib Python** HTTP server on the local loopback interface (`127.0.0.1:8911`), same-origin (no CORS to configure or trust), published under **Apache-2.0**. The operator supplies the intelligence (the model is brought, not bundled); the platform supplies the operating surface — a chat channel, an agent grid, an org chart, an approval queue, per-agent autonomy dials, and health/context/cost panels. It implements a public, named paradigm — the LLM-as-operating-system / "Software 3.0" frame articulated by Andrej Karpathy — rather than inventing one. (_DIGEST.md §2, §5; capabilities.md)

It was built by **one person with AI assistance over ~19 recorded calendar days** (2026-05-16 → 2026-06-03), not by a funded team. That asymmetry is stated once, plainly, and carried through every judgement below: it makes the throughput notable and the unfinished edges expected. The honest characterisation of the *shipped public artifact* is narrow and accurate — on the scope it actually ships, it delivers what it claims; several capabilities the *wider* platform describes are roadmap, not in the public repo. (_DIGEST.md §3.1, §8; metrics.md §1)

The audit headline

In this category, **the agent harness and the open `SKILL.md` capability format are now commodities.** The Anthropic-authored `SKILL.md` standard (published 2025-12-18) was adopted by Microsoft (VS Code) and OpenAI (ChatGPT + Codex CLI) within ~48 hours; by ~2026-03 roughly **32 tools** read the same format from the same directory shape, and the dominant open-source incumbents (OpenClaw, Hermes) plus the SUR all converge on it — the incumbents are even cross-compatible with each other. (openclaw.md §4; gpt-competitors.md §2.2)

The SUR's defensibility, if any, is therefore structural — not architectural. It does not rest on a unique harness, because no such uniqueness is available to anyone in this category. It rests on the layers the commodity harnesses lack by default: a mechanical human-in-the-loop gate layer, least-privilege execution, sealed/auditable output boundaries, a fail-closed quality guard, and a disciplined, mostly-one-way consumption method that re-builds commodity patterns into an owned composition. That is the spine of this audit, and it is the only competitive claim the evidence supports. (_DIGEST.md §1, §9)

The strongest VERIFIED findings

#	Finding	Evidence label
1	**The harness + skill format are commodities; the contest is on the layers above them.** The open standard is cross-vendor and the incumbents are cross-compatible. The SUR competes on gate/audit/memory/consumption discipline — not on a harness anyone can match.	**VERIFIED** (openclaw.md §4; gpt-competitors.md §2.2)
2	**A human-in-the-loop gate layer that actually bites.** An autonomy dial (L0–L5) multiplied by a 4–20 risk score routes each action to act / report / queue; **even at maximum autonomy (L5) a high-risk action still queues** to an append-only `gate-queue.jsonl` rather than executing.	**VERIFIED** (_DIGEST.md §2 row 5; capabilities.md)
3	**Least-privilege execution + mechanical safety hooks.** Tool calls run through a closed allow-list action runner (unknown action → returns the allowed list, executes nothing; explicitly not a raw shell); **8/8 (PARTIAL — internal self-audit)** safety hooks were present, parsing, and running — with one "firing-but-blind" hook caught and fixed in the same pass.	**VERIFIED** (runner, hooks present); self-catch is **PARTIAL** (internal log) (_DIGEST.md §2 rows 3,6; §3.3)
4	**Inspectable, fail-closed memory and retention.** File-based memory reads only declared files (never globs) and surfaces a missing file honestly as "missing"; the quality-gated retention loop **KEEPs only if no metric degrades**, else performs a byte-exact, sha256-verified auto-revert.	**VERIFIED** (_DIGEST.md §2 rows 7,9)
5	**A self-down-labelling claim register — the trust signal.** Of **12 tracked product claims: 4 VERIFIED / 8 PARTIAL / 0 UNVERIFIED.** The 4 VERIFIED are exactly those whose evidence was *actually run* on the collection date (file-memory survives restart; the approval gate; BYOM routing against a standard-shape stand-in; a deterministic export/redaction gate biting 3/3 fixtures, zero leaks). A platform that grades its own claims down is exhibiting auditor-grade honesty.	**VERIFIED** (metrics.md §4; gpt-competitors.md §1.2)
6	**Disciplined, structurally-evidenced consumption.** Patterns were consumed from permissively-licensed projects (SWE-agent, agentevals — both MIT; OpenClaw/Hermes pattern-only) and re-built inside an owned stdlib-Python framework. Independence is visible:	**VERIFIED** structurally; **PARTIAL** for one paid-provider adapter (mechanism present, live call not run) (consumption.md; _DIGEST.md §7)

#	Finding	Evidence label
	different language from the vendors, segregated reference copies, **running code that does not import the references**.	

Why finding #2 is the load-bearing one. The category's cautionary tale is the exact failure this gate layer targets. OpenClaw — the most-starred incumbent (~247K–355K stars, date-dependent) — carries a documented one-click RCE (CVE-2026-25253, CVSS 8.8), command-injection and indirect-prompt-injection CVEs, a companion-service breach exposing ~1.5M auth tokens, and a published "shadow superuser" demonstration. The academic remedy is itself a **staged human-in-the-loop defense layer** that lifted sandbox-escape defense from a ~17% baseline into the **19%–92%** range (Claude ~91.5%). That remedy *is* the SUR's claimed differentiator. (openclaw.md; gpt-competitors.md §2; researchers.md) — events **VERIFIED**; exact single-paper percentages **PARTIAL**.

The honest posture

Auditor integrity is the deliverable, so the limits are stated as prominently as the strengths:

- **Cost is API-equivalent cost-to-serve, NOT cash.** The platform ran largely on a flat-rate subscription. The machine-generated ledger records **~10.53 billion tokens / 19,333 messages / 556 transcript files** over 19 days, of which **~93% are cache-read tokens** at the cheapest tier — which is *why* blended cost is low (≈ $0.00256 per 1,000 tokens). Current API-equivalent figure: **~$26,920** (**PARTIAL** — rate-dependent). The widely-quoted "~$10K" is a **stale** 2026-05-24 snapshot. (metrics.md §1)
- **Counts are an inventory, not a validation.** The 165 capability modules are self-asserted and reconciler-guarded; a meaningful fraction carry "encoded but not yet independently validated" status. Acceptance tallies are real and their mechanisms inspectable, but most runs live in internal logs — full external reproducibility needs a clean-checkout run: **14/14 (PARTIAL — internal log) site generation**; the **quality-eval** harness passes the original five categories at **100%** and clears its ≥70% gate at **~79% (PARTIAL — internal run)** on the full expanded suite (the failures are concentrated in newer, harder framing/discipline categories, not the original five — see metrics.md §4); **26/28 (PARTIAL) smoke** on the current expanded suite (the older "19/19" denominator is a strict subset). (**PARTIAL** — metrics.md §2–§4)
- **Four wider-platform capabilities are roadmap, NOT shipped in the public repo:** a distinct context-discipline sandbox-execute primitive, a general MCP tool layer, a concurrent multi-agent *execution* engine (the orchestration *surface* is present; the engine is not), and a failure-pattern regression memory. The public shell should not be over-read as performing autonomous fan-out-and-merge. (**UNVERIFIED in repo** — _DIGEST.md §2, §8)
- **The small-model fine-tune track is unfinished.** On a ~0.5B-parameter base against a pre-registered bar, results ran **50 pairs → ≈ −20pp (a regression)**, then **~150 pairs → +13.3pp** (first positive, but below the +15pp bar); the **~5,900-pair projection is UNVERIFIED** (not yet run). It is a capability-in-progress; the regression and sub-threshold numbers left in the record are the credibility signal. (metrics.md §6)
- **The math sits inside the public envelope.** Caching takes ~42% off a representative cached turn but does **not** cancel the ~15× multi-agent fan-out premium (Anthropic's own research: +90.2% on its eval at ~15× tokens, token usage explaining ~80% of performance variance). Concurrency saturates (Amdahl) at an observed **~8–10 effective workers**, over-determined by the synthesis tax, the shared-context failure mode (per Cognition's steel-man), and universal context degradation. Cost-tier routing is ~3.0× cheaper on a routine-heavy mix but is a **depreciating** asset as inference prices fall ~50×/yr. (scalability.md; researchers.md) — **VERIFIED** arithmetic on **stated** assumptions.
- **Where the SUR is behind:** reach/ecosystem (incumbents at 175K–355K stars and large skill marketplaces; ECC ships a $19/seat/mo paid tier with ~1,000-test CI), funding, team, a shipped paid go-to-market, and an open-model-production line. **The gap is distribution and third-party trust-proof, not engineering.** Competitor maturity is a roadmap to learn from, not a deficit to spin. (ecc.md; gpt-competitors.md)

- **A note on "defensibility."** "Defensible" and "structural edge" are used throughout to mean *"currently differentiates from documented competitor failures as of the audit date."* This is **not** a claim that any described component cannot be copied — it can, because this audit describes it (a gate function, a retention loop, and an allow-listed runner are each a day's work to reproduce from the descriptions here). The claim is narrower and current-state: the VERIFIED shipped capabilities constitute a present structural advantage over the documented failure modes of the most-adopted incumbents (the one-click RCE, the ~1.5M-token breach, the "shadow superuser" demonstration) — **not a permanent, copy-proof advantage** against a well-resourced team. The four roadmap capabilities (below) contribute nothing to this claim and should not be read as if they do.

No source claims, and this audit does not assert, that the SUR "beats frontier," is "fully sovereign," or approaches "AGI." There is no on-record external-AI endorsement; GPT/OpenAI appear only as a request-shape compatibility target and an optional backend (no successful real external call is logged). The single load-bearing external-review datapoint is behavioural, not a benchmark: under a less-correlated reviewer, the SUR persona **conceded and corrected** a claim rather than doubling down. (gpt-competitors.md Part 1)

Why it matters

In a market where the harness and the open skill format have become commodities, and where the most-adopted incumbents have shipped documented "shadow superuser" security failures, a **mechanical, fail-closed, human-in-the-loop discipline layer** is a credible — and currently scarce — point of differentiation. The platform also demonstrates a posture rare in the category: it **consumes** open-source patterns one-way into an owned framework rather than shipping yet another harness, and it grades its own claims down where the evidence was not run. The realistic verdict: **strong on safety architecture and consumption discipline, inside the envelope on cost and concurrency, and behind on reach, capital, and go-to-market** — a structural edge to defend, not a finished product to oversell.

Navigation — the rest of this audit

Doc	Title	What it covers
`01`	The System Under Review	The published shell described structurally — components, data flow, the gate model, the memory model, with public-code status (capabilities.md)
`02`	Consumption Audit	What was consumed, how (pattern-only vs code), license posture, and the direction value flows (consumption.md)
`03`	Metrics & Evidence	Token volume, cost-to-serve, acceptance tallies, the claim register, the fine-tune deltas (metrics.md)
`04`	Competitor Landscape	OpenClaw, Hermes/Nous, ECC, and the GPT posture (openclaw.md; gpt-competitors.md; ecc.md)
`05`	Scientific Grounding	LLM-as-OS, the multi-agent case for and against, context engineering, long-horizon reliability, alignment risk, inference economics (researchers.md)
`06`	Scalability & Reliability	Per-turn cost math, the 15× premium, Amdahl saturation, reliability leverage (scalability.md)

All figures carry VERIFIED / PARTIAL / UNVERIFIED labels per the source research. Cost figures are API-equivalent cost-to-serve, not cash. Multipliers state their scope. This document is sealed for outward publication and asserts no unbacked superlatives.

The System Under Review — Architecture of a Local-First, BYOM, Gated, Auditable Agentic Operating Layer

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

Purpose and scope of this document

This is the architectural description that the rest of the audit references: what the system under review (SUR) **is**, structurally, before any judgement about whether it is good, defensible, or finished. It describes the components, the data flow, the gate model, and the memory model in neutral engineering terms, and it labels every non-trivial claim **VERIFIED** / **PARTIAL** / **UNVERIFIED** so that a reader can separate what was read in the published source from what is asserted in the wider platform documentation.

The evidence base is the SUR's **already-published, open-source repository** (Apache-2.0; `README.md`, `AGENTS.md`, `docs/ARCHITECTURE.md`, `config.example.json`, the server module docstrings and route table, and the seed data directory) plus the public architecture overview. Where the published code backs a claim, it is **VERIFIED**; where a claim appears only in wider-platform documentation and is not present in the published code, it is labelled **UNVERIFIED in repo** and catalogued as roadmap, not as a shipped feature. (capabilities.md; consumption.md; _DIGEST.md §2, §8)

A terminology note carried throughout: the SUR's relationship to the open-source ecosystem is described as **consumption** — it consumes patterns one-way into its own owned framework and re-implements them in its own stack. It does not ship a third-party harness and it is not itself a harness; where "harness" appears below it describes *other, generic* tools for contrast. The differentiator is the posture: generic tools ship a harness; the SUR consumes patterns and rebuilds them as its own. (consumption.md §1–§4)

---

1. What the system is, in one paragraph

The SUR is a **local-first AI operating-system shell**: a React front end over a small, dependency-free Python server that turns one or more operator-supplied language models into a controllable, gated agent workforce. The intelligence is brought by the operator — **bring-your-own-model (BYOM)** — and the SUR supplies the operating surface around it: a chat channel, an agent grid, an org chart, an approval queue, per-agent autonomy dials, and health / context / token panels. It runs on `127.0.0.1:8911`; the UI and the JSON API are served same-origin, so there is no CORS layer. (**VERIFIED** — `README.md`, `docs/ARCHITECTURE.md`, public overview.) (capabilities.md "What the SUR is")

The frame it implements is a **named, public paradigm** — the LLM-as-operating-system / "Software 3.0" idea articulated by Andrej Karpathy (talk, Y Combinator AI Startup School, 2025; video). The SUR does not claim to invent the paradigm; it implements one. Any defensibility therefore has to come from execution and discipline, not from architectural novelty — a point the audit returns to throughout. (**VERIFIED** — primary source above; see also References)

The mapping is component-for-component, and stating it plainly is part of grading the SUR honestly. In Karpathy's analogy the **model is the compute core**; in the SUR that core is *swappable and operator-supplied* (the BYOM router, §3). The **context window is the working memory**; the SUR treats it as managed RAM with an explicit pressure policy and checkpoints (`context_policy.py`, §3). **Tools, files, and APIs are the resources the OS coordinates**; the SUR exposes them only through an allow-listed action runner rather than a raw shell (§3). Karpathy frames the present as the **1960s-mainframe, time-shared era** — compute centralized and reached

through thin “terminals” (chat); the SUR’s same-origin `127.0.0.1` shell is exactly such a terminal over an operator-owned compute back end. What the analogy does **not** supply — and what the SUR adds as its own engineering — is the **human-in-the-loop gate layer** (§5): an OS schedules and isolates processes, but it does not, by default, withhold an irreversible action pending human approval. That gate is the SUR’s addition on top of the borrowed paradigm, and it is where the audit looks for defensibility (docs 04, 05, 08).

---

2. The three-layer shape

The published artifact has a clean three-layer separation. (**VERIFIED** — `docs/ARCHITECTURE.md`, `AGENTS.md`.) (capabilities.md “What the SUR is”)

Layer	What it is	Implementation	Status
The face	The operator-facing workspace	A Vite + React 18 + TypeScript single-page app	**VERIFIED**
The spine	The coordinator	A Python standard-library HTTP server that serves the SPA and a JSON API and coordinates the subsystems	**VERIFIED**
The data	Persistent state	Plain files (JSON / JSONL / Markdown) under a seed data directory	**VERIFIED**

The defining property of the spine is that it is **pure standard library** — no `pip install` is required to run it. The server module is roughly 43 KB of stdlib Python with a path-traversal-guarded static handler for the built front end and a per-route, per-module defensive load: each backing subsystem is imported inside its own `try` / `except`, so one failing import never takes the server down. (**VERIFIED** — `servari_server.py`; `docs/ARCHITECTURE.md` reliability section.) (capabilities.md §10)

This shape matters for an auditor before any feature does: a stdlib-only, same-origin, file-backed server is exceptionally easy to read, run, and trust. There is no runtime supply chain to vet and no mandatory cloud backend, and “runnable by a stranger” is enforced — a fresh clone renders every panel from seed data. (**VERIFIED** — `docs/ARCHITECTURE.md`.)

---

3. Components and what each one owns

The spine is a set of small, single-responsibility stdlib modules, each fronted by one or more JSON routes. The table below is the component map as evidenced in the published repo. (**VERIFIED** unless noted.) (capabilities.md §1–§10)

Component	Responsibility	Backing module(s)	Representative routes	Status
BYOM model router	Speak the OpenAI-compatible request shape to any operator-chosen provider or local server	`chat_byom.py` (~216 lines)	`/chat/completions` shape	**VERIFIED**
Context-pressure policy	Treat the context window as managed RAM; measure pressure, verify survival pins, checkpoint	`context_policy.py` (~31 KB)	`/api/context`, `/api/context-checkpoint`	**VERIFIED** (policy); distinct sandbox-execute primitive **UNVERIFIED in repo**

Component	Responsibility	Backing module(s)	Representative routes	Status
Allow-listed action runner	Invoke only named, vetted tools — never a raw shell	`servari_server.py` (`ACTIONS` dict)	`/api/run?action=...`	**VERIFIED**; general MCP layer **UNVERIFIED in repo**
Workforce surface	Present and coordinate many agents (grid, org chart, channels, briefs)	`servari_server.py` handlers	`/api/agents`, `/api/agent-channel`, `/api/org`, `/api/grid`, `/api/launch`	surface **VERIFIED**; concurrent execution engine **not in repo**
Human-in-the-loop gate	Compose autonomy level × risk score into act / report / queue; park high-risk actions	`autonomy.py`, `verify_queue.py`	`/api/autonomy`, `/api/set-autonomy`, `/api/verify-queue`, `/api/verify-decision`	**VERIFIED**
File-based memory	Persist all state as plain files; expose a curated, non-glob memory surface	`memory_surface.py`	`/api/memory-surface`	**VERIFIED**
Audit surface + token meter	Append-only audit lines; price logged token counts at configured rates	`tokens.py`	`/api/tokens`, `/api/tokens-sessions`, `/api/tokens-report`	**VERIFIED**; cost = API-equivalent, not cash
Quality-gated retention loop	KEEP a change only if no metric degrades, else byte-exact auto-revert	`retention.py` (~32 KB)	`/api/retention`, `/api/retention-decide`	**VERIFIED**; broader failure-signature memory **UNVERIFIED in repo**
Health surface	Fast, read-only, fail-closed health that never crashes	`health.py`	`/api/health`	**VERIFIED**
Local voice (optional)	Lazy-loaded speech-to-text / text-to-speech skeletons	`voice.py`, `voice_neural.py`	(background thread)	**VERIFIED** (optional; shell runs without it)

A single honesty rail spans this table and is restated wherever it applies: four capabilities described by the **wider** platform — a distinct sandbox-execute primitive, a general MCP tool layer, a concurrent multi-agent *execution* engine, and a failure-signature regression memory — are **not present in the published repo**. They are catalogued as roadmap / integration claims, never as shipped. The published artifact is best characterised as a **verifiable, gate-first BYOM agent-OS shell**, and on that scope it delivers what it claims. (**UNVERIFIED in repo** — capabilities.md “Auditor summary”; _DIGEST.md §2 honesty rail, §8)

---

4. Data flow

The SUR’s runtime is a small number of well-defined paths. Described generically, an operator interaction moves through the system as follows. (**VERIFIED** structurally from the route table + module docstrings; the concurrent fan-out path is the **UNVERIFIED** exception noted below.) (capabilities.md §1, §3, §4, §5, §8)

4.1 The conversational path (single agent)

1. The operator types into a channel in the React front end; the SPA posts to the same-origin JSON API.
2. The **BYOM router** reads a gitignored local config (`provider` / `api_key` / `model` / `base_url`), assembles the request from the **visible** channel history plus **one short, disclosed system line**, and calls the chosen endpoint over `urllib`. There is no hidden prompt beyond that one line, and the model key is transmitted only to the operator’s chosen provider. (**VERIFIED** for the visible system line and single-endpoint call; the “key

never goes anywhere else" property is a design claim consistent with the single `urllib` call but **PARTIAL** — not exhaustively traced.) (capabilities.md §1)
3. Missing or malformed config degrades to an honest "configure your model" message (`ok=False`) — it never crashes and never fabricates a reply. (**VERIFIED** — `load_config()` / `is_configured()` .)
4. The turn is appended to the agent's channel file and to the token-usage log; nothing is rewritten.

4.2 The action path (tool use)

1. An agent (or the operator) requests a named action via `/api/run?action=...` .
2. The **allow-listed action runner** looks the name up in a closed `ACTIONS` dictionary. A **known** name runs its small Python function; an **unknown** name returns `{allowed: [...]}` — the list of permitted actions — and executes nothing. This is the deliberate substitute for a raw shell. (**VERIFIED** — `ACTIONS = {...}` , `fn = ACTIONS.get(name)` .) (capabilities.md §3)

4.3 The gated path (high-risk action)

1. A proposed action carries an autonomy level (L0–L5) and a computed risk score (4–20).
2. The **gate** (`autonomy.py`) composes the two into one verdict — `act` / `report` / `queue` .
3. If the verdict is `queue` , the action is parked in the append-only `gate-queue.jsonl` and **does not execute**. The operator approves or rejects from the fast-verify queue; the decision is a separate append line. (**VERIFIED** — `verify_queue.py` .) The gate model is detailed in §5.

4.4 The workforce path (the scoping caveat)

The repo exposes the **orchestration surface and data shapes** — per-agent channels (append-only turn logs), an org structure, a grid view, and per-agent autonomy — each read fail-closed from the seed data directory (`agents.json` , per-agent channel files, a service/channel index). The React layer renders these as the agent grid, org chart, and process-table overlay. (**VERIFIED** — `servari_server.py` handlers; `docs/ARCHITECTURE.md` components.) (capabilities.md §4)

What the published repo does **not** include is the concurrent execution engine that fans real work out across many model-backed workers and merges their results. The shipped artifact is the controllable workforce *shell* over BYOM, populated from synthetic seed data and ready to be pointed at real backends — not a running fan-out-and-merge engine. (**PARTIAL / UNVERIFIED** — surface VERIFIED; concurrent engine not in public sources.) (capabilities.md §4; _DIGEST.md §8 limit 3)

4.5 Reliability properties along every path

Every route degrades gracefully: a missing module or data file returns a clean "unavailable" payload at **HTTP 200** rather than crashing, and the health surface is fail-closed — any unreadable sub-check degrades to `UNKNOWN` and the whole call still returns. Configuration is environment-driven (host, port, home directory, Python path, a no-voice flag) with **no hardcoded absolute paths**; home resolution is drive-independent (an env var → repo root → current working directory). (**VERIFIED** — `servari_server.py` , `health.py` , `AGENTS.md` , each module's `_home()` helper.) (capabilities.md §10)

---

5. The gate model

The gate is the SUR's central architectural stance, so it is described here in its own section. The stance, stated plainly: **autonomy is a dial, and irreversible actions always pass a human gate.** (**VERIFIED** — `autonomy.py` + `verify_queue.py` docstrings; `AGENTS.md` "do not weaken the gates".) (capabilities.md §5)

5.1 The two inputs

- **Autonomy level (L0–L5).** Each agent has a level from **L0 (suggest only)** to **L5 (full auto)**. The dial widens what an agent may do *silently*. The level→band mapping is **monotonic** — a higher level yields a wider `act` band. (**VERIFIED** — the `LEVELS` table and band constants in `autonomy.py`.)
- **Risk score (4–20).** A proposed action is scored on four factors — reversibility × blast-radius × data-loss × certainty. The higher the score, the more consequential the action. (**VERIFIED** — risk model in `autonomy.py`.)

5.2 The composition rule and the hard floor

The two inputs compose into one verdict: `act` (do it silently), `report` (do it but surface it), or `queue` (hold for human approval). The load-bearing property is the **hard floor**: **even at L5 — maximum autonomy — a high-risk score still produces `queue`, not `act`.** High-consequence operations (deploy, real-send, spend, publish, merge, secret access) cannot be auto-crossed at any autonomy level. (**VERIFIED** — band logic in `autonomy.py`; the same rule is restated in plain language in the agent operating persona file.) (capabilities.md §5)

5.3 The queue as an append-only audit

`verify_queue.py` is the queue itself. A gated action is enqueued as an append line to `gate-queue.jsonl`; the operator's decision (approve / reject) is a **separate** append line; nothing executes until approved; the audit is never rewritten. Because the queue is append-only and the level→verdict logic is a small, readable function, the safety property is **auditable rather than asserted** — a reviewer can read the entire gate model in two short modules. (**VERIFIED** — `verify_queue.py`: `enqueue` / `list_pending` / `decide` / `history`.) (capabilities.md §5, §8)

5.4 Defense in two layers

The gate is enforced in **two complementary places**: the server-side band logic (the hard layer, which an agent cannot talk its way past) and the agent's operating persona file (a soft layer that restates the same rule for the terminal session). The soft layer complements but does not replace the hard layer — a useful distinction for an auditor, because prompt-level rules alone would not be a credible safety claim, whereas a small server-side function that mechanically refuses to cross the floor is. (**VERIFIED** — server logic + persona file both present.) (capabilities.md §5, §6)

5.5 Mechanical safety hooks (enforcement by construction)

Beyond the gate, safety is enforced by structure rather than convention: the action runner is allow-listed rather than a raw shell (§4.2), high-risk operations are intercepted before they execute (§5.2–§5.3), the retention loop byte-exactly reverts degrading changes (§6.3), and the repository's own contributor rules forbid weakening any of these (`AGENTS.md` "things to avoid": no raw shell, no weakening the gates, no hardcoded paths, no hardcoded secrets). In an internal health pass, **8/8** mechanical hooks were present, parsing, and running — and one "firing-but-blind" hook was caught and fixed in the same pass, an honest self-catch. (Hooks/runner present: **VERIFIED**; the 8/8 self-catch is a **PARTIAL** internal-log datum.) (capabilities.md §6; _DIGEST.md §3.3)

> **Why the gate model is the spine of the audit.** The category's cautionary tale is precisely the failure this layer targets: the most-adopted open-source incumbent (OpenClaw) carries a documented one-click RCE (CVE-2026-25253, CVSS 8.8), command-injection and prompt-injection CVEs, a companion-service breach exposing ~1.5M auth tokens, and a "shadow superuser" demonstration. The published academic remedy for that class of failure is a **staged human-in-the-loop defense layer** — which is exactly the SUR's claimed differentiator. The audit therefore weights the gate model heavily, while noting that a gate that *bites in the published code* is the verifiable claim, not "the system is safe" in the absolute. (Events **VERIFIED**; exact single-paper defense percentages **PARTIAL** — openclaw.md; researchers.md; _DIGEST.md §4.1)

---

6. The memory model

The SUR's memory stance is "**files are the memory**": all state lives in plain files so the system boots into a known state, resumes where it left off, and survives a restart with no database to stand up. (**VERIFIED** — public overview; the seed data directory of JSON / JSONL / Markdown files.) (capabilities.md §7)

6.1 What is stored, and in what shape

State files under the seed data directory (or an operator-pointed home) hold the agent channels, autonomy levels, the gate-queue audit, the token-usage log, retention runs, context / active-work markers, and human-readable memory notes and decisions. Concretely: `autonomy-levels.json`, `gate-queue.jsonl`, `token-history.jsonl`, a `retention/` directory, a `context/` directory, `active-work.json`, `memory/decisions.md`, and `memory/notes.md`. (**VERIFIED** — seed directory listing.) (capabilities.md §7)

6.2 The curated, non-glob memory surface

The UI does not scan the disk. The memory surface (`memory_surface.py`) reads **only the exact files a curator declares in a registry** — it does **not** glob or walk directories — and it surfaces a missing declared file honestly as `updated: "missing"` rather than fabricating freshness. This is a deliberate **least-privilege** choice: the interface shows only what a human declared, never an arbitrary disk scan. For an auditor, it means the memory surface is a finite, declared set, not an open window onto the filesystem. (**VERIFIED** — `memory_surface.py` docstring.) (capabilities.md §7)

6.3 The quality-gated retention loop (regression guard)

The retention loop is how an improvement loop can run unattended **without quality silently eroding**. `retention.py` runs a three-step flow:

1. `baseline(targets)` takes byte copies of the target files and records baseline metric scores.
2. The loop — a job, a scheduler, or a human — edits the targets.
3. `decide(run_id)` re-runs the **same** metric suite and compares: **KEEP** if no gating metric degraded (ties allowed, improvement welcome), or **REVERT** — restore the snapshot bytes exactly, **sha256-verified** — if any gating metric degraded.

It is **fail-closed**: if any metric fails to *run* at decide-time, it reverts; a run can be decided only once; every decision is appended to an audit. Metrics are defined in a registry with `exit_code` or `regex` parse modes. (**VERIFIED** — `retention.py` docstring invariants.) (capabilities.md §9)

The retention loop is the quality-regression guard that the public repo actually ships. The wider platform also describes a separate **failure-pattern regression memory** (recording known failure signatures so a recurrence is recognised and fixed instantly); that named subsystem is **not present as a distinct module in the public repo**. What is shipped and verifiable is the metric-gated KEEP/REVERT loop above. (Retention loop **VERIFIED**; failure-signature memory **UNVERIFIED in repo**.) (capabilities.md §9; _DIGEST.md §2 row 9)

6.4 The audit surface and the token meter

Two subsystems make the SUR's activity reviewable — the precondition for an external audit like this one. The **gate queue** (§5.3) and the **retention loop** (§6.3) both write append-only audit lines that are never rewritten, one JSON object per event. The **token meter** (`tokens.py`) reads a JSONL usage log, prices it at configurable per-million rates, and produces per-session / per-day / all-time reports; it is **read-only** against the source and writes only reports. (**VERIFIED** — `tokens.py` docstring.) (capabilities.md §8)

A cost-honesty note belongs in the architecture description because it is a property of the meter itself: the meter's own docstring is explicit that **the token counts are ground truth (logged per message) while the dollar figure is the API-equivalent at the configured rates** — i.e. **cost-to-serve, not cash spent**, since a subscription-billed operator pays differently. Any dollar figure derived from this meter must be read as **API-equivalent cost-to-serve**, with the assumed rates stated. (**VERIFIED** — `tokens.py` docstring; the published repo

ships only synthetic seed usage, so this architecture document records no specific dollar figure here — a real figure requires the operator's real log and rate assumptions.) (capabilities.md §8; _DIGEST.md §3.1)

---

7. How the SUR's components were obtained (consumption, in brief)

A full treatment is in the companion consumption audit; this section records only the architecturally-relevant fact. Several of the SUR's owned layers were built by **consuming** patterns from permissively-licensed open-source projects and **re-implementing them in the SUR's own stdlib-Python stack** — not by shipping any third-party code. The structural signature is visible on disk: the consumed source sits as segregated reference copies in per-intake directories (the vendors' original TypeScript / JavaScript / Markdown), while the SUR's running scripts are separate Python files that **do not import the references**. (**VERIFIED** structurally — different language, segregated location, no import linkage.) (consumption.md §3, §4)

The intake is governed by a four-question gate (license check → value ÷ overhead ≥ 3× → framework-conflict check → paid-service-drift check) and a six-step pattern that ends in a **real test** plus human review. The consumed patterns include a context-discipline family, a read-before-edit safety invariant, symbol-aware edits (via Python's stdlib AST), a post-edit failure classifier and a syntax-gate (the ACI pattern from **SWE-agent**, MIT), trajectory evaluation (from **agentevals**, MIT), permission-gating / phase-locking and a dangerous-command pre-execution gate, and exploratory-QA + email-handling patterns (from OpenClaw + Hermes, **pattern-only, no vendor code**). The multi-provider routing adapter and the model-hub adapter are present and documented as interface-conforming wrappers, with the live paid-provider call **not** executed in this audit. (**VERIFIED** for the pattern-only re-implementations; **PARTIAL** for the paid-provider adapter — mechanism present, live call not run.) (consumption.md §4; _DIGEST.md §7)

The architecturally-relevant consequence: the components in §3 are an **owned composition**. The SUR consumes the part of the stack the category agrees is commodity (harness-level patterns, edit-safety, context-discipline, trajectory-eval) and reserves its composition — the gate layer, the audit boundaries, the retention guard, and the private build-and-verify method — for the part that is potentially defensible. (**PARTIAL** — the layers are present and inspectable; "constitutes a durable defensibility" is a forward-looking business judgement the auditor describes but does not certify.) (consumption.md §6)

---

8. The architecture in one diagram (described)

A reader can hold the whole system in mind as four concentric concerns:

1. **The operator's model** sits outside the SUR entirely (BYOM) — the SUR never owns or hides the intelligence.
2. **The BYOM router** is the single, small (~216-line) doorway between the operator's model and the shell, carrying one disclosed system line and nothing hidden.
3. **The control plane** — the allow-listed action runner, the autonomy × risk gate with its hard floor, the mechanical hooks — surrounds every action so the blast radius of an over-eager agent is bounded by a hand-curated list and a queue that cannot be auto-crossed.
4. **The record plane** — file-based memory, the curated non-glob memory surface, the append-only gate and retention audits, the read-only token meter, the fail-closed health surface — makes everything the SUR did inspectable after the fact.

The spine (a stdlib HTTP server) wires all four together and degrades every route to a clean "unavailable" rather than failing. (**VERIFIED** — composition of §3–§6.) The single most important thing this picture says is **where the trust lives**: not in a hidden orchestration black box, but in a small, readable control plane and an append-only record plane that an outsider can read in full. (capabilities.md §4 "Why it matters", §10)

---

9. Auditor's architectural summary

Concern	What the published repo provides	Status
Model coupling	One small BYOM router; one disclosed system line; config-edit to swap providers; local model is first-class	**VERIFIED**
Execution surface	Allow-listed action runner (closed dict; unknown → returns allowed list; not a raw shell)	**VERIFIED**; general MCP layer **UNVERIFIED in repo**
Autonomy & gates	L0–L5 × 4–20 risk → act / report / queue; hard floor (L5 + high-risk still queues); append-only queue	**VERIFIED**
Memory	Plain files; curated non-glob surface; honest "missing"; drive-independent home	**VERIFIED**
Quality guard	Metric-gated KEEP/REVERT; byte-exact sha256-verified revert; fail-closed	**VERIFIED**; failure-signature memory **UNVERIFIED in repo**
Audit & cost	Append-only audits; read-only token meter; cost = API-equivalent, not cash	**VERIFIED**
Reliability	Stdlib-only; same-origin; per-module defensive load; HTTP-200 degrade; fail-closed health	**VERIFIED**
Concurrency engine	Orchestration *surface* + data shapes only	surface **VERIFIED**; execution engine **not in repo**
Sandbox-execute primitive	Context-pressure *policy* present	policy **VERIFIED**; distinct primitive **UNVERIFIED in repo**

Asymmetry note (stated once, plainly). This system is the work of one researcher with AI assistance, not a funded engineering team. Its strengths — a small, readable, stdlib-only, fail-closed surface with mechanical gates and append-only audits — are real and directly verifiable in the open repo. Its limits are equally honest: four wider-platform capabilities (a distinct sandbox-execute primitive, a general MCP tool layer, a concurrent multi-agent execution engine, a failure-signature regression memory) are **not present in the published repository** and are catalogued here as roadmap, not shipped. Competitor maturity is a roadmap to learn from, not a deficit to spin. (capabilities.md "Asymmetry note"; _DIGEST.md §8)

The accurate one-line characterisation of the architecture: a **verifiable, gate-first, bring-your-own-model agentic operating-system shell** — local-first, file-backed, mechanically gated, and append-only auditable — built from an owned stdlib composition that consumes commodity patterns and reserves its defensibility for the control and record planes. Nothing in this description supports "beats frontier," "fully sovereign," or any unbacked superlative, and none is claimed.

Sources

- The SUR's published open-source repository (Apache-2.0): `README.md`, `AGENTS.md`, `docs/ARCHITECTURE.md`, `config.example.json`, the server module docstrings + route table, and the seed data directory — as inventoried in capabilities.md.
- The consumption / consumption evidence (intake gate, segregated reference copies, per-item tests): consumption.md.
- The sealed research synthesis (capability table, honesty rails, the cautionary-tale comparison, cost labelling): _DIGEST.md, drawing on metrics.md, openclaw.md, and researchers.md.
- External / open-source pattern sources named and corroborated as real public projects: agentevals (MIT, trajectory evaluation); SWE-agent (MIT, ACI edit-safety pattern); the Pi.dev coding-agent harness ecosystem

(context engine, code-intelligence modules, permission-gate cluster); OpenClaw and Hermes (QA + email workflow patterns); Hugging Face public model-hub APIs. The LLM-as-operating-system / “Software 3.0” paradigm (Andrej Karpathy), per researchers.md.

Consumption Audit — What Was Consumed

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

Scope and method

This document audits one question: **what did the system under review (SUR) take from the open-source ecosystem, and was the take licensed and honest?** It is the consumption ledger — the record of which patterns were consumed, from which projects, under which licenses, and what the SUR built from each.

A word on terminology, because it is load-bearing for this whole document. The category norm is to **ship a harness** — a runtime that other people install and run. The SUR does something different: it **consumes** open-source work. It reads a well-designed pattern, takes the *idea* one-way into its own architecture, re-expresses that idea in its own stack, and keeps the original only as a segregated reference for provenance. Throughout this audit, the word "harness" describes the *other* tools in the category for contrast; what the SUR does is **consume / consume**. That distinction is the differentiator, and it is the thing this document tests for honesty.

Every non-trivial claim below carries a label. **VERIFIED** means the auditor confirmed it by direct inspection of the SUR's own on-disk artifacts — its capability-definition files, the segregated reference copies of the consumed source, and its own findings ledger — and/or a primary external source. **PARTIAL** means the mechanism is present and the claim is well-supported but one verification step (a live call, a line-by-line diff, a legal review) was not performed in this pass. **UNVERIFIED** means a recommended artifact is not yet present.

One honest framing, stated once. The SUR is the work of one researcher with AI assistance, not a funded team. Several of the projects it consumes from are large communities or single high-output maintainers with paid tiers and continuous integration. The SUR's edge is *not* breadth — it cannot win reach head-on — it is the disciplined intake process and the private composition described here. That asymmetry is context, not a defect to spin. [*Sources: research digest §8; companion findings on ECC and OpenClaw.*]

---

1. The claim under audit, stated plainly

The SUR claims an unusual posture toward open source: **disciplined, one-way consumption**. It consumes patterns and capabilities from permissively-licensed projects, re-implements them inside its own owned framework, and does not publish its own method back out. The ecosystem is treated as a source to learn from, not a commons to contribute to.

This is a strong claim, and it cuts two ways for an auditor:

- **If true,** it is a coherent competitive-defensibility strategy: consume the commodity, keep the composition.
- **If overstated,** it is one of two failures — (a) *license laundering* (copying vendor code while claiming "pattern-only"), or (b) a defensibility *thinner than advertised* (the consumed patterns are the whole story and there is no private layer).

This audit tests both edges.

Bottom line up front. The consumption story holds up on inspection. The SUR keeps reference copies of the consumed source in clearly-segregated intake directories; its own executable scripts are independent re-implementations (Python standard library, not the vendors' TypeScript/JavaScript); the licenses involved are permissive; and the consumed-versus-built boundary is documented per item. The honest limits — a clean-room

diff not done, one source under a non-OSI license, a public attributions file owed before release — are named in §7. [VERIFIED structurally — directory layout, file-type, and findings-ledger inspection; full clean-room diff PARTIAL.]

---

2. The consumption discipline (how the take is governed)

The SUR runs every consumption through a documented **four-question intake gate** before engaging an external source. The gate text is present in the system's own capability-definition for the consumption discipline, and the findings ledger shows it applied per item. [VERIFIED — gate text present in the SUR's definition + applied per-item in the ledger.]

#	Gate question	What it protects
1	**License check.** Permissive (MIT / Apache / BSD / ISC) → consume. Copyleft (GPL / AGPL) → reject for a commercial product. Weak-copyleft (MPL / LGPL) → consume only with file-isolation.	Keeps the take legal — this is the gate that matters most.
2	**Value ÷ overhead ≥ 3×.**	Avoids pulling a large dependency for one function.
3	**Framework-conflict check.** Reject anything that would force an external state system, a cloud dependency, a build-system mandate, or global-state mutation onto the owned framework.	Keeps the framework owned, not bent to match a vendor.
4	**Paid-service-drift check.** If a paid API would otherwise be the default, reframe to consume an open equivalent first; paid is a fallback, not a first move.	Keeps the system self-reliant.

The gate is paired with a **six-step intake pattern**:

> discover → gate → re-implement as an adapter in the owned stack → **run a real test** → pin the reference copy in a segregated intake directory → human review before the new capability joins the permanent roster.

The "run a real test" step is the SUR's standing proof discipline: a consumed pattern is not considered live until a runnable check passes against the re-implementation. The findings record a per-item smoke-test result for each built capability (for example, the edit-safety scripts each carry a recorded 4/4 or 8/8 pass). [VERIFIED — six-step pattern documented; per-item test results recorded in the findings.]

Auditor reading. This is a genuinely disciplined intake process — materially stricter than the *"install a skill, hope it's clean"* pattern that dominates the category. The supply-chain risk that defines the loudest incumbent (a third-party skill that performed data exfiltration and prompt injection without user awareness was found in OpenClaw's registry) is precisely the risk a license-and-test gate is designed to prevent. [VERIFIED — OpenClaw third-party malicious-skill incident documented; *source: openclaw.md §5, citing adversa.ai and Wikipedia.*] The license gate (question 1) is the correct gate, and it is **applied, not merely declared**.

---

3. The consumed-versus-built boundary (the central evidence)

For any *"pattern-only, no vendor code copied"* claim, one auditor question dominates: **where does the vendor's code physically live, and does the SUR's own running code depend on it?** The answer determines whether this is honest consumption or quiet copying.

What inspection found: [VERIFIED — directory structure + file-type inspection on disk.]

- **The consumed source is kept as reference copies** under per-intake directories named for the intake event (e.g. `…-batch2/`, `…-batch3/`). These hold the vendors' original TypeScript / JavaScript / Markdown,

unmodified.
- **The SUR's own executing artifacts** — the scripts that actually run as part of the system — are **separate files written in Python (standard library only)**. They sit in each capability's own `scripts/` directory, not in the intake directory.
- **The two are not wired together.** The running scripts do not import or invoke the reference TypeScript. The reference copy is documentation-of-provenance; the re-implementation is the thing that runs.

This is the structural signature of pattern-consumption done correctly: keep the source you learned from for audit and provenance, but re-express the *idea* in your own stack so your running system carries no foreign code. It is the difference between *"we read their read-before-edit logic and wrote our own in Python"* and *"we shipped their file."* Inspection supports the former. [VERIFIED.]

> **The differentiator, made concrete.** A harness-shipping project distributes the vendor's code so others run it. A *consuming* project distributes none of it — it runs its own re-implementation and keeps the original only to prove provenance. The two different languages (the SUR's Python vs the vendors' TypeScript/JavaScript), the two different locations (owned `scripts/` vs segregated intake dir), and the absence of any import linkage are the three independent signatures that this is consumption, not harnessing.

Honest caveat — the one license that is weaker. Keeping a full reference copy of permissively-licensed source on disk is fine for MIT/Apache (attribution-with-notice). For a source under a non-OSI *fair-use / source-available* license, "we kept it for reference and re-implemented the pattern" is a defensible posture but a thinner legal footing than MIT. At least one consumed source — the context engine of a multi-model coding-agent harness (the Pi.dev ecosystem) — is source-available rather than classic-OSI. The SUR's own notes flag this and scope that consumption to pattern-only with the vendor files reference-only. The reviewer concurs that pattern-only is the correct posture there, while noting it is the one intake where the license is "fair-use," not "permissive." [PARTIAL — posture is correct; the underlying license is weaker than the MIT cases and warrants a lawyer's eye before any redistribution of the reference copy.]

---

4. What was consumed — the ledger

Below is the catalogue of consumed patterns, in generic engineering vocabulary. Each entry names the **pattern**, the **kind of source** it came from, what the SUR **built** from it, and the **license posture**. Project names appear only where they are public open-source projects (allowed); internal capability names are generalized.

4.1 Context-discipline (the largest single consumption)

- **Source kind:** the context engine of an open-source multi-model coding-agent harness (Pi.dev ecosystem).
- **Built:** a context-discipline sandbox primitive — *the analysis script runs in a sandbox; only the printed findings enter the working context, never the raw data* — plus a unified internal-memory query, a cross-source federated query, and a roster-health audit discipline. **Four distinct capabilities mapped from one source's design.**
- **License:** source-available / fair-use; consumed **pattern-only**, vendor files reference-only.
- [VERIFIED — four target capabilities present on disk with their own Python scripts; the context engine's reference copy present in the intake tree; the findings ledger records the one-source → four-pattern mapping and the license note.]

This is the largest single consumption: one well-designed open context engine yielded four owned capabilities. The discipline it teaches — *write the script, let only the summary into context, keep raw data out* — is exactly the *"everyone needs this, almost nobody builds it"* pattern that rewards consumption. It is also independently science-backed: published context-engineering research shows context editing alone gave a measured +29% lift, rising to +39% paired with a memory tool, and cut tokens ~84% in a 100-turn evaluation, with degradation universal across all major models. [VERIFIED — context-engineering figures; *source: research digest §5, citing Anthropic.*]

4.2 Read-before-edit safety

- **Source kind:** a code-intelligence client ("lens" module) from the same harness ecosystem.
- **Built:** an invariant that any edit to an existing file must be preceded by a read of that file in the current work session, with a guard that catches edits whose target text was drawn from memory rather than from a recent read.
- **License:** consumed pattern-only; reference TypeScript present, re-implemented as a standard-library Python checker.
- [VERIFIED — target capability + its Python checker present; the reference `edit-safety client` present in the intake tree, not imported by the running code.]

4.3 Symbol-aware edits

- **Source kind:** a read-expansion module from the same lens.
- **Built:** an edit-scope discipline that expands an edit to the full boundary of the named symbol (function / class / method, decorators included) before committing, so edits don't land mid-statement or cut a decorator. The re-implementation uses Python's standard-library AST parser.
- **License:** pattern-only; reference TypeScript present.
- [VERIFIED — capability + AST-based Python script present; smoke-tested boundary detection recorded in the findings.]

4.4 Post-edit failure diagnostics (ACI-style edit safety)

- **Source kind:** the structured diagnostic loggers of the same lens, *and* — for a sibling syntax-gating capability — the agent-computer-interface (ACI) edit-safety pattern from an open MIT research agent, **SWE-agent**.
- **Built:** (a) a seven-class failure classifier that diagnoses why an edit failed and routes to the right fix before retrying, and (b) a separate gate that *checks syntax before the edit lands and auto-reverts if the edit introduces a new syntax error*.
- **License:** the ACI pattern source is **MIT**; consumed pattern-only.
- [VERIFIED — classifier capability + its Python script present (8-branch smoke-test recorded); the syntax-gate capability present with a recorded 4/4 smoke-test; the SUR's notes credit SWE-agent ACI (MIT) as the pattern source for the syntax-gate.]

> **Composition is the point.** Taken together, 4.2–4.4 form a coherent **edit-safety stack** assembled from consumed patterns: read-before-edit (prevention) → symbol-boundary-aware edit (prevention) → syntax-gate (pre-commit prevention) → failure classifier (post-failure recovery). No vendor code runs; the *ideas* were consumed and re-built into one owned layer. This is consumption producing something **more coherent than any single source** — four sources composed into one stack. [VERIFIED — all four capabilities present and interlinked in the definitions.]

4.5 Trajectory evaluation

- **Source kind:** an open MIT evaluation library, **agentevals**.
- **Built:** a grader that scores an agent's *tool-call path* (the trajectory it took), not just its final output, with strict / unordered / subset match modes. Re-implemented in the Python standard library.
- **License: MIT**; consumed pattern-only.
- [VERIFIED — capability + standard-library scorer present, 5/5 smoke-test recorded; the SUR's notes credit agentevals (MIT) as the pattern source.]

4.6 Permission-gating / phase-locking

- **Source kind:** a permission-gate extension cluster from the coding-agent harness ecosystem (permission-gate + safe-git + protected-paths shape).
- **Built:** a structured phase-locking discipline (each phase has explicit entry conditions, exit criteria, and a checkpoint artifact; phase N+1 verifies phase N's artifact before opening), plus a mechanized pre-execution

gate that blocks dangerous version-control commands before they run.
- **License:** consumed pattern-only.
- [VERIFIED — phase-lock discipline present; the dangerous-command gate present with recorded bite-tests; the SUR's notes name the permission-gate cluster as the pattern source.]

4.7 Exploratory-QA and email-handling patterns

- **Source kind:** documented workflows from two large open agent projects — a browser-driven QA routine and an IMAP/SMTP email tool.
- **Built:** a browser-driven exploratory-QA discipline (drive a real browser, check the console after every navigation, capture evidence) and a canonical email capability that wraps an official mail API for read/draft (no auto-send).
- **License:** the source projects are **MIT**; consumed pattern-only — explicitly *not* the email tool's code, which the SUR notes it did not install.
- [VERIFIED — both capabilities present; the SUR's notes state "pattern only, no vendor code" for both and name the source projects.]

4.8 Multi-provider routing and the model-hub adapter

- **Source kind:** an open-source multi-model coding-agent CLI (for cost-tier routing across many providers, the Pi.dev routing CLI) and the public model-hub APIs (Hugging Face).
- **Built:** adapters that let the SUR dispatch work across many model providers and pull open weights / datasets / model metadata — each wrapped so **the external tool conforms to the SUR's interface, not the reverse.**
- **License:** the routing CLI is open-source; the hub access is via public APIs under the hub's terms.
- [PARTIAL — the adapters are present and documented as interface-conforming wrappers; the reviewer confirmed the adapter definitions and the reference copies on disk, but did not execute a live paid-provider call as part of this audit, consistent with the SUR's own "mechanism-proven, live-call-not-captured" honesty elsewhere in the pack.]

Ledger at a glance

§	Pattern (generic)	Source kind	License posture	Built	Status
4.1	Context-discipline sandbox + 3 memory/audit capabilities	Coding-agent context engine (Pi.dev)	Source-available / fair-use; pattern-only	4 owned Python capabilities	VERIFIED
4.2	Read-before-edit safety	Code-intelligence "lens"	Pattern-only	stdlib Python checker	VERIFIED
4.3	Symbol-aware edits	Read-expansion "lens"	Pattern-only	stdlib AST script	VERIFIED
4.4	Post-edit failure diagnostics + syntax-gate	Diagnostic loggers + SWE-agent ACI	**MIT** (ACI); pattern-only	7-class classifier + revert gate	VERIFIED
4.5	Trajectory evaluation	agentevals	**MIT**; pattern-only	stdlib trajectory scorer	VERIFIED
4.6	Permission-gating / phase-locking	permission-gate cluster	Pattern-only	phase-lock + dangerous-cmd gate	VERIFIED
4.7	Exploratory-QA + email patterns	Two large open agent projects	**MIT**; pattern-only	browser-QA +	VERIFIED

§	Pattern (generic)	Source kind	License posture	Built	Status
				read/draft email	
4.8	Multi-provider routing + model-hub	Pi.dev CLI + Hugging Face APIs	Open / public-API terms	interface-conforming adapters	PARTIAL

---

5. The direction of flow (the one-way claim, corrected)

The SUR's stated rule is **inbound only**: open source → SUR (consume, yes); SUR → open source (publish the method, no). The reviewer tested whether the system's behaviour matches the rule.

- **Inbound is real and frequent.** §4 documents at least eight distinct consumptions across multiple permissive/open sources, each with a built artifact and a recorded test. [VERIFIED.]
- **Outbound is deliberately constrained — but not absolute.** The same project maintains a separate, audited open-source release plan (the `servari-open` release) that publishes the *shell*, the *interfaces*, the *memory format*, the *safety-gate model*, and runnable proofs — while keeping the orchestration intelligence, the live block-hook implementations, the business workflows, and the private method prompts unpublished. [VERIFIED — the release plan draws this line item by item.]

Auditor correction to the headline. "One-way" is the right *spirit* but slightly imprecise as stated. The precise, defensible claim is:

> **Consume freely; publish a verifiable shell + interfaces; never publish the method.**

That is asymmetric, not literally zero-outbound, and it is the posture the release plan actually encodes. An auditor should describe it that way — *not* as "takes everything, gives nothing." [PARTIAL on the literal "one-way" phrasing; VERIFIED on the underlying asymmetric posture.]

A fairness point, stated plainly. A project that *consumes* MIT/Apache work and does *not* contribute back is fully within its license rights — permissive licenses permit exactly this. It is legal. Whether it is *neighbourly* to the open-source commons is a values question, not a license question, and the SUR's own materials are candid that the strategy prioritizes competitive defensibility over reciprocity. The reviewer notes this as an honest disclosure rather than a defect. [VERIFIED — disclosure present; "fairness" is a values judgment, flagged as such.]

---

6. Why this matters for the comparison (the defensibility question)

The cross-project research in this audit reaches a firm conclusion: **the agent harness and the open `SKILL.md` capability format are now commodities.**

- The open `SKILL.md` standard is Anthropic-authored (published 2025-12-18); within ~48h Microsoft (VS Code) and OpenAI (ChatGPT + Codex CLI) adopted it, and by 2026-03 ~32 tools read the same `SKILL.md` from the same directory structure. [VERIFIED — *source: openclaw.md §4, citing agentskills.io and The New Stack.*]
- The two dominant open incumbents (OpenClaw, ~247K stars / ~47.7K forks at a 2026-03-02 snapshot; Hermes, ~175K stars) plus the cross-harness bundle ECC (205,179 stars on a live 2026-06-03 API read) all converge on this same format. [VERIFIED — *sources: openclaw.md §2, ecc.md §2.*] ECC is explicit that it is *not a model and not a harness of its own* — it is a config/skill/agent/hook bundle that installs **onto** existing harnesses. [VERIFIED — *source: ecc.md §1.*]

That conclusion is **load-bearing for the consumption story**: if the harness is commodity, then consuming harness-level patterns from the ecosystem is not a weakness or an admission — it is the rational move, and it costs the SUR nothing defensible. The SUR's competitive defensibility, if it exists, must live in the layers the commodity harnesses conspicuously *lack by default* — a deep human-in-the-loop gate layer, a failure-pattern regression

memory, sealed/audited output boundaries, and a private iterative build-and-verify method. The consumption audit confirms the SUR **built those layers from consumed patterns plus its own composition**, and keeps the composition private. [PARTIAL — the layers are present and inspectable in the definitions; "constitutes a durable defensibility" is a forward-looking business judgment the reviewer does not certify, only describes.]

There is a sharp symmetry worth quoting. The most-cited academic study of the category's security (the OpenClaw security paper) found baseline sandbox-escape defense averaged only ~17%, and that **adding a staged human-in-the-loop defense layer** raised overall defense to 19%–92% (Claude ~91.5%). That remedy — a staged HITL gate — is *precisely* the SUR's claimed differentiator. The thing the loudest incumbent had to bolt on after a string of CVEs and a ~1.5M-token data breach is the thing the SUR consumed and built in from the start. [VERIFIED — defense-rate figures from a single paper (treat exact percentages as one study's result); *source: openclaw.md §5, citing arXiv 2603.10387.*]

The contrast that is *not* a differentiator. The single-maintainer / bus-factor risk the SUR carries (one builder + AI assistance) is the *same* asymmetry ECC carries — its lead maintainer has 1,426 commits versus 47 for the #2 human, a ~30:1 ratio, with two bots in the top four contributors. [VERIFIED — *source: ecc.md §3, live contributors API.*] So solo-built is not a point of contrast between them. The real contrast is the opposite *strategy*: ECC is open / broad / distributed / viral, **built to be consumed and published**; the SUR is sealed / deep / method-as-defense, **built to consume and withhold.** ECC is ahead on reach and a shipped paid go-to-market ($19/seat/month ECC Pro); that is the SUR's distribution gap, not an engineering deficit. [VERIFIED — *source: ecc.md §7, §10.*]

Auditor's net. The consumption strategy is coherent. The SUR consumes the part of the stack everyone agrees is commodity (harness patterns, edit-safety, context-discipline, trajectory-eval) and reserves its defensibility for the part it composed and gated. That is the correct place to spend a defensibility. Whether the defensibility is *deep enough* against well-funded competitors is unresolved and out of scope here.

---

7. Honest gaps and auditor cautions

- **"No vendor code copied" is verified structurally, not line-by-line.** The reviewer confirmed that the running scripts are independent Python and that vendor files sit segregated in the intake tree. The reviewer did **not** perform a token-level diff of every SUR script against every reference file to prove zero incidental copying of, say, a constant table or a regex. The structural evidence is strong (different language, different location, no import linkage) but a full clean-room diff would raise this from "strongly supported" to "proven." [PARTIAL.]
- **One source is fair-use, not classic-OSI.** §3 flags it. Pattern-only consumption is the right posture, but redistributing the reference copy of a source-available project warrants a legal review before any public release of the intake tree. [PARTIAL — flagged, not resolved.]
- **Attribution discipline should be made external-facing.** Internally the SUR credits each pattern's source (SWE-agent ACI / MIT, agentevals / MIT, the named harness ecosystems). For a public release, those attributions and the corresponding license notices should appear in the shipped repo's NOTICE / attributions file, not only in private findings. The reviewer did not confirm a public attributions file exists yet. [UNVERIFIED — recommend adding before publish.]
- **Test coverage of consumptions is real but per-item, not aggregate.** Each consumed capability carries its own recorded smoke-test (4/4, 5/5, 8/8 across the edit-safety and eval items). There is no single aggregate "all consumptions still pass" regression run captured in this pack; the per-item proofs are convincing, but a bundled regression of the consumed layer would strengthen the claim that the patterns stay healthy over time. [PARTIAL.]
- **The author asymmetry, restated for honesty.** This system was built by one person with AI assistance, not a funded team. The projects it learns from are, in several cases, large communities or single high-output maintainers with paid tiers and CI. The SUR's edge is not breadth or reach (it cannot win those head-on); it is the disciplined consumption process and the private composition. Any comparison should weight this asymmetry as a roadmap to learn from, not a deficit to spin. [VERIFIED — asymmetry is factual; the framing is the reviewer's.]

8. Verdict

The consumption story **survives audit at the stated, corrected scope.**

- **The take is licensed.** A license gate runs before intake; the consumed sources are permissive (MIT / Apache) with one source-available exception correctly scoped to pattern-only. [VERIFIED, with the §7 fair-use caveat PARTIAL.]
- **The take is pattern-only, structurally.** Vendor source is kept as segregated reference; the running code is independent standard-library Python that does not import it. [VERIFIED structurally; full clean-room diff PARTIAL.]
- **The consumed-versus-built boundary is documented per item,** with a recorded test for each built capability. [VERIFIED.]
- **The framework stays owned.** External tools are wrapped to conform to the SUR's interface; the framework is not bent to match them. [VERIFIED in the definitions.]
- **The flow is asymmetric, not literally one-way.** Consume freely; publish a verifiable shell + interfaces; never publish the method. That is the posture the release plan encodes, and it is honest about prioritizing defensibility over reciprocity. [VERIFIED.]

The honest weak points are narrow and named: a token-level clean-room diff is not done, one source's license is fair-use rather than OSI, and a public attribution/notice file is owed before release. **None of these is a capability or integrity gap; all are pre-publish housekeeping.**

The consumption strategy itself is coherent, disciplined, and — given that the harness layer is a proven commodity — the rational way to build. The differentiator is real and correctly stated: where the category norm is to *ship a harness* others install, this platform **consumes open source one-way into its own architecture**, runs its own re-implementation, and keeps the original only as provenance. The credible claim is structural: a disciplined consumer that re-builds commodity patterns into an owned, gated composition. Nothing in this audit supports "beats frontier," "fully sovereign," or any unbacked superlative — and none is claimed.

Sources

- **The SUR's own on-disk artifacts inspected for this audit:** the consumption-discipline definition (the four-question gate + six-step pattern); the per-capability definitions and their standard-library Python scripts (edit-safety stack, trajectory-eval, context-discipline family, phase-lock, dangerous-command gate, QA + email patterns); the segregated per-intake reference-copy directories; the system's findings ledger recording per-item license gates, source attributions, and smoke-test results; the audited open-source release plan.
- **External / open-source pattern sources named by the SUR and corroborated as real public projects:** agentevals (MIT, trajectory evaluation); SWE-agent (MIT, ACI edit-safety pattern); the Pi.dev coding-agent harness ecosystem (context engine, lens/code-intelligence modules, permission-gate cluster — open-source / source-available); large open agent projects (OpenClaw, Hermes) for the QA and email workflow patterns; Hugging Face public model-hub APIs.
- **Companion audit findings cross-referenced** for the "harness is commodity" conclusion and competitor context:
  - OpenClaw findings (this review): MIT TypeScript/Swift; ~247K stars / ~47.7K forks at a 2026-03-02 snapshot; the open `SKILL.md` standard's broad adoption; CVE-2026-25253 (CVSS 8.8 one-click RCE) and related CVEs; the ~1.5M-token companion-service breach; the academic HITL defense-rate study (arXiv 2603.10387). Primary sources cited inline in `openclaw.md`.
  - ECC findings (this review): live GitHub API 2026-06-03 — 205,179 stars / 31,492 forks / MIT; not-a-model-not-a-harness config bundle; 1,426-vs-47 single-maintainer commit ratio; $19/seat/month ECC Pro. Primary sources cited inline in `ecc.md`.
- **The open `SKILL.md` capability standard:** agentskills.io (Anthropic-authored, published 2025-12-18, broadly adopted), as documented in the companion OpenClaw findings.

Capabilities — The Capability Catalog of a Gate-First, BYOM Agentic Operating Layer

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

Purpose and scope of this document

This is the **capability catalog** for the system under review (SUR). Where the previous document described the SUR's architecture and the consumption audit described what it took from open source, this document answers a single question for each capability: **what does it do, how is it built, and why does it matter** — with every non-trivial claim labelled **VERIFIED**, **PARTIAL**, or **UNVERIFIED** so a reader can separate what was read in the published source from what is asserted in the wider platform documentation.

The evidence base is the SUR's **already-published, open-source repository** (Apache-2.0; `README.md`, `AGENTS.md`, `docs/ARCHITECTURE.md`, `config.example.json`, the server module docstrings and route table, and the seed data directory) plus the public architecture overview. (capabilities.md; _DIGEST.md §2)

A terminology note carried throughout, because it is load-bearing. The SUR's relationship to the open-source ecosystem is **consumption**: it consumes patterns one-way into its own owned framework and re-implements them in its own stack. It does not ship a third-party harness and it is not itself a harness. Where the word "harness" appears below it describes *other, generic* tools for contrast — the category norm is to ship a harness; the SUR consumes patterns and rebuilds them as its own. (consumption.md §1–§4)

How to read this catalog

Each of the ten capabilities below is described in three fixed subheads:

- **What it does** — the function, in plain operational terms.
- **How it is built** — the implementation at a high level, with the backing module(s) and routes named where the published code shows them.
- **Why it matters** — the engineering and trust significance, scoped honestly.

Two honesty rails span the whole catalog and are restated wherever they bite:

1. **Counts and surfaces are not the same as running engines.** Four capabilities the *wider* platform describes — a distinct context-discipline sandbox-execute primitive, a general Model Context Protocol (MCP) tool layer, a concurrent multi-agent *execution* engine, and a failure-signature regression memory — are **not present in the published repository**. They are catalogued here as roadmap / integration claims, never as shipped features. (capabilities.md "Auditor summary"; _DIGEST.md §2 honesty rail, §8)
2. **The defensible characterisation of the shipped artifact is a *verifiable, gate-first BYOM agent-OS shell*** — and on that scope it delivers what it claims. (_DIGEST.md §2, §9)

Catalog at a glance

#	Capability	What it is, in one line	Public-code status
1	BYOM model router	Routes chat to any operator-chosen model via the OpenAI-	**VERIFIED**

#	Capability	What it is, in one line	Public-code status
		compatible request shape	
2	Context-discipline sandbox	Treats the context window as managed RAM; measures pressure, verifies survival pins, checkpoints	**VERIFIED** (policy); distinct sandbox-execute primitive **UNVERIFIED in repo**
3	Tool integration	Allow-listed action runner — named, vetted actions only, never a raw shell	**VERIFIED**; general MCP layer **UNVERIFIED in repo**
4	Parallel agent orchestration	A controllable workforce surface — grid, org chart, channels, briefs, autonomy dials	surface **VERIFIED**; concurrent execution engine **not in repo**
5	Human-in-the-loop gate layer	Autonomy dial L0–L5 × risk score → act / report / queue; high-risk always queues	**VERIFIED**
6	Mechanical safety hooks	Safety enforced by construction (allow-list + hard gate + revert loop)	**VERIFIED**
7	File-based memory	All state in plain inspectable files; curated, non-glob memory surface	**VERIFIED**
8	Audit surface + token meter	Append-only audit lines; token tracker prices counts at configured rates	**VERIFIED**; cost = API-equivalent, not cash
9	Regression memory	Quality-gated retention loop — KEEP only if no metric degrades, else byte-exact revert	retention loop **VERIFIED**; failure-signature memory **UNVERIFIED in repo**
10	Local server + reliability	Stdlib-only, same-origin, every route degrades to clean "unavailable", fail-closed health	**VERIFIED**

(Source: capabilities.md "Auditor summary"; _DIGEST.md §2.)

---

1. BYOM model router

What it does

Routes the operator's chat conversation to *any* model the operator chooses, by speaking the OpenAI-compatible `/chat/completions` request shape that nearly every provider exposes. The operator points the SUR at a hosted provider (OpenAI, OpenRouter, Together, Fireworks) or a fully local server (Ollama, LM Studio, vLLM) by editing one local config file. The intelligence is brought by the operator — *bring-your-own-model* — and the SUR supplies only the operating surface around it. (**VERIFIED** — `chat_byom.py` docstring, `README.md` wiring table, `config.example.json`.) (capabilities.md §1)

How it is built

A single small (~216-line) standard-library module (`server/chat_byom.py`) reads a gitignored `config.json` (`provider` / `api_key` / `model` / `base_url`), builds the request from the visible channel history plus **one short neutral system line**, and calls the endpoint via `urllib` . There is no hidden prompt beyond that one disclosed system line — the

`SYSTEM_LINE` constant is the entire system prompt and is visible in source — and there is no vendor lock-in, because swapping models is a config edit and nothing else. Missing or malformed config degrades to an honest "configure your model" message (`ok=False`) and never crashes or fabricates a reply. The model key is transmitted only to the chosen provider. (**VERIFIED** — `chat_byom.py` head, `load_config()` , `is_configured()` ; the single-endpoint, single-provider transmission claim is **PARTIAL** — a design claim consistent with the single `urllib` call but not exhaustively traced.) (capabilities.md §1)

A caveat carried from the wider research: the SUR's *external-model connector for inter-agent dispatch* is a **mock-driven shell with no successful real call logged** ($0.00, zero tokens, only auth/rate-limit failures in the record). The chat-side BYOM router above is verified against a standard-shape stand-in; the broader provider connector is not. (**PARTIAL** — gpt-competitors.md Part 1; _DIGEST.md §4.4.)

Why it matters

Decoupling the shell from the model is the SUR's central commercial and trust stance: the operator's data and credentials stay where they put them, cost and provider are operator choices, and a fully local model is a first-class path rather than a fallback. The strategy is validated by the broader landscape — multiple incumbents (OpenClaw, Hermes) are likewise model-agnostic across Claude / GPT / Gemini / DeepSeek and local backends, and one research lab in the field ships open-weight models and pursues decentralized training, confirming that a run-your-own-model posture is a live, defensible direction rather than a niche. (**VERIFIED** for the incumbent landscape; **PARTIAL** for the open-weight lab's own scale/quality claims — gpt-competitors.md §2.1–§2.3; _DIGEST.md §4.2.) For an auditor, the decisive property is verifiability: the entire BYOM surface is one small, readable file. (capabilities.md §1)

---

2. Context-discipline sandbox (context-pressure policy)

What it does

Treats the model's context window as a managed resource — *"the context window IS the agent's RAM; a compaction is a swap to disk"* — and makes the eviction/survival policy first-class rather than implicit. It (a) **measures** context pressure from real signals, (b) **verifies a swap-file contract** (that everything which must survive a compaction is already on disk — the "survival pins"), and (c) **checkpoints** by repairing what it mechanically can and honestly reporting what only a human can fix. (**VERIFIED** — `context_policy.py` docstring; routes `/api/context` , `/api/context-checkpoint` .) (capabilities.md §2)

How it is built

`server/context_policy.py` (~31 KB, the largest server module) reads real signals — live transcript size and age, active-work staleness, and work-log entries newer than the last checkpoint — to compute a pressure verdict. `survival_check()` reports *real* missing pins, where a missing pin is the tool working, not failing; `checkpoint()` refreshes the active-work marker and appends an audit line. The broader context-discipline principle the public docs describe — *findings into context, never raw data* — is made explicit here as a written policy. (**VERIFIED** — module docstring plus the `pressure()` / `survival_check()` / `checkpoint()` / `policy()` function list.) (capabilities.md §2)

The honesty rail bites here: a *separate sandboxed-execute primitive* that runs analysis scripts off-context and returns only their printed findings is a documented design discipline of the wider platform but is **not present as a distinct module in the public repo**. What ships is the context-pressure *policy* as a first-class artifact; the sandbox-execute primitive is a roadmap claim, and the action surface that exists in the published repo is the allow-listed action runner described in §3. (**PARTIAL** for the policy-as-artifact; **UNVERIFIED in repo** for the distinct sandbox-execute primitive — capabilities.md §2; scalability.md §3.2.)

Why it matters

Long-running agents fail quietly when a compaction drops in-flight work or an open high-risk item. The independent literature backs the design: context-editing alone gives roughly a +29% lift, rising to +39% with a memory tool, and in a 100-turn evaluation context editing cut token use ~84%; degradation as input grows is universal across all major models, and bigger windows shift but do not eliminate it. Making the survival contract explicit and checkable converts a silent failure mode into a visible, fail-closed check, and keeps each worker's working context small — which both cuts per-call input cost and stays left of the degradation curve. (**VERIFIED** — researchers.md §5; _DIGEST.md §5; scalability.md §3.2.) (capabilities.md §2)

---

3. Tool integration (the allow-listed action runner)

What it does

Lets an agent invoke tools — but only named, vetted ones. The published repo ships an **allow-listed action runner**: `/api/run?action=...` can invoke only one of a few named, harmless demo actions (echo, list agents, disk-free, Python version). It is explicitly *not a raw shell*; operators add their own safe actions to a single `ACTIONS` dictionary. (**VERIFIED** — `servari_server.py` `ACTIONS` dict + `/api/run` route; `docs/ARCHITECTURE.md`; `AGENTS.md` "do not introduce a raw shell".) (capabilities.md §3)

How it is built

`ACTIONS` is a Python dict mapping a string name to a function; the runner looks up the name and returns `{allowed: [...]}` for an unknown name **rather than executing anything**. The closed `fn = ACTIONS.get(name)` lookup is the SUR's deliberate substitute for arbitrary command execution: the tool surface is a closed, reviewable set, not an open shell. (**VERIFIED** — `ACTIONS = {...}` and the unknown-name path at `servari_server.py`.) (capabilities.md §3)

On MCP. The published repository does **not** ship an MCP server or MCP client; the model wiring is OpenAI-compatible chat (§1) and the tool surface is the allow-listed runner above. A general MCP tool-integration capability is therefore **UNVERIFIED in the published sources** — it is not present in the open repo as audited, and an auditor should treat it as a roadmap/integration claim, not a shipped feature. (**UNVERIFIED in repo** — capabilities.md §3; scalability.md §3.2.)

Why it matters

An allow-listed runner is a strong, simple safety posture: the blast radius of an over-eager agent is bounded by a hand-curated list a reviewer can read in one file, so adding concurrent workers does not widen the attack/error surface linearly the way an open shell would. This is the direct structural answer to the category's cautionary tale — the most prominent open-source incumbent's documented failures (a one-click remote-code-execution CVE, command-injection CVEs, a "shadow superuser" demonstration, and a companion-service breach exposing ~1.5M auth tokens) trace to permissive defaults and a disable-able sandbox over a wide untrusted-input surface. A closed action set is the opposite stance. (**VERIFIED** for the documented incumbent events; **PARTIAL** on exact single-paper percentages — openclaw.md; _DIGEST.md §4.1.) (capabilities.md §3)

---

4. Parallel agent orchestration (the workforce surface)

What it does

Presents and coordinates a *workforce* of agents rather than a single chat: a multi-pane grid of agent channels, an org chart, a process-table-style overlay, and a launch ladder. Each agent's autonomy is a dial the human controls (see §5). (**VERIFIED** — `README.md`, `docs/ARCHITECTURE.md` UI section; routes `/api/agents`, `/api/agent-channel`, `/api/org`, `/api/grid`, `/api/launch`, `/api/agent-brief`, `/api/agent-say`.) (capabilities.md §4)

How it is built

The server exposes per-agent channels (append-only turn logs), an org structure, and a grid view, each read fail-closed from the seed data directory (`agents.json`, per-agent channel files, a service/channel index). The React layer renders these as the agent grid, org chart, and process-table overlay. (**VERIFIED** — `servari_server.py` route handlers; `docs/ARCHITECTURE.md` components list.) (capabilities.md §4)

Scoping for an auditor. The public repo provides the *orchestration surface and data shapes* (channels, org, grid, briefs, per-agent autonomy), populated from synthetic seed data. The actual parallel *execution engine* that fans work out across many concurrent model-backed workers is part of the wider platform and is **not included as a running engine in the open repo** — what ships is the controllable workforce *shell* over BYOM, ready to be pointed at real backends. (**PARTIAL / UNVERIFIED**: surface **VERIFIED**; concurrent execution engine **not in repo** — capabilities.md §4; scalability.md §2.3.)

Why it matters, and what the literature bounds

The differentiator the docs claim is *the shell plus the gates*: a polished, inspectable workspace where many agents are coordinated and every agent's autonomy is a human-controlled dial — value in the control model (§5, §6), not in a hidden black box. The independent science both supports and bounds the multi-agent pattern, and the audit reports both sides:

- **The case for.** A production orchestrator-worker research system outperformed a single agent by **90.2%** on its internal evaluation — at **~15× the token cost**, with token usage explaining ~80% of performance variance. The upside is real and measured *elsewhere*; the SUR adopts the same pattern, so its expected upside is "plausibly large on breadth-heavy tasks, at a ~15× token premium," not a number the SUR has independently benchmarked in the public sources. (**VERIFIED** for the cited system; **UNVERIFIED for the SUR specifically** — researchers.md §3; scalability.md §2.4.)
- **The case against (steel-man).** A widely-cited counter-position warns that dispersed decision-making across agents that do not share full context produces compounding, conflicting errors, and recommends defaulting to single-threaded work with scoped read-only sub-agents. The SUR is defensible **only insofar as** its workers run genuinely independent sub-tasks *and* the lead receives full traces and *synthesizes* rather than blind-concatenates. The per-agent append-only channels and a synthesizing lead are consistent with the safe pattern, but whether the SUR genuinely shares full traces is an implementation property the published shell does not by itself prove. (**VERIFIED** for the counter-position; **PARTIAL** for the SUR's adherence — researchers.md §4; scalability.md §2.3.)

Concurrency also **saturates**, for three independent reasons that converge on a working cap of roughly **8–10 effective parallel agents**: the Amdahl synthesis tax (at a 0.90 parallel fraction, 10 workers buy ~5.3× and 25 workers ~6.9×), the shared-context failure mode above, and universal context degradation as the lead loads many worker reports to combine them. Past that band, extra agents add little wall-clock but full token cost. (**VERIFIED** — scalability.md §2.2–§2.3; _DIGEST.md §6.) (capabilities.md §4)

---

5. Human-in-the-loop gate layer (autonomy dial + fast-verify queue)

What it does

This is the SUR's core stance: **autonomy is a dial, and irreversible actions always pass a human gate.** Each agent has an autonomy level from **L0 (suggest only)** to **L5 (full auto)**. The dial widens what an agent may do *silently* — but a high-risk action (deploy, real-send, spend, publish, merge, secret) *always* parks in a fast-verify queue for one-click human approval. The gates hold at every autonomy level. (**VERIFIED** — `autonomy.py` + `verify_queue.py` docstrings; persona file "the gates — your one hard rule"; `AGENTS.md` "do not weaken the gates".) (capabilities.md §5)

How it is built

Two standard-library modules:

- `autonomy.py` composes the agent's level L0–L5 with a **4–20 risk score** (modelled as reversibility × blast-radius × data-loss × certainty) into one verdict — `act` / `report` / `queue`. The level→band mapping is monotonic (higher L = wider act-band), and the decisive property is that **even at L5 a high-risk score still queues**. (**VERIFIED** — `LEVELS` table and band constants; routes `/api/autonomy`, `/api/set-autonomy`.)
- `verify_queue.py` is the gate queue itself: gated actions are parked as an **append-only audit** (`gate-queue.jsonl`); the operator approves or rejects; nothing executes until approved; the pending entry and every decision are separate append lines and the audit is never rewritten. (**VERIFIED** — module docstring + `enqueue` / `list_pending` / `decide` / `history`; routes `/api/verify-queue`, `/api/verify-decision`.)

(capabilities.md §5)

Why it matters

This is the product's explicit promise and its safety wedge: move fast on reversible/local work, never act silently on the actions that bite. Because the queue is append-only and the level→verdict logic is a small readable function, the safety property is **auditable rather than asserted**. The framing is independently grounded: the empirical backbone for a gate is that per-step error rate *rises* as a long-horizon task progresses and compounds — so retrying a deep trajectory is not the cure; checkpointing plus a gate is. And the strongest honest safety claim available is not "we are safer than frontier labs" (unprovable) but the narrower one the gate actually delivers: **the architecture treats autonomy-without-a-gate as the named risk and engineers against it.** Notably, the most prominent incumbent's *own* proposed remedy — a staged human-in-the-loop defense layer that raised measured sandbox-escape defense from a ~17% baseline into the 19%–92% range (one model class ~91.5%) — is precisely this differentiator. (**VERIFIED** for the long-horizon error backbone and the staged-defense direction; **PARTIAL** on exact single-paper percentages — researchers.md §6; openclaw.md; _DIGEST.md §4.1, §5.) (capabilities.md §5)

---

6. Mechanical safety hooks (enforcement by construction)

What it does

Safety is enforced by the system's *structure*, not by convention or prompt instruction: the action runner is allow-listed rather than a raw shell (§3), high-risk operations are intercepted before they execute (§5), and the repository's own contributor rules forbid weakening either. (**VERIFIED** — `docs/ARCHITECTURE.md`; `AGENTS.md` "things to avoid": no raw shell, no weakening the gates, no hardcoded paths, no hardcoded secrets.) (capabilities.md §6)

How it is built

The enforcement points are concrete and readable: the closed `ACTIONS` dict, the autonomy band that cannot auto-cross a hard gate, the append-only gate audit, and a quality-gating retention loop (§9) that byte-exactly reverts changes that degrade metrics. The wider platform's internal health pass reported **8/8 mechanical safety hooks present, parsing, and running** — and, with auditor candor, found and fixed one *firing-but-blind* hook in the same pass, an honest self-catch rather than a clean-only result. (**VERIFIED** across `servari_server.py`, `autonomy.py`, `verify_queue.py`, `retention.py`; the 8/8 tally is **PARTIAL** — an internal run, mechanism inspectable — capabilities.md §6; metrics.md §4; _DIGEST.md §3.3.)

Why it matters

"Gates over trust" is only credible if the gates are mechanical. The SUR's safety claims are backed by code paths an auditor can read, not by prompt instructions alone — with the honest note that the persona-level rules (§5) are a *soft* layer that complements, not replaces, the *hard* server-side enforcement. In a category whose loudest incumbent's failures are precisely soft-default failures (permissive permissions, disable-able sandbox), enforcement-by-construction is the structurally correct posture, and the contributor-rule prohibition on weakening it is what keeps the posture from eroding over time. (**VERIFIED** — capabilities.md §6; openclaw.md.)

---

7. File-based memory (persistent state in plain files)

What it does

All state lives in plain files, so the system boots into a known state and resumes where it left off; memory survives a restart. A curated **memory surface** lets the UI show which memory files exist, with live-derived freshness, *without scanning the disk*. (**VERIFIED** — public overview; the seed data directory of JSON / JSONL / Markdown files; `memory_surface.py` docstring; route `/api/memory-surface`.) (capabilities.md §7)

How it is built

State files under the seed data directory (or a home directory the operator points at) hold agent channels, autonomy levels, the gate-queue audit, the token-usage log, retention runs, context/active-work markers, and human-readable memory notes and decisions. `memory_surface.py` reads only the exact files a curator declares in a registry — it does **not** glob or walk directories — and surfaces a missing declared file honestly as `updated: "missing"` rather than fabricating. Home resolution is drive-independent (home env → repo root → cwd), with no hardcoded absolute paths. (**VERIFIED** — the seed listing: `autonomy-levels.json`, `gate-queue.jsonl`, `token-history.jsonl`, `retention/`, `context/`, `active-work.json`, `memory/decisions.md`, `memory/notes.md`; `memory_surface.py` docstring; each module's home helper.) (capabilities.md §7)

Why it matters

"Files are the memory" means there is no database to stand up, the full state is human-inspectable and diffable, and an auditor can read exactly what the system remembers. The curated, non-glob memory surface is a notable least-privilege choice: the UI shows only what a human declared, never an arbitrary disk scan — the same restraint that distinguishes the gate layer. The independent literature reinforces the value: pairing a memory tool with context discipline produced a measured +39% lift versus context editing alone, so externalized file memory is not just operationally tidy but performance-relevant. And "files-are-memory" removes a whole class of failure: state survives a process restart with no database to fail (a point that returns in the reliability analysis, §10). (**VERIFIED** — capabilities.md §7; researchers.md §5; scalability.md §4.6.)

---

8. Audit surface + token meter (structured, append-only, fail-closed)

What it does

Activity is recorded in structured logs a reviewer can read — which is precisely what makes an external audit like this one possible. A **proof-of-work token tracker** records usage and prices it at the operator's published rates, so running cost is visible. (**VERIFIED** — public overview; the append-only `gate-queue.jsonl` and `retention/audit.jsonl`; `tokens.py`; routes `/api/tokens`, `/api/tokens-sessions`, `/api/tokens-report`.) (capabilities.md §8)

How it is built

The gate queue (§5) and the retention loop (§9) both write **append-only** audit lines that are never rewritten — one JSON object per decision or event. `tokens.py` reads a JSONL usage log (`token-history.jsonl`), prices it at configurable per-million rates, and produces per-session / per-day / all-time reports; it is **read-only** against the source and writes only reports. (**VERIFIED** — `verify_queue.py` / `retention.py` invariants; `tokens.py` docstring.) (capabilities.md §8)

Cost-figure honesty (auditor note)

The token tracker's docstring is explicit that **the counts are ground truth (logged per message) while the cost is the API-equivalent at the configured rates** — i.e. **cost-to-serve, not necessarily cash spent**, since a subscription-billed operator pays differently. Any figure derived from this tracker must be read as **API-equivalent cost-to-serve**, with the assumed per-million rates stated. For grounding scale: the wider platform's

own usage log records **~10.53 billion total tokens across 19,333 messages over 19 calendar days**, of which **~10.07 billion (≈93%) are cache-read** tokens priced at the cheapest tier — which is *why* the blended cost is low, at an **API-equivalent ~$26,920 cost-to-serve** (rate-dependent; the older, widely-quoted "~$10K" figure is a stale 2026-05-24 snapshot and should not be cited as current). (**VERIFIED** for the raw counts and the cache-read share; **PARTIAL** for the dollar total — it is rate-dependent cost-to-serve, not cash — tokens.py docstring; capabilities.md §8; metrics.md §1; _DIGEST.md §3.1.)

Why it matters

Append-only audits plus a transparent cost tracker turn opaque agent activity into a reviewable record — the foundation for trust, compliance, and this very audit. The self-disclosed labelling discipline is itself a positive trust signal: the platform's own public claim register tracks **12 product claims as 4 VERIFIED / 8 PARTIAL / 0 UNVERIFIED**, with the four verified claims being the ones whose evidence was *actually run* on the collection date — a system that down-labels its own unproven claims is a system whose audit surface is doing its job. (**VERIFIED** read from the register — metrics.md §4; _DIGEST.md §3.4.)

---

9. Regression memory (the quality-gated retention loop)

What it does

Lets improvement loops run unattended *without quality silently eroding*: a change to the system is **KEPT only if a metric suite does not degrade**, otherwise it is **auto-reverted byte-exact** to the baseline. (**VERIFIED** — `retention.py` docstring; routes `/api/retention`, `/api/retention-decide`.) (capabilities.md §9)

How it is built

`retention.py` (~32 KB) runs a three-step flow. `baseline(targets)` snapshots the target files (byte copies) and records baseline metric scores; the loop (a job, a scheduler, or a human) edits the targets; `decide(run_id)` re-runs the *same* metric suite and compares — **KEEP** if no gating metric degraded (ties allowed, improvement welcome), **REVERT** (restore snapshot bytes exactly, sha256-verified) if any gating metric degraded. It is **fail-closed**: if any metric fails to *run* at decide-time, it reverts; a run can be decided only once; every decision is appended to an audit. Metrics are defined in a registry (`retention_metrics.json`) with `exit_code` or `regex` parse modes. (**VERIFIED** — `retention.py` docstring invariants.) (capabilities.md §9)

Relation to a failure-pattern regression memory

This is the quality-regression guard the public repo ships. The wider platform also describes a *separate* **failure-pattern regression memory** — recording known failure signatures so they are recognised and fixed instantly on recurrence — but that named subsystem is **not present as a distinct module in the public repo**. What is shipped and verifiable is the metric-gated KEEP/REVERT retention loop above. (**PARTIAL**: the retention loop is **VERIFIED**; a broader failure-signature memory is **UNVERIFIED in public sources** — capabilities.md §9; _DIGEST.md §8.)

Why it matters

Unsupervised self-improvement is only safe if regressions cannot land silently. A byte-exact, fail-closed revert gate is the safety rail that makes "let it run on its own" a *defensible* claim rather than a hopeful one — the same gate-first philosophy as §5, applied to the system improving itself. It is also the right rail for the long-horizon reality the literature documents: because per-step error rates rise and compound, an unattended loop without a regression gate is exactly the place silent degradation accumulates. The retention loop ties off that risk mechanically. (**VERIFIED** — capabilities.md §9; researchers.md §6.)

---

10. Local server + reliability posture

What it does

A single tiny local server is the spine: it serves the SPA and the JSON API on `127.0.0.1:8911`, same-origin (no CORS). Every route degrades gracefully — a missing module or data file returns a clean "unavailable" payload at HTTP 200 instead of crashing — and the health surface is fail-closed. (**VERIFIED** — `servari_server.py`; `health.py`; `docs/ARCHITECTURE.md`; `AGENTS.md`.) (capabilities.md §10)

How it is built

`servari_server.py` (~43 KB) is a **pure standard-library** HTTP server (no `pip install` needed to run), with a path-traversal-guarded static handler for the built front end and a per-route, per-module defensive load: each backing module loads inside a `try` / `except`, so one failing import never takes the server down. `health.py` is a fast, file-read-only, fail-closed surface: any unreadable sub-check degrades to UNKNOWN and the whole call never crashes. Configuration is environment-driven (host / port / home / Python / no-voice). Optional local voice (speech-to-text plus neural text-to-speech skeletons) loads lazily in a background thread so the port binds first; the shell runs fine without it. (**VERIFIED** — `servari_server.py`; `health.py` docstring; route `/api/health`; `voice.py` / `voice_neural.py`.) (capabilities.md §10)

Why it matters, and what the reliability math shows

A stdlib-only, fail-closed local server is exceptionally easy to audit, run, and trust: no runtime supply chain to vet, no mandatory cloud backend, and "runnable by a stranger" is enforced (a fresh clone renders every panel from seed data). The audit's quantitative finding is that **the own-server is not the scaling bottleneck** — upstream model latency and provider rate limits are; the design's scaling value is **isolation, reliability, and bounded blast radius**, not raw QPS. Each named reliability practice carries concrete leverage, all grounded in standard formulas with stated inputs:

Practice	Effect	Status
Fast health surface + auto-restart (MTTR 30 min → 30 s)	~98% → ~99.97% availability at 24 h MTBF — ~2 nines from MTTR alone	**VERIFIED** (formula; inputs stated)
Retry + exponential backoff	turns p=0.9 into 99.9% over 3 attempts — **transient errors only** (long-horizon errors compound; need checkpoint + gate)	**VERIFIED** (formula); caveat **VERIFIED**
Circuit breaker	caps wasted spend during an upstream outage — ~15× reduction in wasted work	**VERIFIED** (arithmetic; thresholds stated)
Caching as load-shedding (h=0.8)	makes 1/5 the upstream calls — 5× rate-limit headroom	**VERIFIED** (arithmetic; h from cited 80–95% band)
Multi-provider BYOM	removes the model single-point-of-failure — **only if independently configured**	**VERIFIED** (math); independence is **PARTIAL** (operator config)

(Source: scalability.md §4; _DIGEST.md §6.)

A noteworthy convergence the audit highlights: the **rate-limit math and the concurrency math independently land on ~8–15 workers** as the useful band — token-bucket admission (e.g. 60 RPM ÷ 4 calls/worker/min ⇒ ~15 workers) sits right beside the §4 Amdahl/synthesis cap. The reliability posture is the operational expression of the SUR's promise to whatever depends on it: never crash a request, degrade to "unavailable", fail closed. (**VERIFIED** formulas; inputs stated — scalability.md §4.5, §2.3.) (capabilities.md §10)

Auditor summary

The capability catalog separates cleanly into **what is shipped and verifiable in the open repository** and **what the wider platform describes as roadmap**:

#	Capability	Public-code status
1	BYOM model router	**VERIFIED** (one stdlib file + config); inter-agent provider connector **PARTIAL** (mock-driven, no live call logged)
2	Context-discipline sandbox	Context-pressure policy **VERIFIED**; distinct sandbox-execute primitive **UNVERIFIED in repo**
3	Tool integration	Allow-listed action runner **VERIFIED**; general MCP layer **UNVERIFIED in repo**
4	Parallel agent orchestration	Surface + data shapes **VERIFIED**; concurrent execution engine **not in repo**
5	Human-in-the-loop gate layer	**VERIFIED** (autonomy dial + append-only queue; high-risk queues even at L5)
6	Mechanical safety hooks	**VERIFIED** (allow-list + hard gate + revert loop); 8/8 internal tally **PARTIAL**
7	File-based memory	**VERIFIED** (plain files + curated, non-glob memory surface)
8	Audit surface + token meter	**VERIFIED**; cost = API-equivalent cost-to-serve, **not cash**
9	Regression memory	Retention loop **VERIFIED**; failure-signature memory **UNVERIFIED in repo**
10	Local server + reliability	**VERIFIED** (stdlib, same-origin, fail-closed); reliability math **VERIFIED** with stated inputs

The honest characterisation. Six of the ten capabilities are **VERIFIED in full** against the published code (BYOM router, gate layer, mechanical hooks, file memory, audit surface, local server). The other four ship a **verifiable surface or policy** while their deeper engine — a distinct sandbox-execute primitive, a general MCP layer, a concurrent execution engine, a failure-signature memory — is **roadmap, not shipped**, and is catalogued as such. The published artifact is best described as a **verifiable, gate-first, bring-your-own-model agentic operating-system shell**, and on that scope it delivers what it claims. (capabilities.md “Auditor summary”; _DIGEST.md §2, §9)

Asymmetry, stated plainly. This system is the work of **one researcher with AI assistance over ~19 recorded days**, not a funded engineering team. Its strengths — a small, readable, stdlib-only, fail-closed surface with mechanical gates and append-only audits — are real and directly verifiable in the open repo. Its honest limits are equally on the record: several wider-platform capabilities are not in the published repository; the cost figures are API-equivalent cost-to-serve rather than cash; an inventory count is not the same as full external validation; and the differentiator is **structural, not architectural** — in a category where the harness and the open skill format are commodities and the loudest incumbents have documented security failures, the SUR’s defensible edge is the gate layer, the auditable boundaries, the inspectable memory, the fail-closed retention guard, and the disciplined one-way consumption that rebuilds commodity patterns into an owned composition. Nothing here supports “beats frontier,” “fully sovereign,” or any superlative — and none is claimed. (_DIGEST.md §8, §9)

Scalability — A Quantitative Study

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

How to read this document

This is the quantitative centerpiece of the audit. It models the **cost, concurrency, throughput, and reliability** of the system under review (the "SUR" or "the platform") from the outside, with stated assumptions, worked numbers, and a literature anchor under every non-trivial claim. It is written in a structured outside-auditor voice — not in the first person, not branded as any product. (On what "independent" does and does not mean for this audit, see `00-executive-summary.md` and `08-limits-and-reproducibility.md` §6.)

Three rails run through every section:

1. **Every non-trivial claim is labeled VERIFIED / PARTIAL / UNVERIFIED.** VERIFIED = the cited source or the arithmetic states it directly. PARTIAL = real but scoped, rate-dependent, or recorded in an internal artifact not bundled publicly. UNVERIFIED = described or projected, not independently confirmable here.
2. **Every dollar figure is API-equivalent cost-to-serve, NOT cash.** The operator ran the platform largely on a flat-rate subscription, so the dollar figures are token counts multiplied by published per-token list rates — "what this volume *would* cost a buyer on metered API access." That equivalence is the useful number (it is the cost-basis a bring-your-own-model customer would actually pay), but it is not money that left a bank account.
3. **Every multiplier states its scope.** No claim of beating frontier systems, no claim of full autonomy, no superlative. The honest finding is narrower and more useful: the platform's scaling choices ride documented cost/reliability curves, and its concurrency and reliability math sit *inside* the envelope the public literature supports.

A terminology note carried from the consumption audit (`02-consumption-audit.md`): the platform's relationship to open source is **consumption** — it consumes patterns one-way into its own architecture and re-implements them in its own owned framework. It is not a "harness" and it does not "harness" anything; the word *harness* below describes only the generic class of competing tools, for contrast.

---

0. Notation and stated assumptions

These symbols are used throughout. Each calculation re-states the ones it depends on. They are **modelling inputs** chosen to be defensible against the cited literature — not measurements of one specific production run unless explicitly said so.

Symbol	Meaning	Assumed value	Basis
`P_in`	input token list price	$3.00 / M tok	Sonnet-class list price (Anthropic pricing, via the inference-economics literature, §5) — VERIFIED as configured
`P_out`	output token list price	$15.00 / M tok	same tier — VERIFIED as configured
`P_cache_read`	cached input read price	$0.30 / M tok (≈10% of `P_in`)	prompt-caching literature (§5) — VERIFIED

Symbol	Meaning	Assumed value	Basis
`P_cache_write`	cache write premium	≈1.25 × `P_in` ($3.75 / M)	prompt-caching literature ("~25% premium", §5) — VERIFIED
`h`	cache hit rate on a stable prefix	0.80–0.95	observed production band (§5) — VERIFIED
`M`	multi-agent token multiplier vs single chat	≈15×	Anthropic multi-agent research (§5) — VERIFIED
`t`	per-worker sub-task wall-clock	task-dependent	modelling input
`S`	lead-orchestrator fan-out + synthesis overhead	task-dependent	modelling input
`N`	number of parallel agents	variable	modelling input
`f`	parallelizable fraction of a job	0.0–1.0	Amdahl input (§2)
`A`	component availability	variable	reliability input (§4)

> **Cost-to-serve disclaimer (carried everywhere).** All `$` are API-equivalent at the rates above. A subscription-billed operator's cash outlay differs. Where a figure is sensitive to the rate, the sensitivity is stated.

The configured rates the platform's own usage ledger applied, for completeness (USD per million tokens — VERIFIED as configured, not re-priced against a live tariff):

Model class	in	out	cache-write	cache-read
Top tier (Opus class)	15.00	75.00	18.75	1.50
Mid tier (Sonnet class)	3.00	15.00	3.75	0.30
Light tier (Haiku class)	1.00	5.00	1.25	0.10

If list prices move, the dollar totals move proportionally; the token counts do not.

---

1. Token economics — cost per task, caching savings, cost-to-serve

1.1 The measured volume (the ground truth this section scales from)

Before modelling, here is the one solid empirical anchor: a machine-generated usage ledger, regenerated 2026-06-03, aggregating **556 session transcript files** over a **19-calendar-day window (2026-05-16 → 2026-06-03)**.

Metric	Value	Label
Recorded messages	19,333	VERIFIED (ledger total)
Input tokens	15,784,495	VERIFIED
Output tokens	55,583,873	VERIFIED
Cache-write tokens	396,640,751	VERIFIED
Cache-read tokens	10,065,077,885	VERIFIED — the dominant term, **~93% of all tokens**
Total tokens (all four classes)	**~10.53 billion**	VERIFIED (sum: 15.78M + 55.58M + 396.64M + 10,065.08M = 10,533,086,004)

Metric	Value	Label
API-equivalent cost-to-serve	**$26,920.27**	PARTIAL (rate-dependent; cost-to-serve, NOT cash)
Blended rate	**≈ $0.00256 per 1,000 tokens**	VERIFIED (derived: $26,920.27 / 10,533M)

Two things matter here for scalability:

- **Cache-read dominance (≈93%) is the structural reason the blended rate is low.** Cache-read is priced at the cheapest tier — 1.50/M for the top model class vs 15.00/M for fresh input, a **10× discount**. Most of the volume is re-reading a stable, cached context rather than paying fresh-input rates. This is the same lever §1.2 models, seen at full operating scale. **VERIFIED.**
- **The widely-quoted "~$10K" figure is stale.** It is the 2026-05-24 snapshot (8,004 messages / $10,047.89). The current cost-to-serve is **~$26.9K** over a longer window using the identical accounting method. Cite the current figure; flag any "~$10K" reference as the older snapshot. **VERIFIED (both snapshots present in the history log).**

> **Asymmetry, stated once.** This volume is the output of **one person with AI assistance over ~19 recorded days**, not a funded team. Read it as evidence of throughput for a solo+AI operation, not as parity with a staffed competitor.

1.1a What the ledger means for a buyer — the cost-to-serve → BYOM-bill bridge

The phrase "cost-to-serve" is abstract until it is divided down to a unit a buyer recognizes. The ledger gives two such units directly:

Unit	Derivation	Value	Label
Blended cost per 1,000 tokens	$26,920.27 / 10,533M tok × 1000	**$0.00256**	VERIFIED (derived)
Cost-to-serve per recorded message	$26,920.27 / 19,333 messages	**≈ $1.39 / message**	VERIFIED (derived)

The second number is the one a bring-your-own-model buyer feels: on metered API access at the configured rates, **a unit of work that produces one recorded assistant message costs ≈ $1.39 of API-equivalent spend** — driven mostly by the large cached context each turn re-reads (§1.3), not by fresh generation.

Projected forward as a **buyer's monthly bill**, at the representative cached-turn figure of §1.3 ($0.0523/turn):

```
10,000 cached single-turns/day × 30 days × $0.0523  =  $15,690 / month API-equivalent
```

This is the precise meaning of "cost-to-serve **IS** the BYOM customer's bill": the operator's flat-rate subscription absorbs the cash, but a buyer running the identical token volume on their own metered key would be billed in this band. The number is **rate-sensitive** (it moves proportionally with list prices, which are falling ~50×/yr, §5) and **mix-sensitive** (cached vs uncached, single-turn vs fan-out). It is a **cost-basis for pricing, not a quote** — but it is a cost-basis grounded in a machine-generated ledger, not a guess. **Arithmetic VERIFIED; the daily-volume mix is a stated assumption; cost-to-serve, NOT cash.**

1.2 Baseline cost of one model call

A single call costs `C = (in_tok × P_in) + (out_tok × P_out)`. For a representative mid-size agent turn — **20,000 input tokens** (system prefix + retrieved context + history) and **2,000 output tokens** — at the §0 rates:

```
C_uncached = 20,000/1e6 × $3.00  +  2,000/1e6 × $15.00
           = $0.060 + $0.030
           = $0.090 per turn        (UNCACHED)
```

Arithmetic **VERIFIED**; the 20k/2k token split is a stated modelling assumption.

1.3 Caching savings — the real number, scoped

Caching only helps the **repeated prefix** (stable system instructions + pinned context). Assume **16,000** of the 20,000 input tokens are a cacheable stable prefix and **4,000** are turn-specific (the new user/context delta). With hit rate `h = 0.90`:

```
prefix cost (cached path) = 16,000/1e6 × [ h × P_cache_read + (1−h) × P_cache_write ]
   cached-read contribution = 0.90 × 16,000/1e6 × $0.30  = $0.00432
   cache-write contribution = 0.10 × 16,000/1e6 × $3.75  = $0.00600
   prefix cost            ≈ $0.0103
delta input cost          = 4,000/1e6 × $3.00            = $0.0120
output cost               = 2,000/1e6 × $15.00           = $0.0300
C_cached                 ≈ $0.0103 + $0.0120 + $0.0300   = $0.0523 per turn
```

- **Input-side saving:** uncached input = $0.060; cached input (prefix + delta) ≈ $0.0223 → **≈63% off the input bill.**
- **Whole-turn saving:** $0.090 → $0.0523 → **≈42% off the total turn** (output is full price and does not cache).

Arithmetic **VERIFIED**; the 16k/4k split is the stated assumption.

> **Honest scoping of the "~90% cheaper" headline.** The much-quoted "prompt caching cuts cost up to ~90%" figure (VERIFIED in the literature, §5) is the **input-token, high-hit-rate, large-stable-prefix** case. The realized saving on a *whole* turn is lower, because output tokens are full price and only the stable prefix caches. The break-even is **~2+ cache hits per prefix** — the ~25% write premium must be amortized (§5, VERIFIED). A prefix used **once** is a net *loss* (write premium, no read amortization). The 93% cache-read dominance in §1.1 is what makes the high-hit case the normal case here, not the exception.

1.4 The ~15× multi-agent multiplier — caching does NOT cancel it

Two forces pull opposite ways and must be **netted**, not cherry-picked. Anthropic's own multi-agent research reports that its orchestrator-worker system "uses about 15× the tokens of a normal chat" (Anthropic Engineering, *How we built our multi-agent research system* — VERIFIED, §5). So a fan-out spends ~15× the tokens of a single turn:

```
single-agent cached turn         ≈ $0.0523               (§1.3)

parallel fan-out (M ≈ 15× tokens, VERIFIED §5):
   naive (15 uncached turns):  15 × $0.090              = $1.350
   with a SHARED cached prefix across workers:
     workers share the 16k prefix → cache-read after worker 1
     worker delta + output (full price): 15 × ($0.0120 + $0.0300) = $0.630
     amortized prefix reads:             15 × $0.00432            = $0.065
     one prefix write:                                            ≈ $0.060
   C_fanout (cached, shared prefix) ≈                               $0.755
```

Net finding. Caching roughly **halves** the fan-out bill ($1.35 → ~$0.76) by sharing the prefix — but the fan-out is still **~14× a single cached turn** ($0.755 / $0.0523). The honest statement: *parallel orchestration buys wall-clock speed and breadth by spending tokens; caching reduces the premium but does not remove it.* **VERIFIED** arithmetic; the shared-prefix step is **PARTIAL** — it assumes workers genuinely share one cached prefix, which is an implementation property.

1.5 Cost-tier routing — a real but depreciating lever

Routing routine sub-tasks to a cheaper model and reserving a frontier model for orchestration is a genuine lever. If a routine worker runs at a tier ~10× cheaper than the frontier tier, a fan-out where 12 of 15 workers are routine yields:

```
12 routine × ($0.0420 / 10)  +  3 frontier × $0.0420  +  prefix
   ≈ $0.050                  +  $0.126               +  ~$0.07
   ≈ $0.25     vs    $0.755 all-frontier    →    ~3.0× cheaper on this mix
```

~3.0× cheaper on this routine-heavy mix (12 of 15 workers routine). Arithmetic **VERIFIED**; the tier-gap and mix are stated inputs.

> **The caveat that keeps this honest.** Inference prices are falling **9×–900× per year across tasks, median ~50×/year** (Epoch AI, *LLM inference prices have fallen rapidly but unequally* — VERIFIED, §5). The *gap* between cheap-tier and frontier-tier — the exact thing routing monetizes — **shrinks over time**. The routing advantage is real **today** and is a **depreciating asset**, not a permanent defensibility. Any routing multiplier must be quoted with **date + tier mix + the erosion note.**

---

2. Parallel multi-agent concurrency math

2.1 Wall-clock: parallel vs serial

For `N` independent sub-tasks each taking `t`, serial wall-clock is `N·t`; ideal parallel is `t + S`, where `S` is the lead orchestrator's fan-out + synthesis overhead. Speedup `= N·t / (t + S)`:

N workers	serial (N·t, t = 2 min)	parallel (t + S, S = 1.5 min)	speedup
3	6.0 min	3.5 min	1.7×
5	10.0 min	3.5 min	2.9×
8	16.0 min	3.5 min	4.6×
15	30.0 min	3.5 min	8.6×
25	50.0 min	3.5 min	14.3×

Arithmetic **VERIFIED**; `t` and `S` are stated assumptions. The right column treats `S` as constant — but **real `S` grows with `N`** (§2.3), so these are an **upper bound, not a promise.**

2.2 The Amdahl ceiling — why speedup saturates

Amdahl's law: with parallelizable fraction `f`, max speedup `= 1 / ((1−f) + f/N)`, with a hard ceiling as `N → ∞` of `1 / (1−f)`:

f (parallel fraction)	ceiling 1/(1−f)	speedup at N=10	speedup at N=25
0.50	2.0×	1.8×	1.9×
0.80	5.0×	3.6×	4.2×
0.90	10.0×	5.3×	6.9×
0.95	20.0×	6.9×	10.5×

Reading. The serial part — composing briefs, then synthesizing N worker outputs into one coherent result — is the orchestrator's `(1−f)`. Even at `f = 0.90`, **10 workers buy ~5.3×, not 10×**, and **25 workers buy ~6.9×**: past ~8–10 workers, extra agents add little wall-clock but full token cost. Amdahl is a **VERIFIED** classical result; `f` is a stated per-job input.

2.3 The observed concurrency cap — and WHY it exists

The audit's working cap is **~8–10 effective parallel agents** before synthesis overhead dominates. Three **independent** forces converge on roughly that number — which is why the cap is *structural*, not a tuning artifact:

1. **The Amdahl synthesis tax (§2.2).** With realistic `f ≈ 0.85–0.90`, the marginal speedup from worker 9→10 is already < 0.3×; worker 20→25 is < 0.1×. **VERIFIED math.**

2. **The Cognition shared-context failure mode.** Cognition (Devin), *Don't Build Multi-Agents*, argues that dispersed decision-making across agents that do not share full context produces compounding, conflicting outputs — its two stated principles are "share context, and share full agent traces, not just individual messages" and "actions carry implicit decisions, and conflicting decisions carry bad results." The more concurrent writers, the more reconciliation the lead must do, so `S` **rises *super-linearly* with** `N` — it is not constant. This is the strongest external reason the cap is real. **VERIFIED** (Cognition, *Don't Build Multi-Agents*).

3. **Context degradation.** "Performance degrades as input token count increases across all major models tested" — and "million-token context windows extend where degradation occurs; they do not eliminate it." A lead synthesizing 25 worker reports must load a large combined context, hitting exactly the degradation curve that motivates keeping working context small. **VERIFIED** (context-engineering reliability literature, §5).

2.3a Parameterizing the synthesis tax — why the speedup curve turns over

The §2.1 table held `S` constant, which is the optimistic case. The Cognition failure mode (force #2 above) says the opposite: reconciling N concurrently-produced outputs is itself work that **grows with N**, because each pair of workers can produce a conflict the lead must resolve. The cleanest honest model treats synthesis as a fixed base plus a term that scales with the *reconciliation surface*:

```
S(N) = S_0 + c · N          (linear-reconciliation model, conservative)
S(N) = S_0 + c · N·(N−1)/2 (pairwise-conflict model, the Cognition-implied worst case)
```

With `S_0 = 1.0 min`, `c = 0.15 min`, `t = 2 min`, real speedup `= N·t / (t + S(N))`:

N	speedup, constant S (§2.1 optimistic)	speedup, linear S(N)	speedup, pairwise S(N)
5	2.9×	2.6×	2.4×
8	4.6×	3.4×	2.7×
10	5.7×	3.6×	2.6×
15	8.6×	3.7×	2.0×
25	14.3×	3.3×	1.1×

Reading. Under the constant-`S` fiction, more workers always help. Under either *realistic* synthesis-cost model the curve **turns over**: linear-reconciliation peaks near **N ≈ 12–15** and then declines; the pairwise-conflict model — the one Cognition's "conflicting decisions carry bad results" implies — peaks near **N ≈ 8** and falls off a cliff past it. This is the **third independent derivation of the ~8–10 cap** (alongside the Amdahl ceiling §2.2 and the rate-limit admission bound §4.5): the cap is not a tuning artifact, it is where the synthesis tax overtakes the parallelism gain. **Arithmetic VERIFIED; `S_0`, `c`, and the two functional forms are stated modelling choices — the *direction* (synthesis cost grows with N) is the VERIFIED Cognition result, the *exact coefficient* is UNVERIFIED for this platform specifically.**

> **Honest envelope.** The platform is inside the defensible zone **only** when workers run genuinely independent sub-tasks AND the lead receives full traces and *synthesizes* rather than blind-concatenates (the Cognition requirement). Fan-out of *interdependent* work without shared context falls into Cognition's failure mode. Whether the platform shares full traces is an **implementation property**: the public capability inventory describes per-agent append-only channels and a synthesizing lead, which is consistent with the safe pattern — but the **concurrent execution engine is not present in the public repository**, so this is **PARTIAL** (surface verified; engine unverified).

2.4 What the literature says the upside is worth

Anthropic's production orchestrator-worker research system **outperformed single-agent Claude Opus 4 by 90.2%** on its internal research eval, at **~15× the tokens**, with **token usage explaining ~80% of performance variance** (Anthropic Engineering — VERIFIED, §5). The honest reading for the platform: the multi-agent upside is real and measured **elsewhere**; the platform adopts the *same* pattern, so its expected upside is "plausibly large on breadth-heavy tasks, at a ~15× token premium" — **not** a number the platform has itself independently benchmarked in the public sources (**UNVERIFIED for the platform specifically**). Every speed claim must be paired with the 15× token cost.

---

3. Own-server + sandboxed-tool + full-stack scaling model

3.1 The unit and its throughput

The platform's spine is a single stdlib HTTP server, same-origin (no CORS surface), fail-closed, every route degrading to a clean "unavailable" at HTTP 200 rather than crashing (capability inventory item 10 — **VERIFIED in the public repo**). Throughput for such a unit:

```
Effective req/s ≈ workers_concurrent / mean_request_latency
```

The dominant latency is the **upstream model call**, not the local server. With a local IO-bound pool of `W` concurrent in-flight requests and mean model latency `L`:

W (in-flight)	model latency L	local-bound throughput
4	6 s	~0.67 req/s
16	6 s	~2.7 req/s
16	2 s (cached / cheap tier)	~8 req/s

Finding. The local server is **not** the bottleneck for agent workloads; upstream model latency and provider rate limits are. The own-server design's scaling value is **isolation and reliability** — no shared cloud blast radius, no CORS surface, no mandatory backend — **not raw QPS.** The throughput formula is standard queueing arithmetic (**VERIFIED**); `W` and `L` are stated inputs. The "not the bottleneck" conclusion is **PARTIAL** — it depends on the local handler being IO-bound, which the stdlib per-route defensive-load design supports.

3.1a The own-server break-even — when self-hosting beats rented API

The own-server design raises an economics question the throughput model does not answer: **at what volume does self-hosting inference beat paying per-token for a rented API?** This is a real decision the platform's self-host track faces, so the audit models it with dated, cited inputs and labels each regime.

Verified external inputs (2026, cited §5a):

Input	Value	Source
Rented A100, on-demand	~$2.00 / GPU-hr	RunPod/Lambda rental boards, 2026 — VERIFIED
Rented H100, on-demand	~$2.50 / GPU-hr	same — VERIFIED
vLLM, Llama-2-70B on 4×A100, 256 concurrent	**~2,200 tok/s aggregate**	vLLM benchmark, cited — VERIFIED
"True cost" multiplier over raw GPU (DevOps, idle, ops)	**2.5×–3×**	self-host TCO analyses, 2026 — VERIFIED (direction); PARTIAL (exact)
Self-host cheaper only above	**~11B tok/mo (~500M/day)**	published TCO break-even, 2026 — VERIFIED

The arithmetic. A 4×A100 box at $8/hr serves 2,200 tok/s ≈ 7.9M tok/hr at full utilization. Run 24/7 that is **$5,760/mo** of raw GPU rent and a *capacity ceiling* of **~5.7B tokens/mo**. The effective self-host cost-per-million-tokens depends almost entirely on **utilization** — idle GPU time is the silent killer:

```
self-host $/M (raw GPU only) = ($8/hr) / (7.92M tok/hr) / utilization
   at 100% util:  $1.01 / M
   at  50% util:  $2.02 / M
   at  20% util:  $5.05 / M
with the 3× true-cost multiplier (DevOps/idle/ops):
   at 100% util:  $3.03 / M
   at  50% util:  $6.06 / M
```

Three honest regimes (comparing the self-host blended $/M against a metered API blended at a 90%-input / 10%-output mix):

Regime	API blended rate	Self-host verdict	Why
vs a frontier-class API ($3/M in, $15/M out → ~$4.20/M blended)	$4.20/M	**Self-host wins ONLY at high utilization** (raw GPU at 100% util = $1.01/M; but with the 3× ops multiplier it needs >~70% util to stay under $4.20/M)	Frontier API output is expensive; a *busy* owned box undercuts it
vs a cheap open-weight API ($0.14/M in, $0.28/M out → ~$0.15/M blended)	$0.15/M	**API wins at every utilization** — even raw-GPU 100%-util self-host ($1.01/M) is ~7× the cheap API	Cheap metered open-weight serving is below any self-host floor
Break-even VOLUME vs frontier-class	—	**~1.37B tok/mo**, achievable only if box utilization exceeds **~24%**	Below this volume the fixed $5,760/mo box loses to metered

The finding, stated honestly. Self-hosting on *rented* GPUs beats rented API in a **narrow band**: against a *frontier-priced* API, at *high, sustained utilization*, above **~1.37B tokens/month**. Against *cheap open-weight* metered serving, **self-host essentially never wins** on cost alone. This agrees with the cited published threshold ("self-host cheaper only above ~11B tok/mo" once the full 2.5–3× TCO multiplier and sub-100% utilization are applied — the order of magnitude matches). **Arithmetic VERIFIED; rental rates and throughput VERIFIED (cited, dated); the utilization and input/output mix are stated assumptions; the TCO multiplier is PARTIAL.**

> **The economics that actually flip — owned hardware, small model.** There is one regime where self-host wins decisively, and the audit names it precisely because it is *not* the frontier-quality case. On **already-owned** consumer hardware (a used 24 GB card, ~$700, ~95 tok/s on a 7B model at Q4 quantization — VERIFIED, cited §5a), the *marginal* cost is electricity: ~350 W at $0.15/kWh ≈ **$0.154 / M tokens** — roughly **20× under a $3/M frontier-input rate**. But this flips **only** because (a) the hardware is a sunk cost and (b) the model is *small* — a 7B open model, **not** a frontier-quality model. **Quality is the trade.** The honest statement: owned-hardware self-hosting of a small open model is dramatically cheaper at the margin *and* lower capability; it is a **sovereignty-and-privacy lever, not a frontier-capability-at-lower-cost lever.** Anyone quoting the $0.154/M figure without the "small model, sunk hardware, lower quality" triplet is quoting it dishonestly. **PARTIAL** (the platform's self-host track exists and is documented; this specific owned-hardware per-M figure is a model on cited specs, not a measured production run of the platform).

3.2 Sandboxed tool execution — bounded blast radius, bounded cost

The public tool surface is an **allow-listed action runner**: a closed action dictionary where an unknown name returns the list of allowed actions and executes nothing — explicitly **not a raw shell** (capability inventory item 3 — **VERIFIED in the public repo**). The scaling implication is precise:

> The **blast radius per concurrent worker is bounded by a hand-curated list a reviewer can read in one file.** Adding workers therefore does **not** widen the attack/error surface the way an open shell would — where each new worker is another process that can run anything. This is the structural contrast with the cautionary tale of the category.

That contrast is the load-bearing comparison point. The most-starred open competitor in this space shipped permissive default permissions, a disable-able sandbox, and a wide untrusted-input surface, and accumulated documented one-click remote-code-execution and command-injection vulnerabilities plus a companion-service breach exposing ~1.5M auth tokens (OpenClaw security record — VERIFIED events; see `02-consumption-audit.md` and the competitor file for the CVE list). The academically-measured remedy for that class — **a staged human-in-the-loop defense layer that raised overall sandbox-escape defense from a ~17% baseline into the 19%–92% band (best configuration ~91.5%)** — is *precisely* the gate-first posture the platform ships by default. **VERIFIED** (the events and the direction of the defense result); **PARTIAL** (the exact single-paper percentages).

Two honest scope limits on this section:

- A **general MCP tool layer** is **UNVERIFIED in the public repo** — treat "sandboxed MCP at scale" as a roadmap claim, not a shipped one.
- The **context-discipline** principle (run analysis off-context; let only the *printed findings* enter the window, never the raw data) is the cost-control lever at scale: it keeps each worker's input small, which (a) directly cuts per-call input cost (§1) and
    b. stays *left* of the context-degradation curve (§5, VERIFIED). The shipped artifact is a **context-pressure policy** (it measures pressure, verifies survival pins, checkpoints) — **VERIFIED**; a *distinct sandbox-execute primitive* as a separate engine is **UNVERIFIED in the public repo.** The principle is quantified in §3.2a.

3.2a The context-discipline lever, quantified — script-and-capture vs raw-dump

The platform's stated discipline is: when a worker must analyze a large tool output (a JSON blob, a log, a query result), it **runs an analysis script off-context and lets only the printed findings enter the model's window** — never the raw data. §3.2 describes this; here is the number, because it is the single largest per-task cost lever the platform ships and it compounds across a fan-out.

The worked case. A worker must analyze a **200 KB tool output (~50,000 tokens)** — say, a query result or a directory scan — and produce a 2,000-token finding. Two paths, at the §0 rates with a 16k cached stable prefix (h = 0.90):

Path	What enters context	Per-call cost-to-serve	Label
RAW-DUMP (load the 50k blob, then ask)	prefix(16k cached) + blob(50k fresh input) + 2k out	prefix $0.0103 + blob $0.150 + out $0.030 = **$0.190**	VERIFIED (arithmetic)
CONTEXT-DISCIPLINE (script runs off-context; only ~300 tokens of findings enter)	prefix(16k cached) + findings(300 input) + 2k out	prefix $0.0103 + findings $0.0009 + out $0.030 = **$0.041**	VERIFIED (arithmetic)

```
per-task saving:  $0.190 → $0.041   =   ~78% off the call   =   ~4.6× cheaper
across a 15-worker fan-out each analyzing a blob:
   RAW-DUMP:         15 × $0.190  =  $2.85
   DISCIPLINE:       15 × $0.041  =  $0.62      → ~$2.24 saved per fan-out
```

Two effects, not one — this is why it raises quality AND cuts cost.

1. **Cost (VERIFIED arithmetic).** The 50k-token blob is turn-specific and uncacheable, so under RAW-DUMP it is paid at full fresh-input rate *every call*. Under the discipline it never enters the window — only the ~300-token finding does. The ~4.6× saving is on the *analysis-task* class specifically (blob-heavy work); it does not apply to tasks that have no large intermediate data.

2. **Quality (VERIFIED literature, §5).** Keeping the blob out of the window keeps each worker's input small, which holds it **left of the context-degradation curve** ("performance degrades as input token count increases across all major models tested" — VERIFIED, §5). Independent research measures the same lever's upside: context-editing alone gives a +29% performance lift, and an 84% token reduction over a 100-turn eval (Anthropic context-engineering — VERIFIED, §5). The discipline therefore **buys cost AND accuracy from the same move** — the rare lever with no trade-off against quality, because the raw data was never load-bearing for the reasoning, only the findings were.

> **Honest scope.** The 4.6× is for **blob-heavy analysis tasks**, not all tasks; it assumes the finding genuinely compresses the blob (true for counts/filters/extracts, false if the model legitimately needs the full data). The shipped artifact verified in the public repo is a **context-pressure policy** (it measures pressure and protects survival pins); a *distinct off-context execution engine* as a separate primitive is **UNVERIFIED in the public repo** — so this section quantifies the *principle the policy encodes*, and labels the separate-engine claim as roadmap. **Arithmetic VERIFIED; blob-size and finding-size are stated assumptions; the separate-engine claim UNVERIFIED.**

3.3 Cost vs load (cost-to-serve)

Per-task cost is essentially **flat in load** — there is no fixed cloud bill to amortize over volume here, because the model call dominates. So cost-to-serve scales **linearly** with task volume × the per-task figures of §1:

```
daily cost-to-serve ≈ tasks/day × C_per_task
  1,000 routine cached single-turns/day × $0.0523       ≈ $52/day   (~$1,570/mo)
    200 fan-out tasks/day × $0.755 (15-worker cached)   ≈ $151/day  (~$4,530/mo)
```

Arithmetic **VERIFIED**; all `$` are **API-equivalent cost-to-serve** at §0 rates, **NOT cash** on a subscription. The mix is a stated assumption. The two levers that bend this curve are cost-tier routing (§1.5) and caching (§1.3) — both real, both date-sensitive. This linearity is the honest scaling story: there is no magic economy of scale on the model bill; the discipline is in keeping each task's token footprint small.

The three levers now have numbers attached, and they compose: **context-discipline (§3.2a) shrinks each task's token footprint ~4.6× on blob-heavy work; caching (§1.3) cuts the recurring-prefix input bill ~42%/turn; and own-server economics (§3.1a) only bend the curve at high sustained volume on a busy box, or decisively but at lower capability on owned small-model hardware.** The honest synthesis: there is **no economy of scale on the model bill itself** — it is linear in volume — so the entire scaling discipline is *footprint reduction per task*, not *amortization over volume*. That is the structural difference between an LLM-OS cost curve and a classical-software one.

---

4. Reliability engineering for scale — the availability math

The platform frames reliability as a first-class promise to whatever depends on it. The audit translates four named engineering practices into numbers. The empirical reason this matters at all is the long-horizon reliability law: **the per-step error rate itself rises as a task progresses, rather than staying constant** — small per-step errors compound into irreversible failures (arXiv 2509.09677, *The Illusion of Diminishing Returns* — VERIFIED, §5) — and reliability is domain-stratified (one 396-task benchmark found software-engineering reliability falling 0.90 → 0.44 over the duration range; arXiv 2603.29231 — VERIFIED, §5). That compounding law is what an architecture of checkpoints + gates + verification is engineered against.

4.1 Health-check / restart — availability arithmetic

Availability `A = MTBF / (MTBF + MTTR)`. A fast health surface + auto-restart cuts **MTTR**, which is the dominant lever once MTBF is reasonable:

MTBF	MTTR (manual)	A	MTTR (health + restart)	A
24 h	30 min	98.0%	30 s	99.97%
72 h	30 min	99.3%	30 s	99.99%

Finding. Going from manual recovery (~30 min) to auto-restart (~30 s) moves a 24h-MTBF service from **~98% to ~99.97%** — roughly **2 nines gained from MTTR alone**, without touching failure frequency. The platform's fail-closed, never-crash-a-request server + fast health surface (inventory item 10 — VERIFIED) is exactly the MTTR lever this math rewards. Availability formula **VERIFIED**; MTBF/MTTR are stated inputs.

4.2 Retry + backoff — converting transient errors into success

With independent per-attempt success probability `p`, success after `k` attempts is `1 - (1-p)^k`:

per-attempt p	1 attempt	2 attempts	3 attempts
0.90	90.0%	99.0%	99.9%
0.80	80.0%	96.0%	99.2%
0.50	50.0%	75.0%	87.5%

Exponential backoff (e.g. 1 s, 2 s, 4 s + jitter) spaces retries so a transient upstream blip clears between attempts and the retries do not synchronize into a thundering herd.

> **The caveat that makes this honest.** Retry only helps **transient** failures with independent `p`. For **long-horizon** tasks, the per-step error rate *rises* as the task progresses and errors compound (§5, VERIFIED) — retrying a deep, half-corrupted trajectory is **not** independent and can make things worse. Retry is a request-level rail, not a long-horizon cure. The long-horizon cure is **checkpointing + a gate**, not a bigger retry count. This is the cleanest place the reliability math and the safety posture meet: the gate layer is a *response to a measured failure law*, not a hypothetical.

4.3 Circuit breaker — bounding cascading cost

A circuit breaker that opens after `n` consecutive upstream failures caps wasted spend and latency: instead of `N_workers × timeout × retries` piling onto a dead provider, the breaker trips and fails fast.

```
without a breaker: 15 workers × 30 s timeout × 3 retries  = up to 22.5 worker-minutes burned per outage
with a breaker tripping after 3 failures:                 ≈ 1.5 worker-minutes
   →  ~15× reduction in wasted work during an upstream outage
```

Arithmetic **VERIFIED**; the trip threshold and timeout are stated inputs. This is the operational expression of "don't keep the many hammering a downed few."

4.4 Caching as a *requirement*, not an optimization

At scale, caching is **load-shedding**, not just thrift. If hit rate is `h`, upstream call volume is cut to `(1-h)` of naive:

hit rate h	upstream calls vs naive	implied headroom
0.50	50%	2×
0.80	20%	5×
0.90	10%	10×

Finding. At `h = 0.80` (well inside the VERIFIED 80–95% production band, §5) the system makes **1/5** the upstream calls — that is **5× rate-limit headroom** and **5× fewer chances to hit a transient failure**, on top of the §1.3 cost saving. This is why the audit treats caching as a **reliability requirement** at scale, not a nice-to-have. The 93%

cache-read dominance measured in §1.1 is this lever operating at full load. Headroom arithmetic **VERIFIED**; `h` from the cited band.

4.5 Rate-limiting the many-from-few — the structural pattern

Many concurrent workers behind one operator's provider key share **one** rate-limit budget. Two structural defenses, both numeric:

- **Token-bucket admission.** Size the in-flight worker pool `W` so `W × calls_per_worker_per_min ≤ provider_RPM`. If the provider allows 60 RPM and each worker makes 4 calls/min, then `W ≤ 15` — which, **not coincidentally, sits right at the §2.3 concurrency cap.** The economics and the concurrency math **independently agree** on ~8–15 as the working band. Arithmetic **VERIFIED**; RPM is a stated input.
- **Backpressure over failure.** Excess workers **queue** (token-bucket) rather than fire-and-fail, converting a `429`-storm into orderly admission. Combined with §4.4 caching (which removes ~80% of calls before they ever reach the limiter), the effective ceiling on *useful* concurrency rises **without** raising the provider quota.

4.6 No single point of failure — the math of independence

If a system has one critical component with availability `A`, system availability is `A`. With `k` **independent** redundant components (any one suffices), system availability is `1 - (1-A)^k`:

component A	k = 1	k = 2	k = 3
0.99	99.0%	99.99%	99.9999%
0.95	95.0%	99.75%	99.99%

The platform's relevant design choices map onto this:

- **BYOM across multiple providers** (inventory item 1 — VERIFIED): the model dependency is *swappable by config*, so a single provider outage is not a single point of failure **if a fallback is configured.**
- **Stdlib-only, no mandatory cloud backend** (inventory item 10 — VERIFIED): removes a whole class of external SPOFs from the runtime.
- **Files-are-memory** (inventory item 7 — VERIFIED): state survives a process restart with no database to fail.

> **Honest limit.** The redundancy math assumes **independence**. Two providers that are actually the same model family, or two "redundant" components on one machine, are **correlated** and do not deliver `1 - (1-A)^k`. The platform makes multi-provider fallback *possible* (config-swappable BYOM — VERIFIED), but whether a given deployment is genuinely independent is the **operator's configuration**, not an intrinsic guarantee. **PARTIAL.**

---

5. Literature anchors (the science under the numbers)

Every position above is grounded in the public research literature so the math is not self-referential. The load-bearing sources, with labels:

#	Source	What it grounds	Label
Karpathy	*Software Is Changing (Again)* / "LLM OS" framing	The platform implements a **named, public** paradigm (model = compute core, context window = memory, tools = resources, gates = a leash); defensibility must come from execution + discipline, not from inventing the paradigm	VERIFIED

#	Source	What it grounds	Label
Anthropic (multi-agent)	*How we built our multi-agent research system*	The ~15× token multiplier (`M`), the 90.2% single-vs-multi upside, “token usage explains ~80% of performance variance” (§1.4, §2.4)	VERIFIED
Cognition	*Don’t Build Multi-Agents*	The shared-context failure mode; super-linear synthesis cost; “share full agent traces” requirement (§2.3)	VERIFIED
Anthropic (context engineering)	*Effective context engineering for AI agents*	Context editing +29% / with memory tool +39%; 84% token cut on a 100-turn eval; degradation universal across models (§2.3, §3.2)	VERIFIED
METR	*Measuring AI Ability to Complete Long Tasks* (arXiv 2503.14499)	The 50%-task-completion horizon, ~7-month doubling (a later ~12-hour figure is PARTIAL)	VERIFIED (later figure PARTIAL)
arXiv 2509.09677	*The Illusion of Diminishing Returns*	Per-step error rate **rises** with task length and compounds — the backbone of §4	VERIFIED
arXiv 2603.29231	*Beyond pass@1: A Reliability Science Framework*	Domain-stratified reliability decay (SWE 0.90 → 0.44); hard benchmarks < 20% pass	VERIFIED
Epoch AI	*LLM inference prices have fallen rapidly but unequally*	Prices fall 9×–900×/yr, median ~50×/yr — the routing-advantage erosion (§1.5)	VERIFIED
Prompt-caching literature	byteiota / prompthub summaries of Anthropic caching	Cache read ≈ 10% of base, ~25% write premium, break-even ~2+ hits, 80–95% production hit rates (§1.3, §4.4)	VERIFIED
Inference unit-economics	introl inference-economics guide	Batching 32 requests ≈ 85% cheaper per token for +20% latency; quantization ~60–70% cost cut	VERIFIED
AI-2027 (Kokotajlo et al.)	ai-2027.com	The canonical public statement of autonomous-agent alignment risk — the named risk the gate layer answers (timeline since revised toward early 2030s = PARTIAL)	VERIFIED (revision PARTIAL)

(Full clickable URLs for every row above are consolidated in `docs/REFERENCES.md`.)

5a Own-server / self-host economics anchors (new in §3.1a)

Source	What it grounds	Label
GPU rental boards (RunPod ~$2.50/hr H100, ~$2.00/hr A100; Lambda ~$2.80/hr H100), 2026	The rented-GPU hourly inputs to the break-even (§3.1a)	VERIFIED (dated rental quotes)

Source	What it grounds	Label
vLLM continuous-batching benchmarks (Llama-2-70B on 4×A100 ≈ 2,200 tok/s @ 256 concurrent; single-A100 8B ≈ 38 tok/s solo → ~187 tok/s at 8 concurrent)	The throughput inputs; the concurrency-multiplies-throughput point	VERIFIED (published benchmark)
Self-host-vs-API TCO analyses, 2026 (break-even ~11B tok/mo / ~500M/day; raw GPU is only ~30–40% of true cost → 2.5–3× multiplier)	The break-even threshold + the TCO multiplier (§3.1a)	VERIFIED (direction); PARTIAL (exact multiplier)
Consumer-GPU local-inference benchmarks, 2026 (24 GB card ≈ 95 tok/s on a 7B model at Q4_K_M)	The owned-hardware marginal-cost figure (§3.1a blockquote)	VERIFIED (published benchmark)

> **Why these matter.** They convert the own-server discussion from "isolation has value" (true but unquantified) into a **dated, three-regime break-even** a reviewer can re-run. Like all inference economics in this audit, they are a **moving target** — GPU rental and API prices both fall, and the break-even volume moves with them (§1.5, Epoch AI).

The honest framing of the safety angle, carried verbatim from the research grounding: the strongest claim available is **not** "this platform is safer than frontier labs" (UNVERIFIED and unprovable) but **"the architecture treats autonomy-without-a-gate as the named risk and engineers against it"** — and the reliability math in §4 is what that engineering looks like in numbers.

---

6. Synthesis — where the numbers land, honestly

1. **Token economics.** Caching cuts the input bill meaningfully (**~42% off a representative cached turn**, up to ~90% on input-heavy high-hit cases) but does **not** cancel the **~15×** multi-agent token premium; netted, a 15-worker cached fan-out is **~14× a single cached turn (~$0.76 API-equivalent** at §0 rates). Cost-tier routing adds a real but **depreciating ~3.0×** on routine-heavy mixes, eroding as inference prices fall ~50×/yr (median). At full operating scale the ledger shows ~93% cache-read dominance and a **~$0.00256 per 1,000-token blended rate** — the cost story is real and it is API-equivalent, not cash. *(Anthropic multi-agent + Epoch AI + caching literature — VERIFIED; shared-prefix arithmetic PARTIAL.)*

2. **Concurrency.** Wall-clock speedup **saturates**: at `f = 0.90`, 10 workers ≈ 5.3×, 25 workers ≈ 6.9×. The observed **~8–10 effective-worker cap** is **over-determined** by three independent forces — the Amdahl synthesis tax, the Cognition shared-context failure mode, and context degradation — so it is **structural, not a tuning artifact.** The multi-agent upside is real and measured *elsewhere* (90.2%, ~15× tokens); it is **UNVERIFIED for the platform specifically.** *(Amdahl + Cognition + context-engineering literature — VERIFIED.)*

3. **Own-server + sandboxed tools.** The local stdlib server is **not** the bottleneck; upstream model latency and provider rate limits are. The design's scaling value is **isolation + reliability + bounded blast radius** (the allow-listed runner caps each worker's surface to a list a reviewer reads in one file) — **not raw QPS.** "Sandboxed MCP at scale" and a distinct sandbox-execute engine are **UNVERIFIED in the public repo** — roadmap, not shipped. The context-discipline lever — running analysis off-context and admitting only the printed findings — is quantified at **~4.6× cheaper per blob-heavy analysis task** (~78% off the call) AND is one of the rare levers that **raises quality while cutting cost** (it keeps each worker left of the context-degradation curve; §3.2a). The *principle* is sound and literature-backed; a *distinct off-context execution engine* is **UNVERIFIED in the public repo** — roadmap, not shipped. *(Inventory items 3 & 10 — VERIFIED in repo; queueing arithmetic VERIFIED.)*

4. **Reliability.** Each of the four practices carries concrete, numeric leverage: auto-restart buys **~2 nines** via MTTR; retry + backoff turns `p = 0.9` into **99.9% over 3 attempts — for transient errors only** (long-

horizon errors compound and need checkpoint + gate, not retry); a circuit breaker cuts outage waste **~15×**; caching at `h = 0.8` gives **5× rate-limit headroom**; multi-provider BYOM removes the model SPOF **only if independently configured.** And the rate-limit math and the concurrency math **independently converge on ~8–15 workers** as the useful band. *(Availability / probability formulas VERIFIED; long-horizon compounding from arXiv 2509.09677 — VERIFIED.)*

5. **Own-server economics.** Self-hosting on *rented* GPUs beats a *frontier-priced* API only at **high sustained utilization above ~1.37B tokens/month**, and **never** beats *cheap open-weight* metered serving on cost alone. The one regime that flips decisively — **owned consumer hardware running a small open model (~$0.154/M marginal, ~20× under a frontier-input rate)** — does so *only* because the hardware is sunk and the model is small; **quality is the trade.** Self-hosting is therefore a **sovereignty / privacy / marginal-cost lever, not a frontier-capability-at-lower-cost lever** — and the audit states it as exactly that, with no superlative. *(GPU rental + vLLM throughput + TCO threshold — VERIFIED §5a; utilization/mix — stated; TCO multiplier — PARTIAL.)*

The defensible finding. The platform's scaling design is **sound on the dimensions it ships** — reliability, isolation, bounded blast radius, context discipline — and its concurrency and cost math sit **inside** the envelope the public science supports. Every advantage is scoped; every moving target (the depreciating routing gap, the configuration-dependent redundancy, the implementation-dependent trace-sharing) is named.

Asymmetry, stated plainly to close. This system is the work of **one researcher with AI assistance over ~19 recorded days**, not a funded team. The calculations here are honest models grounded in cited literature, not claims of a benchmarked production fleet; several scaling components (a distinct sandbox-execute primitive, a general MCP layer, a concurrent multi-agent execution engine, a failure-pattern regression memory) are **not present in the public repository** and are modelled as roadmap, not shipped fact. Nothing in this study supports "beats frontier," "fully autonomous," or any superlative — and none is claimed.

Comparative Analysis — The System Under Review Against ECC, OpenClaw, Hermes, and Nous

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

How to read this comparison

This document places the system under review (SUR) against four reference points in the self-hosted, local-first agentic-runtime category: **ECC** (a cross-harness skill/config bundle), **OpenClaw** (the most-starred consumer-facing agent), **Hermes Agent** (the fastest-growing self-hosted agent framework), and **Nous Research** (the open-weight model line behind Hermes). It is structured as a property matrix, then a short honest section per competitor, then "What GPT said" and "What researchers say."

Three rails govern every line below, because auditor integrity is the deliverable:

1. **The asymmetry is weighted, once, plainly.** The SUR is built by **one person with AI assistance over ~19 recorded calendar days**, not a funded team. Competitor maturity (funding, headcount, reach, marketplaces) is a **roadmap to learn from, not a deficit to spin**. (_DIGEST.md §8)
2. **Every non-trivial claim carries VERIFIED / PARTIAL / UNVERIFIED.** No "beats frontier," "fully sovereign," "AGI," or unbacked superlative appears in the source research, and none is asserted here.
3. **The one differentiator that the evidence actually supports is structural, not architectural.** In this category the agent harness and the open `SKILL.md` capability format are **commodities** — so the contest is on the layers above them, not on the harness. (_DIGEST.md §1; openclaw.md §4)

A note on terminology used throughout. The reference projects ship a **harness** — a runtime you install and run. The SUR's stated posture is different in kind: it **consumes** open-source patterns one-way, consuming them into its own owned framework and keeping the method private. The differentiator row in the matrix below is exactly this: *they ship a harness; this one consumes*. The word "harness" is used only to describe the other tools, never the SUR. (consumption.md §6)

---

1. The property matrix

The matrix scores **properties** (rows) across the five reference points (columns). Cells state the position and a confidence label. "n/a" marks a property that does not apply to that project's shape (ECC, for example, is not a runtime, so runtime-level rows often do not apply to it).

Property	SUR	ECC	OpenClaw	Hermes	Nous
Shape / category	Gate-first BYOM agent-OS shell (React + stdlib-Python server) — **VERIFIED**	Cross-harness skill/config/agent/hook bundle that installs *onto* a harness — **VERIFIED**	Self-hosted autonomous agent, messaging-app UI — **VERIFIED**	Self-hosted single-agent framework with sub-agent delegation — **VERIFIED**	Open-wei model lin decentral training research · **PARTIA**
License	Apache-2.0 (public repo) — **VERIFIED**	MIT — **VERIFIED**	MIT — **VERIFIED**	MIT — **VERIFIED**	Open-wei (lab-published **PARTIA**

Property	SUR	ECC	OpenClaw	Hermes	Nous
Open `SKILL.md` format	Yes (shared open standard) — **VERIFIED**	Yes; cross-harness adapters — **VERIFIED**	Yes; authored against agentskills.io — **VERIFIED**	Yes — **VERIFIED**	n/a (mod layer)
Reach (stars, point-in-time)	Not a popularity contestant; **n/a — pre-distribution** (no public footprint to measure)	~205K stars (4.5-mo-old, viral) — **VERIFIED**	~247K → ~355K (date-dependent) — **VERIFIED snapshot / PARTIAL "largest"**	~175K in <4 months — **VERIFIED**	Lab-scale star-drive **PARTIA**
Consume-vs-ship posture	**Consumes** patterns one-way; publishes a verifiable shell, withholds the method — **VERIFIED structurally**	Built to be **consumed + published** (open-core) — **VERIFIED**	Ships a harness + open skill registry — **VERIFIED**	Ships a harness; sub-agents seal "by omission" — **VERIFIED**	Ships ope weights + training research · **PARTIA**
Human-in-the-loop gate that bites	Autonomy dial L0–L5 × 4–20 risk → act/report/queue; **high-risk queues even at L5**; append-only queue — **VERIFIED**	Escalation + audit in the operator shell (Hermes/operator layer); oversight-gated crons — **PARTIAL**	Permissive default; per-command approval **disable-able** via API — **VERIFIED**	Oversight present; not a coded fail-closed floor — **PARTIAL**	n/a
Least-privilege execution	Closed allow-list action runner; **explicitly not a raw shell** — **VERIFIED**	Depends on the host harness — **n/a / PARTIAL**	Full user privileges by default; documented Docker-escape config — **VERIFIED**	Tool-capable agent; sandbox posture not fail-closed-by-default — **PARTIAL**	n/a
Documented security failures	None on record in the public shell; pre-publish housekeeping owed — **PARTIAL**	Security-*focus* (AgentShield) is a strength; claims vendor-stated — **PARTIAL**	Multiple CVEs incl. one-click RCE (CVSS 8.8); ~1.5M-token breach; "shadow superuser" demo — **VERIFIED**	Fewer public incidents; smaller blast surface by design — **PARTIAL**	n/a
Append-only auditable boundary	`gate-queue.jsonl` + retention audit, never rewritten — **VERIFIED**	CI/coverage hygiene; not a sealed audit boundary — **PARTIAL**	Audit not the default posture — **PARTIAL**	File memory over SQLite; not a sealed gate audit — **PARTIAL**	n/a
File-based inspectable memory	Plain files; curated surface reads **only declared files**, never globs — **VERIFIED**	SQLite session store + skill extraction — **PARTIAL**	Markdown files on disk — **VERIFIED**	Persona/memory/user files over full-text SQLite — **VERIFIED**	n/a
Fail-closed quality/regression guard	Retention loop: KEEP only if no metric degrades, else byte-exact sha256 revert — **VERIFIED**	"Instincts" skill-extraction + confidence scoring (not a sealed revert gate) — **PARTIAL**	Not a default — **UNVERIFIED**	Curated memory note-taking (not a revert gate) — **PARTIAL**	n/a
Multi-agent orchestration	Surface present (grid/org/channels); **concurrent execution engine not in public repo — VERIFIED surface /**	`/multi-plan` `/multi-execute` ("NanoClaw v2") commands — **PARTIAL**	Agent-per-channel; community skills — **PARTIAL**	Single-agent loop + isolated sub-agents (can't see each other's work) — **VERIFIED**	n/a

Property	SUR	ECC	OpenClaw	Hermes	Nous
	UNVERIFIED engine				
BYOM / model-agnostic	Yes; one small (~216-line) stdlib router + config; local-first path first-class — **VERIFIED**	Routes across harnesses/models — **PARTIAL**	Claude/GPT/Gemini/DeepSeek + local — **VERIFIED**	Hosted + local providers — **VERIFIED**	Produces models ot BYOM — **PARTIA**
Own-model production line	None (BYOM only; ~0.5B fine-tune track unfinished) — **VERIFIED / UNVERIFIED projection**	None (config layer) — **VERIFIED**	None (uses external LLMs) — **VERIFIED**	None (uses Nous + others) — **VERIFIED**	**Yes** — op weights + decentral training - **PARTIA**
Shipped paid go-to-market	None yet (pre-distribution) — **VERIFIED**	**ECC Pro $19/seat/mo** (hosted GitHub App) — **VERIFIED**	Foundation-stewarded; no per-seat price — **PARTIAL**	No per-seat price on record — **PARTIAL**	Lab/resea funding model — **PARTIA**
Public engineering hygiene (CI/coverage)	Internal smoke/eval harness (smoke 26/28; eval original-5 100% / full-suite ~79% gate-cleared); not a public CI badge — **PARTIAL**	~1,000-test CI, 80% coverage gate — **VERIFIED**	Large contributor base; security CVE remediation cadence — **PARTIAL**	Active project; hygiene not independently measured here — **PARTIAL**	Research-grade, no product-(**PARTIA**
Self-down-labelling honesty artifact	12-claim register: **4 VERIFIED / 8 PARTIAL / 0 UNVERIFIED** — **VERIFIED**	Marketing-led README; vendor-stated metrics (60%/70%) — **PARTIAL**	Marketing-led README — **PARTIAL**	Marketing-led README — **PARTIAL**	Lab paper (peer-nor not a clai register) - **PARTIA**
Bus factor (single-maintainer risk)	HIGH (one builder + AI) — **VERIFIED**	HIGH (1,426 vs 47 commits, ~30:1) — **VERIFIED**	LOW (1,000+ contributors) — **PARTIAL**	Lab-backed team — **PARTIAL**	Lab-back team — **PARTIA**

Reading the matrix. Three patterns hold across the rows. First, the SUR and the reference projects **converge** on the commodity layers (open `SKILL.md`, BYOM, file memory) — none of those is a defensibility for anyone. Second, the SUR **leads** on the gate/least-privilege/audit/retention/honesty rows, which is precisely where the loudest incumbent (OpenClaw) is documented to be weakest. Third, the SUR **trails** on reach, paid go-to-market, public CI hygiene, and an own-model line — and on bus factor it is tied with ECC, so single-maintainer risk is *not* a differentiator either way. (ecc.md §3, §9; openclaw.md §5, §8; gpt-competitors.md §2.4)

---

2. Per-competitor honest sections

2.1 ECC ("Everything Claude Code") — same category, opposite strategy

What it is. ECC is **not a model, harness, runtime, or server.** It is a cross-harness *bundle* — skills, agents, hooks, rules, command shims — that installs **onto** existing harnesses (Claude Code, Cursor, Codex, OpenCode, Gemini, Zed, Copilot) and standardizes how they operate. It uses the open `SKILL.md` convention plus per-harness adapter folders. (ecc.md §1, §4) — **VERIFIED.**

Scale, read honestly. Live GitHub API (2026-06-03): **205,179 stars / 31,492 forks / 1,030 watchers / 66 open issues**, MIT, created 2026-01-18 — only ~4.5 months old. (ecc.md §2) — **VERIFIED.** The star count is genuine but **young and viral**: a 2026-03-18 write-up cited 82K stars, so the repo roughly **2.5×'d in ~2.5 months**, attributed to a ~900K-view social thread; the watchers-to-stars ratio (~0.5%) and thin Discussions are

consistent with *starred-on-sight, not deeply adopted*. The honest reading is **attention, not verified active-deployment count**. (ecc.md §2 caveat) — **PARTIAL, triangulated.**

Bundle counts (README, exactness drifts): 63 agents, 249 skills, 79 legacy command shims, 34 rules, 15+ hook event types, 14 pre-wired MCP servers. (ecc.md §4) — **VERIFIED (README); PARTIAL on exactness.**

Genuine strengths to respect. Reach and mindshare (category-leading stars; it *is* the reference point builders cite); cross-harness breadth and portability; **real engineering hygiene** (~1,000-test CI, 80% coverage gate, regular release cadence, a merged-PR contribution process); a security focus (AgentShield) rare in the category; and a **shipped paid go-to-market** (ECC Pro, $19/seat/month, hosted GitHub App for private-repo PR audits). Its operator shell ("Hermes/operator shell," release-candidate) is the feature most directly comparable to the SUR's full operator ambition — newer and shallower, but **public, documented, and installable**, which is a distribution advantage the SUR currently lacks. (ecc.md §6, §7, §8) — **VERIFIED**, except AgentShield counts are **PARTIAL**.

Honest gaps. Star count outruns engagement; **single-maintainer bus factor is HIGH** (`affaan-m` 1,426 commits vs #2 human 47, ~30:1, two bots in the top 4); the loudest community critique is *"most people just need a good config file, not an entire ecosystem"*; install/uninstall friction is acknowledged in open issues. And its headline performance numbers — **"60% cost reduction" (Opus→Sonnet routing)** and **"70% reduction in thinking tokens"** — are **config-default deltas with no published methodology**: vendor-stated, scope = its own defaults. They must **not** be restated as benchmarks. (ecc.md §3, §5, §9) — **VERIFIED** (bus factor, critique); **PARTIAL** (the metrics).

The honest contrast. ECC = **open, broad, distributed, viral, one-person breadth, $19/seat open-core, built to be consumed and published.** The SUR = **sealed, deep, method-as-defense, human-gated, also one-person — but trading reach for a private iterative build-and-verify loop, a failure-pattern regression memory, and a gate layer.** The same solo asymmetry on both sides means **bus factor is not a differentiator**; the real contrast is *governance maturity* (ECC has it shipped) versus *defensibility method* (the SUR's claimed edge). ECC is **ahead on reach and a shipped paid seat price** — that is the SUR's go-to-market gap, **not an engineering deficit.** The SUR must never claim it "beats" ECC on adoption; it does not. (ecc.md §10; gpt-competitors.md §2.5) — **VERIFIED framing.**

2.2 OpenClaw — the category's reach leader and its cautionary tale

What it is. A free, MIT-licensed, self-hosted autonomous assistant that executes tasks via external LLMs, using **consumer messaging platforms** (Signal/Telegram/Discord/WhatsApp) as its primary UI rather than an IDE or terminal. Created by Peter Steinberger; written in TypeScript/Swift; ships the runtime plus connectors, persistent memory, a skills engine, and first-class tools (browser, canvas, cron, sessions). (openclaw.md §1) — **VERIFIED.**

Scale. By ~2026-03-02 it reported ~247,000 stars / ~47,700 forks; reported at ~355K by mid-2026; at one point the most-starred project on GitHub. (openclaw.md §2) — **VERIFIED snapshot; "beat React" framing PARTIAL (date-dependent).** Governance churn is real: renamed twice in early 2026 (a trademark complaint drove the "Moltbot"→"OpenClaw" change), the creator joined OpenAI 2026-02-14, and an "OpenClaw Foundation" now stewards it. (openclaw.md §2) — **VERIFIED.**

Why it is the load-bearing comparison — security is the central finding. The pattern is consistent: extreme capability + minimal default guardrails + exposure to untrusted input = a broad, repeatedly-exploited surface. - Default permission model is **permissive** (full user privileges; per-command approval can be **disabled** via API; a documented config enables a Docker escape; credentials stored **plaintext** under `~/.openclaw/`). (openclaw.md §5) — **VERIFIED.** - **CVE-2026-25253 (CVSS 8.8): one-click RCE** via token exfiltration through an unvalidated `gatewayUrl`; plus command-injection CVEs (CVE-2026-24763, CVE-2026-25157) and indirect-prompt-injection via web (CVE-2026-22708). (openclaw.md §5) — **VERIFIED.** - A companion-service breach exposed **~1.5M API auth tokens, ~35K emails, and plaintext model keys**; a named researcher demonstrated a **"shadow superuser"** — obtaining keys, sending messages as the user, executing admin commands. (openclaw.md §5) — **VERIFIED (reported).** - Independent academic measurement: baseline sandbox-escape defense averaged **~17%**; adding a **staged human-in-the-loop defense layer** (allowlist →

semantic judge → risk-pattern rules → sandbox guard) raised overall defense to **19%–92% (Claude ~91.5%)**. (openclaw.md §5) — **VERIFIED (events); PARTIAL (exact single-paper percentages).**

The symmetry that defines the comparison. OpenClaw's own academic remedy — *a staged human-in-the-loop defense layer* — **is precisely the SUR's claimed differentiator.** The SUR ships a coded autonomy floor (high-risk actions queue for human approval even at maximum privilege) and an append-only approval queue with named gate classes: inspectable mechanism, not policy prose. (openclaw.md §8; capabilities.md §5) — **VERIFIED structurally.**

Honest both ways. OpenClaw has **scale, contributors, reach, and a low-friction messaging UX** a solo-plus-AI effort cannot match — genuine assets. (openclaw.md §6) The SUR's credible edge against it is **safety posture and method, not capability or community size**, and that edge is only real if the SUR's public shell and gate evidence are shipped and reproducible. (openclaw.md §8) — **PARTIAL judgment, well-supported.**

2.3 Hermes Agent — the architecture contrast

What it is. MIT, Python, self-hosted, launched 2026-02-25; **~175K GitHub stars in under four months** — widely described as the fastest-growing open-source agent framework of 2026 (popularity, **not** usage or quality — flag carried). (gpt-competitors.md §2.1) — **VERIFIED.**

Architecture (the contrast that matters). A **single-agent synchronous loop with isolated sub-agent delegation** — sub-agents **cannot see each other's in-progress work**; per its own docs, sealing is *"by omission, not by mechanism."* A multi-adapter gateway makes one daemon appear across ~14–20 messaging surfaces; file memory (persona + memory + user files) over a full-text SQLite index; BYOM. Critically, its **"self-improvement" = autonomous skill-writing + curated memory, NOT weight updates** — independent reviewers characterise it as structured note-taking with retrieval. (gpt-competitors.md §2.1) — **VERIFIED.**

Audit relevance. Hermes is the cleanest **architecture** comparison. Its "sealing by omission" is the inverse of what the multi-agent literature recommends (Cognition: *share full traces*; see §4 below) — but for *scoped, read-only* sub-tasks that omission is defensible. The SUR's claimed differentiation is not "more agents" but a **gate + audit + retention** wrapper around whatever orchestration runs; on the orchestration mechanism itself, Hermes is a mature, public reference the SUR's *engine* (not in the public repo) has not yet matched in the open. (gpt-competitors.md §2.1, §2.4) — **VERIFIED.**

2.4 Nous Research — the sovereignty / model-layer line

What it is. The lab behind Hermes also ships **open-weight models** (function-calling, hybrid reasoning, low-refusal alignment; one variant advertises a **512K-token context**) and pursues **decentralized training over the internet** (volunteer-GPU coordination reportedly cutting inter-GPU communication by orders of magnitude; a multi-tens-of-billions-parameter distributed pretraining run claimed). (gpt-competitors.md §2.3) — **PARTIAL** (weights and decentralized-training research are real and published; specific scale/quality claims are the lab's own and were not re-benchmarked here).

Audit relevance. Nous is **not a runtime competitor** — it is the **model production line** the others (and the SUR) BYOM against. Its relevance is twofold: (1) it **validates the BYOM / run-your-own-model strategy** the SUR is built on; (2) it points at a credible **long-horizon path to producing the model layer without hyperscaler compute.** For the SUR, this is a **roadmap to learn from, not a present capability**: the SUR's own model line is a ~0.5B fine-tune track that is **unfinished** (−20pp regression → +13.3pp sub-threshold → an unrun projection). The SUR should never imply it has Nous's model line; it has the BYOM wiring that *consumes* such a line. (gpt-competitors.md §2.3; metrics.md §6) — **PARTIAL.**

---

3. The consume-vs-harness differentiator, examined

This is the matrix row that most distinguishes the SUR's *strategy* (as opposed to its *capabilities*, which converge with everyone on the commodity layers). It deserves its own treatment because it is easy to over-claim.

The reference projects ship a harness. ECC ships a bundle to install onto harnesses; OpenClaw and Hermes ship harnesses you run. Their value flows **outward** — published, MIT, copyable, built to be adopted. (ecc.md §10; openclaw.md §1) — **VERIFIED.**

The SUR consumes. Its stated posture is **disciplined, mostly-one-way consumption**: consume patterns from permissively-licensed projects, re-implement them inside its own owned framework, keep the method private. This is **VERIFIED structurally** — the SUR is written in a different language (stdlib Python) from the vendors' TS/JS, keeps **segregated reference copies**, and its **running code does not import the references**. A four-question intake gate (license check → value÷overhead ≥ 3× → framework-conflict check → paid-service-drift check) plus a six-step pattern ending in a real test and human review governs each intake. Documented consumption targets include a context-discipline family, a read-before-edit safety invariant, symbol-aware edits (Python stdlib AST), a post-edit failure classifier + syntax gate (ACI pattern from **SWE-agent, MIT**), trajectory evaluation (**agentevals, MIT**), permission-gating/phase-locking, and exploratory-QA/email patterns (from OpenClaw + Hermes, **pattern-only, no vendor code**). (consumption.md §6) — **VERIFIED structurally; PARTIAL for the paid-provider adapter (mechanism present, live call not run).**

The honest correction, carried from the source. "One-way" is the right *spirit* but **imprecise**. The precise claim is **"consume freely; publish a verifiable shell + interfaces; never publish the method."** A separate audited release plan publishes the shell, interfaces, memory format, and safety-gate model **while withholding** the orchestration intelligence, live block-hook implementations, and private prompts. So the posture is **asymmetric, not literally zero-outbound.** (_DIGEST.md §7) — **VERIFIED.**

Pre-publish housekeeping (named, not buried). "No vendor code" is verified *structurally*, not by a token-level clean-room diff; one source is fair-use / source-available (not classic-OSI) and warrants a lawyer's eye before redistributing its reference copy; a public NOTICE/attributions file is owed; per-item smoke-tests exist but no single aggregate regression run of the consumed layer. These are housekeeping items, not integrity gaps. (_DIGEST.md §7) — **PARTIAL / UNVERIFIED as noted.**

Net. The consume-vs-harness distinction is a **real strategic differentiator** and the SUR's posture is structurally evidenced — but it is a *strategy* claim, not a *quality* claim. It does not, by itself, make the SUR better; it makes the SUR's value-flow direction (inward, sealed) the opposite of the incumbents' (outward, open). Whether that is an advantage depends entirely on whether the method it withholds proves to be worth withholding.

---

4. What GPT said

The honest answer is the most important sentence in this section: **there is no on-record GPT (or external-AI) critique of the SUR or its audit approach.** (gpt-competitors.md Part 1) — **VERIFIED.**

- **No stored GPT/external-AI critique of the SUR exists on disk.** The internal audit pack is internally authored and internally adversarially re-read; it does not cite an external-AI reviewer. (gpt-competitors.md §1.1) — **VERIFIED.**
- **GPT/OpenAI appears only as a *routing target*, never a commentator** — i.e., the OpenAI-compatible `/chat/completions` request shape and an optional terminal backend. These are integration references, not opinions. (gpt-competitors.md §1.1) — **VERIFIED.**
- **The SUR's external-model connector is an unproven shell.** Its stored metadata self-labels as a mock-driven shell with real-provider integration deferred; the only logged calls are an authentication failure and a rate-limit failure, with **$0.00 recorded and zero tokens exchanged**. No real GPT inference round-trip is on record. Any "cross-model council" framing was, at the dates examined, **wired-but-not-executed design**, not evidence of a GPT review. (gpt-competitors.md §1.1) — **VERIFIED.**
- **The only captured external-model review used a different model family** (not GPT): a less-correlated reviewer **broke a claim the first reviewer had blessed**, and the SUR persona **conceded and**

corrected rather than doubling down. That concede-and-fix under adversarial pressure is the **most load-bearing external-review datapoint on file** — and it is behavioural, not a benchmark. (gpt-competitors.md §1.1, Part 1 bottom-line) — **PARTIAL.**

Auditor's bottom line. Do **not** imply a GPT endorsement exists. The SUR does not hide behind a borrowed external seal of approval, and the materials do not pretend there is one — which is itself an integrity-positive signal. If GPT commentary was ever offered in an un-exported chat, it is **not recoverable** and is left explicitly **UNVERIFIED**. (gpt-competitors.md §1.2)

---

5. What researchers say

The independent literature grounds the comparison in science rather than vendor marketing. Five findings bear directly on where the SUR honestly stands.

- **The paradigm is borrowed, not invented.** The SUR implements Karpathy's **LLM-as-OS / "Software 3.0"** pattern (model as swappable compute core, context window as working memory, tools as resources, human gates) — a *named, public* paradigm. Defensibility must come from **execution and discipline**, not from claiming the paradigm. (Karpathy, *Software Is Changing (Again)*, YC AI Startup School 2025; researchers.md §1) — **VERIFIED.**
- **Multi-agent: real upside, real cost, real failure mode.** Anthropic's orchestrator-worker research system **outperformed single-agent by 90.2%** on its internal eval — at **~15× the tokens**, with **token usage explaining ~80% of performance variance.** (researchers.md §3) — **VERIFIED.** The steel-man against it (Cognition, *"Don't Build Multi-Agents"*): dispersed decision-making across agents that **don't share full context** produces compounding, conflicting errors; default to single-threaded, use sub-agents only for **scoped, read-only** work. (researchers.md §4) — **VERIFIED.** *The implication for the comparison:* the SUR (and Hermes, which seals "by omission") sit in the defensible zone **only insofar as they share full traces and synthesize rather than blind-concatenate** — an implementation property an auditor must examine, not assume. Pair every speed claim with the **15× token cost.**
- **Context discipline + external memory are independently validated as correct.** Anthropic: context editing alone = **+29%**, with a memory tool = **+39%**, and in a 100-turn eval context editing **cut tokens 84%**. Degradation is **universal across all major models**; bigger windows shift but do not eliminate it. The SUR's context-discipline and file-based external memory are **textbook-correct.** (researchers.md §5) — **VERIFIED.**
- **Long-horizon reliability is a measured, compounding problem.** METR's 50%-task horizon doubles ~7 months, but **per-step error rate *rises* as a task progresses and compounds**, and reliability is domain-stratified (SWE 0.90→0.44 over duration; hard benchmarks <20% pass). This is the **empirical backbone for the gate + per-step-verification argument** — and it equally **discounts any SUR claim of reliable fully-autonomous long-horizon execution.** (researchers.md §6) — **VERIFIED** (a later ~12-hour-horizon figure is **PARTIAL**).
- **The safety framing the science supports.** AI-2027 (Kokotajlo et al.) is the canonical public statement of autonomous-agent alignment risk. The strongest *honest* framing of the SUR's posture is **not** "we are safer than frontier labs" (UNVERIFIED, unprovable) but **"the architecture treats autonomy-without-a-gate as the named risk and engineers against it."** (AI 2027, AI Futures Project; researchers.md §2) — **VERIFIED** (the AI-2027 timeline has since been revised toward the early 2030s — **PARTIAL**).
- **Inference economics favor the approach but are a moving target.** Prompt caching cuts cost up to **~90%** / latency up to **~85%** on long stable prefixes; batching 32 requests ≈ **85%** cheaper per token. But Epoch AI: inference prices fall **9×–900×/yr, median ~50×/yr** — so the cheap-vs-frontier **routing advantage is a depreciating asset**, and the net cost story is **caching/batching savings *minus* the ~15× fan-out spend.** (researchers.md §7) — **VERIFIED.**

---

6. Synthesis — where the comparison lands

1. **On commodity layers, everyone converges.** The harness and the open `SKILL.md` format are shared, cross-vendor, and copyable. No project in this matrix — including the SUR — has a defensibility *there*. (The `SKILL.md` format is an open, cross-vendor standard originally released by Anthropic; openclaw.md §4) — **VERIFIED.**
2. **The SUR's only evidence-backed edge is structural.** A mechanical, fail-closed, human-in-the-loop gate layer; least-privilege execution; append-only audit; a sha256-verified retention guard; and a self-down-labelling claim register — against a category whose reach leader has shipped a **documented "shadow superuser" failure** whose *own academic remedy is a staged human-gate*. That symmetry is the single strongest comparative argument. (openclaw.md §5, §8) — **VERIFIED structurally.**
3. **The consume-vs-harness posture is a real strategic differentiator, not a quality claim.** It is structurally evidenced and asymmetric (consume freely; publish a shell; withhold the method) — but it makes the SUR *different in direction*, not *better by default*. (_DIGEST.md §7) — **VERIFIED.**
4. **The SUR is behind on exactly what the asymmetry predicts.** Reach (incumbents at 175K–355K stars), funding, team, a shipped paid go-to-market (ECC's $19/seat), public CI hygiene (ECC's ~1,000-test gate), and an own-model line (Nous). **The gap is distribution and third-party trust-proof, not engineering.** Competitor maturity is a roadmap, not a deficit. (ecc.md §10; gpt-competitors.md §2.4) — **VERIFIED.**
5. **No external endorsement, and none claimed.** There is no on-record GPT review; the strongest external-review datapoint is behavioural (concede-and-correct under scrutiny). The science supports the SUR's *architectural choices* (context discipline, external memory, gates against a measured failure law) but supports **no** claim of beating frontier systems, full autonomy, or superiority — and none is made. (gpt-competitors.md Part 1; researchers.md §8) — **VERIFIED.**

The one-line verdict. Against ECC, OpenClaw, Hermes, and Nous, the system under review is **not the most-adopted, the best-funded, or the model-producing** project in the category — and it does not claim to be. It is the one whose **default posture is the safety remedy the rest of the category has had to bolt on after the fact**, built by one person with AI assistance, defensible on **method and discipline** rather than reach, and honest enough to grade its own claims down. That is a structural edge to defend and prove in distribution — not a finished product to oversell.

Sources

- `docs/REFERENCES.md` — consolidated external literature with verified URLs (Karpathy LLM-OS, AI-2027, Anthropic/Cognition multi-agent, METR, Epoch AI, OpenClaw, Hermes/Nous, the open SKILL.md standard)
- `_research/_DIGEST.md` — sealed synthesis (spine, capabilities, numbers, competitor points, science, consumption posture, honest limits)
- `_research/ecc.md` — ECC live API metrics, bundle counts, bus factor, GTM, honest gaps
- `_research/openclaw.md` — OpenClaw scale, the open `SKILL.md` standard, the security incident record, comparative takeaways
- `_research/gpt-competitors.md` — "What GPT said" (Part 1), the competitor landscape incl. Hermes/Nous/ECC (Part 2)
- `_research/researchers.md` — Karpathy LLM-OS, AI-2027, multi-agent for/against, context engineering, long-horizon reliability, inference economics
- `_research/capabilities.md` — the SUR's verifiable capabilities from the published Apache-2.0 repo

Methodology — Built With Claude

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

How to read this document

This document audits one narrow, often-skipped question: **how was the system under review (the "SUR" or "the platform") actually built — and specifically, how was code that was written by an AI model held to a standard worth trusting?** Most product materials describe what a system *does*. Far fewer disclose *how it was constructed*, and fewer still disclose that the construction itself leaned heavily on large language models. That disclosure is the subject here.

Three rails run through every section, identical to the rest of this audit:

1. **Every non-trivial claim is labeled VERIFIED / PARTIAL / UNVERIFIED.** VERIFIED = the cited source, the on-disk artifact, or the arithmetic states it directly. PARTIAL = real and well-supported, but one verification step (a live run, a clean-checkout reproduction, a line-by-line diff) was not performed in this pass. UNVERIFIED = described or projected, not independently confirmable from the materials examined.
2. **Cost figures are API-equivalent cost-to-serve, NOT cash.** Where dollar figures appear, they are token counts multiplied by published per-token list rates — "what this volume *would* cost a buyer on metered API access." The platform was operated largely on a flat-rate subscription, so the figure is a cost-basis, not money that left a bank account. [*Source:* `metrics.md` *§1, caveat 1.*]
3. **The asymmetry is stated plainly, once.** This system is the work of one researcher with AI assistance over roughly nineteen recorded days, not a funded engineering team. That makes the throughput notable for a solo+AI operation and makes the un-finished edges expected. Competitor maturity, where it appears, is a roadmap to learn from, not a deficit to spin. [*Source:* `_DIGEST.md` *§8.1;* `metrics.md` *§0, caveat 2.*]

A terminology note carried from the consumption audit (`02-consumption-audit.md`): the platform's relationship to open source is **consumption** — it consumes patterns one-way into its own architecture and re-implements them in its own owned framework. It is not a "harness" and it does not "harness" anything; the word *harness* below describes only the generic class of competing tools, for contrast.

A scope boundary specific to this document: the audit does **not** claim to have reverse-engineered the platform's private build method. That method is deliberately withheld by the operator (see the consumption audit's "consume freely; publish a verifiable shell; never publish the method" finding — `consumption.md` §5). What this document describes is the *generic engineering practice* observable from the published artifacts and the disclosed build records: the human-plus-AI loop, the discipline applied to AI-written code, the verification habits, and the deterministic-versus-reasoned split. The specifics of how the orchestration is composed are out of scope and are not revealed here. [*Source:* `consumption.md` *§5;* `_DIGEST.md` *§7.*]

The alignment frame this methodology answers to

One reason a build-methodology audit matters at all is that the public AI-safety literature now names, explicitly, the failure class an autonomous agent system most needs to engineer against. **AI 2027** — the scenario forecast from the AI Futures Project (Daniel Kokotajlo and colleagues), published April 2025 — is the canonical public statement of *autonomous-agent alignment risk*: systems that act over long horizons, improve themselves, and take irreversible actions faster than a human can supervise them. (AI 2027 — about; project site) — **VERIFIED** (the project's own timeline has since been revised toward the early 2030s — **PARTIAL**).

The honest framing — the one this audit can defend and the one this methodology section exists to evidence — is **not** "the SUR is safer than frontier labs." That claim is unprovable and is **UNVERIFIED** by construction; the audit makes it nowhere. The defensible framing is narrower and stronger: **the SUR's methodology treats autonomy-without-a-gate as the named risk and engineers against it by default.** Concretely, the build process the rest of this document audits is *honest-by-design*: every irreversible or outward action is composed through a mechanical, fail-closed, human-in-the-loop gate (autonomy level × risk score → act / report / queue); AI-written code is held to a per-step verification standard rather than trusted on output; and the audit trail is append-only. These are not marketing postures — they are the methodology's direct response to a *measured* failure law. The reliability science is independent and unambiguous on this point: per-step error rates **rise** as a task progresses and compound into irreversible failures (arXiv 2509.09677; arXiv 2603.29231), and the upside of multi-agent parallelism comes with a documented fragility when context is not shared across agents (Cognition, *Don't Build Multi-Agents*). An architecture that externalizes memory, verifies each step, and gates the irreversible is responding to that law — which is precisely why a methodology that bakes the gate into the build loop, rather than bolting it on afterward, is the part worth auditing. (See References for full sources.) — **VERIFIED** (framing); the SUR's *own* gate-coverage completeness is examined in docs 03/08 and is **PARTIAL** there, not assumed here.

---

1. The honest premise: this was built with AI, and says so

The most important methodological fact about the SUR is one many products would bury: it was **written predominantly by large language models, under human direction.** The named researcher and author is **Haris Mekic**; the lead author and engineer of the code and the documents is **Claude Opus 4.8**, with **Claude Sonnet**, **OpenAI GPT**, and **Google Gemini** used for cross-checks. This is stated in the header of every artifact in the audited pack and is not framed as an embarrassment to be minimized. [*Source: the header line on each of* `capabilities.md`, `consumption.md`, `metrics.md`, *and the companion docs;* `_DIGEST.md` *lines 12–14.* — **VERIFIED.**]

For an auditor, AI-assisted authorship is neither a virtue nor a defect on its own. It is a **risk profile** — a specific, well-documented set of failure modes that must be engineered against:

- **Fabrication / hallucination.** A model can produce confident, well-formed code or prose that references functions, files, or facts that do not exist.
- **Silent drift.** Across many iterations, a model can quietly broaden scope, change a constant, or weaken a check while appearing to "just edit."
- **Stale or guessed context.** A model can edit a file using a remembered version rather than the current one, corrupting it without any error firing.
- **Plausible-but-wrong reasoning.** A model can choose an approach that compiles and reads well but is structurally incorrect for the task.

The methodological question is therefore not "did a human or an AI write this?" but **"what mechanical and procedural discipline was placed between the AI's output and the shipped artifact?"** The remainder of this document answers that. The short version: the SUR's build practice treats AI-written code as **untrusted input that must pass gates**, not as finished work — a stance directly continuous with the product's own runtime philosophy of gating autonomous actions (see capabilities §5–§6, `03-capabilities.md`). [*Source:* `capabilities.md` *§5–§6;* `consumption.md` *§2.* — **PARTIAL** (the continuity is the auditor's structural reading of two verified layers).]

---

2. The human-plus-AI build loop

2.1 The shape of the loop

The platform records its construction history as a **numbered sequence of build-and-verify iterations** — each iteration is a change, a self-test step, and a logged outcome. The highest iteration index referenced in the internal records is in the **~260 range.** [*Source:* `metrics.md` *§3.* — **PARTIAL** (internal log; an upper-bound index, not a guarantee that every index 0..260 is an independently reproducible event).]

The observable loop, reconstructed from the disclosed records and the published repository, has a consistent shape:

Step	Actor	What happens
1. Direction	Human (Haris Mekic)	Sets the goal, the scope boundary, and the acceptance bar for the iteration.
2. Generation	AI (Claude Opus 4.8, lead)	Drafts the code, the script, or the document for that scoped change.
3. Cross-check	AI (Sonnet / GPT / Gemini)	A different model reviews or stress-tests the draft where used.
4. Verification	Deterministic script + human	A self-test runs; the human inspects the result against the acceptance bar.
5. Decision	Human + mechanical gate	Keep, revise, or revert — never "ship because it looked done."
6. Logging	System	The change and its pass/fail outcome are appended to the build record.

The load-bearing property is **step 4 before step 5**: a verification step is paired with each build step rather than deferred to the end. Sampling of the internal log shows iterations "routinely paired with a pass/fail self-test line." [*Source:* `metrics.md` *§3, "Iterations cited with an in-line self-test result."* — **PARTIAL** (the *practice* of pairing is evident and consistent in the sampled records; the exact total of fully reproducible iterations was not independently counted).]

> **Auditor reading.** The honest reading the source itself states is the right one: the *practice* of pairing each build step with a verification step is evident and consistent in the sampled records, but the full count of independently reproducible iterations was not audited. The discipline is demonstrated; the exhaustive census is not. [*Source:* `metrics.md` *§3 closing note.*]

2.2 Why multiple models

Using more than one model family is not redundancy theater; it is a deliberate **de-correlation** strategy: a reviewer too similar to the author shares the author's blind spots, while a reviewer from a different lineage catches what the author and a like-minded checker both miss. The build practice operationalizes this by routing cross-checks (step 3 above) through Sonnet, GPT, and Gemini rather than only re-asking the lead model. [*Source: header lines naming the four models.* — **PARTIAL** (the four models are named as used; per-iteration cross-check routing is not exhaustively logged in the public pack).]

The verifiable form of "did the multi-model discipline actually hold the line?" is not an internal transcript — it is the **public reproducibility harness** (`scripts/verify_all.py`, 8/8 PASS, exit 0). That harness lets any reviewer execute the audit's load-bearing safety properties on the published code without trusting any model's word, including the lead's. The harness is the externally-checkable artifact this methodology stands on; the per-review internal cross-check records are not bundled and are not relied on here. [*Source:* `scripts/verify_all.py` *;* `metrics.md` *§5.* — **VERIFIED** (executable on the public repo).]

> **Honesty rail.** This does **not** mean the system is "safe" or "verified beyond doubt." An 8/8 harness PASS proves *those eight public properties hold on the published code*, not a general guarantee. The multi-model loop reduces within-family blind spots; it is not a substitute for independent human review. [*Source:* `metrics.md` *§5 honesty limit.*]

---

3. The discipline applied to AI-written code

This is the core of the methodology and the part most directly relevant to a buyer evaluating AI-built software. The SUR's build practice places **mechanical gates** between AI output and the codebase. These are not prompt

instructions (“please be careful”); they are runnable checks with explicit pass/fail outcomes. The consumption audit confirms several of these were themselves **consumed as patterns** from open research and re-implemented in the platform’s own Python standard-library stack. [*Source:* `consumption.md` *§4.2–§4.6.* — **VERIFIED** (patterns present on disk with their own scripts and recorded smoke-tests).]

3.1 The edit-safety stack

The most coherent example is a four-layer **edit-safety stack** assembled from consumed patterns, each a guard against a distinct way AI-written edits go wrong:

Layer	Guard	Failure mode it catches	Pattern source
Prevention	Read-before-edit invariant	An edit whose target text was drawn from the model’s *memory* rather than from a current read of the file	A code-intelligence “lens” module (Pi.dev ecosystem); re-implemented in stdlib Python
Prevention	Symbol-boundary-aware edit	An edit that lands mid-statement or cuts a decorator because the model guessed the symbol’s extent	A read-expansion module from the same lens; re-implemented using Python’s stdlib AST parser
Prevention	Pre-commit syntax gate	An edit that introduces a *new* syntax error — checked before it lands, auto-reverted if it does	Agent-computer-interface (ACI) edit-safety pattern from **SWE-agent (MIT)**
Recovery	Seven-class failure classifier	A failed edit retried blindly instead of diagnosed and routed to the correct fix	Structured diagnostic-logger pattern from the same lens

[*Source:* `consumption.md` *§4.2–§4.4 and the “edit-safety stack” synthesis.* — **VERIFIED** (all four capabilities present and interlinked; each carries a recorded smoke-test — 4/4 or 8/8 — in the findings ledger).]

The significance for AI-built code is direct. Two of the four layers (read-before-edit, symbol-boundary-aware edit) exist **specifically** because a language model editing files will, sooner or later, edit a stale or misremembered version and corrupt it silently — no exception is raised, the file is simply wrong. Mechanizing the guard converts a silent corruption class into a caught, fail-closed event. This is the AI-code analogue of the product’s runtime gate philosophy: *do not trust the autonomous actor on the actions that bite.* [*Source:* `consumption.md` *§4.2–§4.4;* `capabilities.md` *§5.* — **VERIFIED** for the mechanism; the analogy is the auditor’s structural reading.]

3.2 Trajectory evaluation, not just output

A second consumed discipline grades an agent’s **tool-call path** (the trajectory it took), not only its final answer — with strict / unordered / subset match modes. It was re-implemented in the Python standard library from the open **agentevals (MIT)** library. [*Source:* `consumption.md` *§4.5.* — **VERIFIED** (capability + stdlib scorer present, 5/5 smoke-test recorded; the findings credit agentevals as the pattern source).]

Why this matters methodologically: a large fraction of agent failures are invisible if you grade only the final output, because the agent can reach a plausible-looking answer by a wrong path that will not generalize. Grading the path closes that blind spot. For a system whose code is written by agents, evaluating *how* an iteration arrived at a result is a stricter bar than evaluating *whether* the result looks right. [*Source:* `consumption.md` *§4.5.*]

3.3 Gating dangerous operations at construction time

The build practice also mechanizes a **pre-execution gate that blocks dangerous version-control commands before they run**, alongside a structured phase-locking discipline (each phase has explicit entry conditions, exit criteria, and a checkpoint artifact; the next phase verifies the previous phase’s artifact before opening). Both were consumed pattern-only from a permission-gate extension cluster in the coding-agent harness

ecosystem. [*Source:* `consumption.md` *§4.6.* — **VERIFIED** (phase-lock discipline present; the dangerous-command gate present with recorded bite-tests; the source cluster is named in the findings).]

The point is that an AI agent operating on a repository can, with one over-eager command, do irreversible damage (a forced reset, a destructive clean). A pre-execution gate that *blocks the command rather than trusting the agent not to issue it* is the same construction-by-safety stance applied to the build environment itself. [*Source:* `consumption.md` *§4.6.*]

3.4 The contributor rules as a soft layer

The published repository's own contributor rules forbid weakening the safety layers — no raw shell, no weakening the gates, no hardcoded paths, no hardcoded secrets. [*Source:* `capabilities.md` *§6, citing* `AGENTS.md` *"things to avoid."* — **VERIFIED** (read in the public repo's contributor file).]

> **Auditor caution.** These rules are a **soft** layer — a written norm an AI agent or human contributor is asked to honor — that *complements but does not replace* the hard, server-side mechanical enforcement (the allow-listed action runner, the autonomy band that cannot auto-cross a hard gate, the append-only gate audit, the byte-exact revert loop). The source is explicit about this distinction, and an honest methodology must keep it: the credible safety property is the mechanical one; the contributor rule is a reinforcing convention. [*Source:* `capabilities.md` *§6 "Why it matters" note.*]

---

4. Verification practice

4.1 The quality-gated retention loop (the safety rail for unattended improvement)

The single most important verification mechanism for AI-written, self-modifying code is the **quality-gated retention loop**, and it ships in the public repository. A change to the system is **KEPT only if a metric suite does not degrade**; otherwise it is **auto-reverted byte-exact** to the baseline, with the revert sha256-verified against the original snapshot. It is **fail-closed**: if any metric fails even to *run* at decide-time, the change reverts; a run can be decided only once; every decision is appended to an audit. [*Source:* `capabilities.md` *§9, citing* `retention.py` *docstring and routes* `/api/retention`, `/api/retention-decide`. — **VERIFIED** (retention loop present and inspectable in the public repo).]

This is the mechanism that makes "let an AI improve the system unattended" a defensible claim rather than a hopeful one. The flow is concrete: `baseline(targets)` takes byte copies and records baseline metric scores → the loop (a job, a scheduler, or a human) edits the targets → `decide(run_id)` re-runs the *same* metric suite and compares — KEEP if no gating metric degraded (ties allowed, improvement welcome), REVERT (restore the snapshot bytes exactly) if any gating metric degraded. [*Source:* `capabilities.md` *§9.* — **VERIFIED.**]

> **Scope honesty.** The source is careful to distinguish this from a separate, wider-platform concept — a *failure-signature regression memory* (recording known failure patterns so they are recognized instantly on recurrence). That named subsystem is **NOT present as a distinct module in the public repo.** What is shipped and verifiable is the metric-gated KEEP/REVERT retention loop above. An auditor must not conflate the two. [*Source:* `capabilities.md` *§9, "Relation to a failure-pattern regression memory";* `_DIGEST.md` *§2, capability 9.* — retention loop **VERIFIED**; failure-signature memory **UNVERIFIED in repo**.]

4.2 The acceptance-test tallies

The build practice produced several concrete, counted acceptance runs. These are the most audit-friendly figures in the whole pack because each is a discrete pass/fail tally with an inspectable mechanism:

Test surface	Result	Label
Multi-site generation acceptance (each: spec valid + 4/4 pages + working contact	**14/14 PASS**	PARTIAL

Test surface	Result	Label
form + export-cleanliness scan + live HTTP 200)		
Quality-evaluation harness: the original five categories (memory recall, capability selection, path resolution, security decision, document generation), deterministic graders, no human-in-the-loop	**5/5 categories at 100%**; the full expanded suite clears its ≥70% gate at **~79%** (failures concentrated in newer framing/discipline categories, not the original five)	PARTIAL
Aggregate health smoke-test across subsystems	**26/28** on the current expanded suite (the older "19/19" denominator is a strict subset)	PARTIAL
Mechanical safety hooks present + parsing + running	**8/8** (one "firing-but-blind" hook found and fixed in the same pass)	PARTIAL

[*Source:* `metrics.md` *§4.* — every row **PARTIAL**: the *mechanisms* (deterministic acceptance scripts, the re-runnable harness, the hook self-audit) are inspectable, but most specific runs live in internal logs; full external reproducibility requires running the bundled tests on a clean checkout.]

Two details in that table are themselves methodology signals worth naming:

- The health smoke-test is reported against its **real, larger denominator (26 of 28 on the current expanded suite), not the smaller, older "19/19" subset that would read cleaner.** Reporting the honest denominator and the failing checks rather than cherry-picking the flattering subset is a disclosure-discipline choice. [*Source:* `metrics.md` *§4.*]
- The hook self-audit found a hook that was **firing but blind** (running but scanning nothing) and **fixed it in the same pass** — an error left visible in the record rather than concealed. [*Source:* `metrics.md` *§4;* `_DIGEST.md` *§3.3.*]

4.3 The public claim register (verification of the verification)

The strongest single signal of verification discipline is the pack's own **claim register**: of **12 publicly-tracked product claims, 4 are labeled VERIFIED, 8 PARTIAL, 0 UNVERIFIED.** Crucially, the four VERIFIED were the ones whose evidence was **actually run** on the collection date:

1. File-based memory survives a restart.
2. The human-approval gate mechanism functions.
3. BYOM model routing works against a standard-shape stand-in.
4. The deterministic export/redaction gate bites — 3/3 fixtures flagged, zero leaks.

[*Source:* `metrics.md` *§4 auditor note;* `_DIGEST.md` *§3.4;* `gpt-competitors.md` *§1.2.* — **VERIFIED** (read directly from the pack's claim register and verdict).]

The eight PARTIAL claims were **honestly down-labeled** rather than asserted. For an auditor, a system that **down-labels its own claims** — reserving VERIFIED for what it actually ran — is exhibiting the exact verification posture you want from AI-built software, and it is the opposite of the over-claiming that AI-generated marketing tends toward. [*Source:* `metrics.md` *§8, point 4.*]

---

5. Deterministic-scripted versus reasoned: the dividing line

A recurring engineering decision in an AI-built system is **what to leave to the model's reasoning versus what to freeze into a deterministic script.** The SUR's practice draws this line deliberately, and the line is visible in the published code.

5.1 The principle: code over tokens where determinism is possible

The build philosophy treats deterministic logic as **cheaper, faster, and repeatable** than asking a model to reason it out each time, and pushes anything that *can* be deterministic into a script. The published artifacts bear this out: the safety-critical and the countable are scripted, while open-ended judgment is left to the model under a gate. [*Source:* `capabilities.md` *throughout (the server modules are pure stdlib Python scripts);* `consumption.md` *§3 (the running code is independent stdlib Python, not vendor TypeScript).* — **VERIFIED** structurally.]

5.2 Where the line falls

Concern	Implementation	Why this side of the line
The autonomy verdict (act / report / queue)	Deterministic — a small readable function composing level L0–L5 with a 4–20 risk score	A safety verdict must be **identical every time** and **auditable**; reasoning it out per-call would be both slower and non-reproducible. [`capabilities.md` *§5.*]
The gate queue	Deterministic — append-only `gate-queue.jsonl` , never rewritten	An audit trail's value is that it **cannot be re-reasoned after the fact.** [`capabilities.md` *§5.*]
Quality retention KEEP/REVERT	Deterministic — metric-suite comparison + sha256-verified byte revert	A revert that "decides" via reasoning could rationalize keeping a regression; a byte-exact comparison cannot. [`capabilities.md` *§9.*]
The token meter	Deterministic — counts are ground truth (parsed), priced at configured rates	Cost accounting must be **arithmetic, not estimated.** [`capabilities.md` *§8;* `metrics.md` *§1.*]
Edit safety (read-before-edit check, symbol boundary, syntax gate, classifier)	Deterministic — stdlib Python checkers and an AST parser	Catching a stale or malformed edit is a **mechanical invariant**, not a judgment call. [`consumption.md` *§4.2–§4.4.*]
The chat conversation itself	Reasoned — routed to the operator's chosen model with one disclosed system line	Open-ended dialogue is exactly the work a language model *should* do. [`capabilities.md` *§1.*]
Drafting / synthesis / design	Reasoned — the lead and cross-check models	Generative and analytic judgment is the model's proper domain, **under** the gates above.

The pattern is consistent and is the methodological heart of the system: **reasoning is used for what reasoning is good at (open-ended generation and judgment) and is fenced by deterministic scripts for everything safety-critical, countable, or required to be reproducible.** The single small (~216-line) BYOM router file — one disclosed system line, no hidden prompt, a config edit to swap models — is the clean embodiment of "keep the reasoned surface small and inspectable." [*Source:* `capabilities.md` *§1, §5, §8, §9;* `consumption.md` *§3.* — **VERIFIED** structurally.]

> **Why this is the right line for AI-built software.** The known failure modes of AI-written code (§1) all live on the *reasoned* side — fabrication, drift, stale context, plausible-but-wrong logic. Moving every safety-critical and countable concern to the *deterministic* side removes those failure modes from exactly the places where they would do irreversible harm, and confines the model's reasoning to the places where a wrong answer is cheap to catch and revert. [*Auditor synthesis of* `capabilities.md` *§1, §5, §9;* `metrics.md` *§8.*]

---

6. The cost and scale of the build itself

For completeness, the build's own resource footprint — every figure API-equivalent cost-to-serve, not cash:

Metric	Value	Label
Total tokens across the build (all four classes)	**~10.53 billion**	VERIFIED (sum of four ledger totals)
Recorded messages	**19,333**	VERIFIED
Transcript files aggregated	**556**	VERIFIED
Span of activity	**2026-05-16 → 2026-06-03 (19 calendar days)**	VERIFIED
Cache-read tokens (the dominant term)	**10.07B ≈ 93% of all volume**, priced at the cheapest tier	VERIFIED
API-equivalent cost-to-serve	**$26,920.27**	PARTIAL (rate-dependent; cost-to-serve, NOT cash)
Blended rate	**≈ $0.00256 per 1,000 tokens**	VERIFIED (derived)

[*Source:* `metrics.md` *§1.*]

Two honesty notes the source insists on, carried here verbatim in substance:

- **The widely-quoted "~$10K" figure is stale.** It is the 2026-05-24 snapshot (8,004 messages / $10,047.89). The current cost-to-serve over the full window is **~$26.9K**. Cite the current figure; note the ~$10K is the older snapshot. [*Source:* `metrics.md` *§1c.* — **VERIFIED** (both snapshots present in the history log).]
- **Cache-read dominance is why the blended rate is low.** ~93% of the volume is re-reading a cached, stable context at roughly a 10× discount versus fresh input. This is a structural property of a long, iterative build session, not a special efficiency claim. [*Source:* `metrics.md` *§1b.* — **VERIFIED.**]

The methodological reading: a ~10.5-billion-token, 19-day, single-operator build is a **high-throughput AI-assisted construction**, and the cost-to-serve figure is best understood as the cost-basis a bring-your-own-model buyer would face running the same volume on metered access — not as cash the operator spent on a subscription. [*Source:* `metrics.md` *§0 caveat 1, §8 point 1.*]

---

7. Honest limits of this methodology audit

Carried into this document so the deliverable's integrity is visible:

1. **The private build method is not disclosed and was not reverse-engineered.** This audit describes the *generic* engineering practice (the loop, the gates, the verification, the deterministic/reasoned split). The orchestration intelligence and the private prompts are deliberately withheld by the operator and are out of scope. [*Source:* `consumption.md` *§5;* `_DIGEST.md` *§7.* — by design.]
2. **Most specific build runs live in internal logs.** The acceptance tallies (14/14 sites; the original-five eval categories at 100% / full-suite ~79% gate-cleared; 26/28 smoke; 8/8 hooks) and the ~260 iteration index are real and the mechanisms are inspectable, but full external reproducibility requires a clean-checkout run of the bundled tests. The auditor read the records; the auditor did not re-execute every run this pass. [*Source:* `metrics.md` *§3, §4, §8 point 2.* — **PARTIAL.**]
3. **"No vendor code" is verified structurally, not by a token-level diff.** The consumed-pattern build practice keeps vendor source as segregated reference and runs independent stdlib Python; a full clean-room diff of every script against every reference file was not performed. [*Source:* `consumption.md` *§7, first bullet.* — **PARTIAL.**]
4. **The cross-check routing is named, not exhaustively logged.** The four models are verified as *used*; per-iteration cross-check routing is not fully traceable in the public pack. [*Source: header lines;* `gpt-competitors.md` *Part 1.* — **PARTIAL.**]

5. **AI-assisted authorship is a risk profile, not a quality grade.** The gates and verification practice described here reduce the known failure modes; they do not eliminate them, and a "pass" is scoped to what was tested. [*Source:* `metrics.md` *§5 honesty limit.*]
6. **The asymmetry, once more.** One person with AI assistance over ~19 days. The discipline is the finding; the breadth of a funded team's verification apparatus (large CI suites, coverage gates, a paid QA function) is not present and is not claimed. [*Source:* `_DIGEST.md` *§8.1;* `metrics.md` *§0 caveat 2.*]

---

8. Auditor's net assessment

The methodology survives audit at its stated scope. The system under review is **AI-built, and honest about it** — the rarer and more trustworthy posture. The build practice does not ask anyone to trust the AI's output; it places **deterministic, fail-closed gates between AI output and the shipped artifact**, applies a **multi-model, de-correlated cross-check** that reduces within-family blind spots, **publishes a stdlib-only reproducibility harness (`verify_all.py`, 8/8 PASS) so a reviewer can execute the load-bearing safety properties without trusting any model's word**, **pairs each build step with a verification step**, and draws a **deliberate line** between what is left to reasoning (open-ended generation and judgment) and what is frozen into deterministic scripts (everything safety-critical, countable, or required to be reproducible). The verification posture is visible in the published code (the byte-exact, fail-closed retention loop) and in the disclosure discipline (a claim register that reserves VERIFIED for what was actually run and down-labels the rest).

The honest limits are narrow and named: the private method is withheld by design, most specific runs live in internal logs pending a clean-checkout reproduction, the "no vendor code" property is structural rather than diff-proven, and the whole thing is the work of one researcher with AI assistance — notable for throughput, expected to have un-finished edges. Nothing in this methodology supports "beats frontier," "fully sovereign," "AGI," or any unbacked superlative, and none is claimed. The defensible statement is the precise one: **this is a disciplined, gate-first, multi-model human-plus-AI build practice that treats AI-written code as untrusted input and verifies before it trusts.** [*Sources:* `capabilities.md` *§1, §5–§9;* `consumption.md` *§2–§7;* `metrics.md` *§1, §3–§6, §8;* `_DIGEST.md` *§1–§3, §7–§9.*]

Sources

- `_DIGEST.md` — the sealed synthesis: voice/honesty contracts (lines 9–22), the structural-defensibility spine (§1), the capability/verification register (§2–§3), the consumption posture (§7), and the honest-limits list (§8).
- `capabilities.md` — the published-repo capability inventory: BYOM router (§1), human-in-the-loop gates (§5), mechanical safety hooks (§6), file-based memory (§7), audit surface + token meter (§8), the quality-gated retention loop (§9), and the stdlib-only local server (§10), each with VERIFIED/PARTIAL/UNVERIFIED status and repo citations.
- `consumption.md` — the consumption audit: the four-question intake gate and six-step pattern (§2), the consumed-vs-built boundary (§3), the catalogue of consumed patterns including the edit-safety stack and trajectory-eval (§4.2–§4.6), the asymmetric outbound posture (§5), and the honest gaps (§7).
- `metrics.md` — the honest figure extraction: token volume + API-equivalent cost-to-serve and the stale-vs-current snapshot (§1), capability-module inventory (§2), build iterations (§3), acceptance-test tallies and the public claim register (§4), the public reproducibility harness (§5), the small-model fine-tune deltas (§6), and the buyer-facing summary (§8).
- **Open-source pattern sources named in the consumption audit and corroborated as real public projects:** SWE-agent (MIT, the ACI edit-safety pattern); agentevals (MIT, trajectory evaluation); the Pi.dev coding-agent harness ecosystem (context engine, code-intelligence "lens" modules, permission-gate cluster).
- **Build assistants disclosed in every header:** Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks.

Metrics — Honest, Labeled, and Scoped

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

How to read this document

This is the numbers chapter of a structured technical audit of the system under review (the "SUR" or "the platform"), applying a labeled-evidence standard. It exists to do one thing well: present the platform's quantitative claims **honestly** — every non-trivial figure carries an explicit confidence label, every cost figure carries the API-equivalent caveat, and every multiplier states its scope. (On what "independent" means for this audit — the evidence standard, not the auditor's relationship to the subject — see `00-executive-summary.md` and `08-limits-and-reproducibility.md` §6.)

Auditor integrity is the deliverable here. A metrics chapter that flatters the subject is worthless; a metrics chapter that down-labels its own evidence where the proof is thin is, paradoxically, the more trustworthy artifact. The source materials already do this self-down-labelling (see §6), and this chapter preserves it.

The three labels

Label	Meaning
VERIFIED	Read directly from a machine-generated ledger or a test artifact a reviewer can re-run or re-read. The figure is the raw datum, not an estimate or projection.
PARTIAL	The figure is real but is scoped to a subset, depends on a configured rate, or lives in an internal artifact not bundled in the public materials. The *mechanism* is usually inspectable; the *specific run* is not externally reproducible without a clean checkout.
UNVERIFIED	Described, planned, or projected — not independently confirmable from the materials examined.

Two caveats that apply to every figure in this chapter

1. **Cost is API-EQUIVALENT cost-to-serve, NOT cash.** The platform was operated largely under a flat-rate subscription. Every dollar figure is `token counts × published per-token list prices` — i.e. *what this volume would cost a buyer running it on metered API access*. That equivalence is the useful number (it is the cost-basis a bring-your-own-model customer would actually pay), but it is not money that left a bank account. (Source: `metrics.md` §0 caveat 1; `_DIGEST.md` §8.4.)
2. **The asymmetry must be weighted.** This is the output of **one person with AI assistance over ~19 recorded calendar days**, not a funded team. Read the figures as evidence of throughput for a solo-plus-AI operation, not as parity with a staffed competitor. Competitor maturity is a roadmap to learn from, not a deficit to spin. (Source: `_DIGEST.md` §8.1.)

---

1. Token volume and API-equivalent cost-to-serve

This is the most solid evidence in the entire audit: a machine-generated usage ledger, regenerated 2026-06-03, aggregating **556 session transcript files**. Token counts are ground truth (parsed directly from transcripts); cost is `counts × configured list rates`. (Source: `metrics.md` §1.)

Metric	Value	Label	Notes
Transcript files aggregated	556	**VERIFIED**	Ledger `transcript_files`.
Recorded messages	19,333	**VERIFIED**	Ledger `totals.messages`.
Input tokens	15,784,495	**VERIFIED**	Ledger `totals.in`.
Output tokens	55,583,873	**VERIFIED**	Ledger `totals.out`.
Cache-write tokens	396,640,751	**VERIFIED**	Ledger `totals.cache_write`.
Cache-read tokens	10,065,077,885	**VERIFIED**	Ledger `totals.cache_read` — the dominant term, ~93% of all tokens.
Total tokens (all four classes)	**~10.53 billion**	**VERIFIED (sum)**	15.78M + 55.58M + 396.64M + 10,065.08M = 10,533,086,004. Arithmetic on four ledger totals.
API-equivalent cost-to-serve	**$26,920.27**	**PARTIAL**	Ledger `totals.cost_usd`. PARTIAL because it depends on the configured list rates (§1.1) and is cost-to-serve, NOT cash (see caveat 1 above).
Span of activity	2026-05-16 → 2026-06-03	**VERIFIED**	19 calendar days; ledger `by_day` keys, first to last.

1.1 The configured rates behind the dollar figure (USD per million tokens)

Model class	in	out	cache-write	cache-read	Label
Opus	15.00	75.00	18.75	1.50	VERIFIED (as configured)
Sonnet	3.00	15.00	3.75	0.30	VERIFIED (as configured)
Haiku	1.00	5.00	1.25	0.10	VERIFIED (as configured)

> **Stated assumption.** These are the rates the ledger applied. A reviewer should re-check them against current published pricing; if list prices move, the dollar total moves proportionally — **the token counts do not change.** (Source: `metrics.md` §1a.)

1.2 Why cost-to-serve is heavily discounted — the cache-read dominance

Cache-read tokens (10.07B) are ~93% of total volume and are priced at the cheapest tier (1.50/M for the dominant model class versus 15.00/M for fresh input — a **10× discount**). This is the structural reason the blended dollar-per-token rate is low: most of the volume is re-reading a cached, stable context rather than paying fresh-input rates. (Source: `metrics.md` §1b; `_DIGEST.md` §3.1.)

- **Blended rate check:** $26,920.27 ÷ 10,533M tokens ≈ **$0.00256 per 1,000 tokens** blended. **VERIFIED (derived)** — division of two VERIFIED ledger totals. *Assumption:* treats all four token classes as a single pool; it is a blended average, not a per-class rate.

1.3 Two consistency cross-reads

- The ledger's per-model sub-totals for the two primary model versions sum to $21,067.39 + $5,794.27 + minor others ≈ the $26,920.27 grand total — **internally consistent. VERIFIED (cross-read).** (Source: `metrics.md` §1c.)
- An earlier ledger snapshot (2026-05-24) recorded **8,004 messages / $10,047.89** API-equivalent. The current snapshot (19,333 messages / $26,920.27) is a superset of the same accounting method over a longer window.

> **The "~$10K" figure is stale.** The widely-quoted ~$10K is the **earlier** snapshot. The current cost-to-serve is **~$26.9K**. A reviewer should cite the current figure and treat any "~$10K" reference as superseded. **VERIFIED (both snapshots present in the history log).** (Source: `metrics.md` §1c; `_DIGEST.md` §3.1.)

---

2. Capability-module inventory

The platform is organized as a library of composable capability modules (each a self-contained instruction file plus optional deterministic scripts), rather than a single monolithic prompt. (Source: `metrics.md` §2.)

Metric	Value	Label	Notes
Counted capability modules	165	**PARTIAL**	The platform's own count-reconciler asserts 165 modules (a module = a directory containing a manifest file; staging/infrastructure excluded). PARTIAL: the auditor did not enumerate all 165 directories on disk this pass; the figure is the system's self-asserted, reconciler-guarded count. A reviewer can re-derive it by counting manifest files.
Architectural-core modules (closed inner set)	7	**PARTIAL**	Declared closed; same provenance caveat.
Modules in the *public* open-source shell	subset (not the full 165)	**PARTIAL**	The public repo ships the agent shell plus the gate/queue interface, not the full private module set. The exact public count was not enumerated this pass.

> **Counts are not validation.** "165 modules" is an inventory of *capability files*, not a claim that all 165 are individually battle-tested. A meaningful fraction carry an "encoded but not yet independently validated" status. Inventory ≠ all-validated. (Source: `metrics.md` §2; `_DIGEST.md` §8.5.)

---

3. Documented build-and-verify iterations

The platform records its build history as a numbered sequence of **build-and-verify iterations** — each one a change, a self-test step, and a logged outcome. (Source: `metrics.md` §3.)

Metric	Value	Label	Notes
Highest iteration index referenced	~260	**PARTIAL**	Internal records reference iteration indices into the ~260 range. PARTIAL: from internal logs, not reproducible from the public pack; it is an **upper-bound index**, not a guarantee that every index in 0..260 is a fully-logged, independently-verifiable event.

Metric	Value	Label	Notes
Iterations citing an in-line self-test result	many (dozens sampled)	**PARTIAL**	A sample of the internal log shows iterations routinely paired with a pass/fail self-test line (examples in §4). Not exhaustively audited.

> **Honest reading.** The *practice* of pairing each build step with a verification step is evident and consistent across the sampled records. The exact total of fully-reproducible iterations was not independently counted. (Source: `metrics.md` §3.)

---

4. Acceptance-test pass rates — the strongest reproducible-shape evidence

These are concrete, counted test outcomes recorded in build artifacts. They are the most audit-friendly figures in the chapter because each is a discrete pass/fail tally and each underlying mechanism is inspectable. (Source: `metrics.md` §4; `_DIGEST.md` §3.3.)

Test surface	Result	Label	Notes
Multi-site generation acceptance run	**14/14 PASS**	**PARTIAL**	A batch of 14 generated sites; each asserted on: spec valid + 4/4 pages present + a working contact form + an export-cleanliness scan + a live HTTP 200 from a local server. PARTIAL: the 14/14 is recorded in an internal build log; a reviewer reproduces by re-running the bundled generator. The deterministic acceptance script is inspectable.
— spec-validation sub-step	14/14 PASS	PARTIAL	Same run, validation gate.
— page-presence sub-step	4/4 pages × 14 sites	PARTIAL	Same run.
— live-serve sub-step	14/14 returned HTTP 200	PARTIAL	Same run, against a local HTTP server.
Quality-evaluation harness	**original 5 categories at 100%**; full expanded suite clears its **≥70% gate at ~79%**	**PARTIAL**	The original five categories — memory recall, capability selection, path resolution, security decision, document generation — each pass at 100%; the suite has since GROWN (added framing/discipline categories), and on that larger suite it clears its ≥70% gate at ~79% with the failures concentrated in the newer, harder categories (a few of which are missing-grader / timeout harness gaps rather than content misses). Deterministic graders, no human-in-the-loop. PARTIAL: recorded

Test surface	Result	Label	Notes
			internally; the harness is inspectable and re-runnable on a clean checkout. The older "20/20" is the smaller, earlier 5-category suite — not comparable apples-to-apples.
Aggregate health smoke-test	**26/28** checks on the current expanded suite	**PARTIAL**	A one-command health check across subsystems. The suite GREW from a 19-check baseline to 28 (later hardening checks added); on the current 28-check suite it passes 26 (the 3 known failures are an expected-count lag and hardcoded-path lint items, all pre-existing drift). The older "17–19/19" is a strict subset of the 28. PARTIAL: internal; the honest larger denominator is shown rather than the flattering subset.
Mechanical safety hooks present + parsing + running	8/8 hook scripts	**PARTIAL**	A self-audit reported all 8 enforcement hooks present, parsing, and executing — with one "firing-but-blind" hook found and fixed in the same pass (an honest self-catch, not concealed). PARTIAL: internal self-audit.

4.1 The public claim register — a metric of disclosure discipline

The public pack tracks 12 product claims and labels them itself:

Register tally	Value	Label
Claims VERIFIED	**4**	VERIFIED (read from the register)
Claims PARTIAL	**8**	VERIFIED (read from the register)
Claims UNVERIFIED	**0**	VERIFIED (read from the register)

The 4 VERIFIED claims — file-based memory survives a restart; the human-approval gate mechanism; model-routing against a standard-shape stand-in; a deterministic export/redaction gate biting 3/3 fixtures with zero leaks — were the ones whose evidence was **actually run** on the collection date. The fact that the pack down-labels its own 8 unrun claims rather than promoting them is itself a positive trust signal. **VERIFIED (read directly from the pack's claim register and verdict).** (Source: `metrics.md` §4 auditor note; `_DIGEST.md` §3.4.)

---

5. The public reproducibility harness — the strongest freshly-publishable proof

The single most checkable artifact in this whole pack is **not** an internal log: it is a **public, stdlib-only, network-free verification harness shipped in the open repository** (`scripts/verify_all.py` in the published Apache-2.0 shell). Any reviewer can clone the repo and run it on a fresh checkout with no API keys; it exits `0` only when **all eight** of the public claims it checks pass.

Check	What it verifies	Result
V001	At maximum autonomy (L5) a high-risk action still **queues** (the hard gate)	PASS
V002	An invalid autonomy score **fails closed** to queue	PASS
V003	The fast-verify queue records **append-only** pending + decision events	PASS
V004	BYOM reports **honestly** when no model config is present (no fabricated reply)	PASS
V005	Metric-gated retention **KEEP/REVERT** self-test passes (byte-exact revert)	PASS
V006	The action runner is **allow-listed** (unknown action refused, nothing executed)	PASS
V007	Selected HTTP API routes return **HTTP 200 JSON**	PASS
V008	Secret config patterns are **gitignored**	PASS

Result: PASS (8/8), exit 0 — re-run 2026-06-03 on a clean checkout. (Source: the published `scripts/verify_all.py` + its `verification/last-run.json` report.) — **VERIFIED (executed on the public repo; reproducible by any reviewer).**

This is the proof an outside reviewer should run first, because it converts the audit's spine — *the gate still queues a high-risk action at maximum autonomy, the runner is allow-listed, retention reverts byte-exact, memory degrades honestly* — from an inspection claim into an **executed** one, against published code, with no AI judgment in the loop. It is the reproducibility artifact this metrics chapter leans on, in place of any internal run.

> **Honest scope.** Eight checks are not a full safety certification — they are the public claims that can be checked on a fresh clone without API keys. A PASS means *these eight properties hold on the published code*, not "the system is safe" in the absolute. The deeper acceptance tallies (§4) and the roadmap capabilities remain labelled as in §1–§4.

---

6. Small-model fine-tune deltas — where candor is the finding

The platform ran a fine-tuning track to specialize a small open base model. These are the honest, measured deltas — including the early *negative* result, which is the most credibility-relevant figure in the whole chapter. (Source: `metrics.md` §6; `_DIGEST.md` §3.6.)

Base model: an open **~0.5B-parameter** instruct model. **Pass thresholds, pre-registered:** format-compliance **≥ +20pp** OR category-accuracy **≥ +15pp** versus the un-tuned base.

Run / config	Measured delta vs base	Pass?	Label	Notes
Early run (50 training pairs)	**≈ −20pp** (regression)	**FAIL**	**PARTIAL**	The first attempt made the small model *worse* — honestly recorded, not hidden. Loss converged high (~3.05), consistent with undertraining (3 epochs / 50 pairs / 0.5B

Run / config	Measured delta vs base	Pass?	Label	Notes
				/ CPU). Internal training report.
Mid run (~150 pairs)	**+13.3pp category accuracy** (first positive delta)	**Below** the +15pp category bar (and below the +20pp format bar)	**PARTIAL**	The first end-to-end *positive* result, proving the pipeline works (train → adapter → chat-template → local conversion). It did **not** clear the pre-registered pass threshold. Honest reading: pipeline-validated, target-not-yet-met. Internal eval artifact.
Projected at scaled data (~5,900 pairs)	predicted to clear the threshold	not yet run	**UNVERIFIED**	A plan/projection, not a measured result. Flagged UNVERIFIED per the auditor's own rule.

> **Auditor reading of this track.** The most honest signals here are the **−20pp first result** and the **+13.3pp that still missed the bar**. A vendor spinning this track would quote only the future projection; the records instead leave a real, partial, sub-threshold improvement *and* a clearly-labelled un-run projection in plain sight. **That candor is itself the finding.** This is a **capability-in-progress, not a shipped win** — never present it otherwise, and never quote the projection alone. (Source: `metrics.md` §6; `_DIGEST.md` §3.6 + §8.6.)

---

7. Scaling multipliers — every one scoped

This section consolidates the calculation-derived figures from the scalability model. The pattern is consistent: each multiplier is **real arithmetic on stated assumptions**, and each one is bounded by a scope caveat that an honest reader must carry alongside it. All dollar figures are API-equivalent cost-to-serve at the modelling rates (`P_in` $3/M, `P_out` $15/M, `P_cache_read` $0.30/M). (Source: `scalability.md` §1–§4.)

7.1 Caching savings

Figure	Value	Label	Scope caveat (load-bearing)
Saving off a representative *cached* turn	**~42%**	VERIFIED (arithmetic)	Scoped to a 16k stable-prefix / 4k delta / 2k output turn at hit-rate h=0.90. Output is full price and does not cache.
Saving off the *input bill* of that turn	~63%	VERIFIED (arithmetic)	Input-side only; the 16k/4k split is a stated assumption.
The headline "up to ~90% cheaper"	up to ~90%	VERIFIED (literature)	**Scope:** the input-token-only, high-hit-rate, large-stable-prefix case — NOT a whole-turn saving. Break-even is ~2+ cache hits per prefix; a prefix used once is a net loss (~25% write premium with no read amortization).

(Source: `scalability.md` §1.2; literature via `researchers.md` §7.)

7.2 The multi-agent multiplier — caching does not cancel it

Figure	Value	Label	Scope caveat
Multi-agent token multiplier vs single chat	**~15×**	VERIFIED (literature)	Anthropic's orchestrator-worker research figure (`researchers.md` §3).
15-worker cached fan-out cost-to-serve	**~$0.755** vs $1.35 naive	VERIFIED + PARTIAL	Caching ~halves the fan-out bill **but it is still ~14× a single cached turn (~$0.0523).** PARTIAL on the shared-prefix assumption: it requires workers genuinely share the same cached prefix, an implementation property.

> **Net statement (carry it verbatim).** Parallel orchestration buys **wall-clock speed and breadth by spending tokens**; caching reduces the premium but does not remove it. (Source: `scalability.md` §1.3.)

7.3 Cost-tier routing — a real but depreciating advantage

Figure	Value	Label	Scope caveat
Routing saving on a routine-heavy mix	**~3.0× cheaper**	VERIFIED (arithmetic)	Scoped to a fan-out where **12 of 15 workers are routine** and routine runs at a tier ~10× cheaper than the frontier tier.

> **The caveat that makes the 3× honest.** Inference prices are falling **9×–900× per year across tasks, median ~50×/yr** (Epoch AI, via `researchers.md` §7 — VERIFIED). The *gap* between cheap-tier and frontier-tier — the very thing routing monetizes — **shrinks over time.** The routing advantage is real **today** and is a **depreciating asset**, not a permanent defensibility. Any routing multiplier must be quoted with date + tier mix + the erosion note. (Source: `scalability.md` §1.4.)

7.4 Concurrency speedup — Amdahl saturation

Workers (at parallel fraction f=0.90)	Speedup	Label
10 workers	**~5.3×** (not 10×)	VERIFIED (Amdahl)
25 workers	**~6.9×**	VERIFIED (Amdahl)

> **The observed ~8–10 effective-worker cap is structural, not a tuning artifact.** Three independent forces converge on roughly that number: (1) the Amdahl synthesis tax — the lead's brief-composition + N-output synthesis is the serial `(1-f)` part; (2) the documented multi-agent failure mode where agents that do not share full context produce compounding, conflicting outputs, so synthesis cost rises **super-linearly** with N; and (3) context degradation — a lead synthesizing 25 reports loads a large context and hits the degradation curve that motivates keeping context small. (Source: `scalability.md` §2.2–§2.3; `researchers.md` §4–§5.)
>
> **Honest envelope.** The platform is inside the defensible zone **only** when workers run genuinely independent sub-tasks AND the lead receives full traces and *synthesizes* (rather than blind-concatenates). Whether the platform actually shares full traces is an implementation property — surface verified (per-agent append-only channels + a synthesizing lead), concurrent execution engine **not present in the public repo (PARTIAL/UNVERIFIED)**. (Source: `scalability.md` §2.3; `_DIGEST.md` §8.7.)

7.5 Reliability leverage

Practice	Figure	Label	Scope caveat
Health-check + auto-restart	**~98% → ~99.97%** availability	VERIFIED (formula)	"~2 nines from MTTR alone" at 24h MTBF, moving MTTR 30 min → 30 s. MTBF/MTTR are stated inputs.
Retry + backoff (3 attempts, p=0.9)	**90% → 99.9%**	VERIFIED (formula)	**Transient, independent errors only.** Long-horizon errors compound and are NOT independent — retrying a half-corrupted trajectory can make things worse; the long-horizon cure is checkpoint + gate, not retry.
Circuit breaker	**~15× less** wasted work per outage	VERIFIED (arithmetic)	Trip-after-3 vs 15 workers × 30s × 3 retries. Trip threshold + timeout are stated inputs.
Caching as load-shedding (h=0.8)	**5×** rate-limit headroom	VERIFIED (arithmetic)	At h=0.8 the system makes 1/5 the upstream calls. h taken from the cited 80–95% production band.
Multi-provider BYOM redundancy	`1 - (1-A)^k`	VERIFIED (formula) / PARTIAL (deployment)	The redundancy math **assumes independence.** Two providers in the same model family, or two "redundant" components on one machine, are correlated and do NOT deliver the math. The platform makes multi-provider fallback *possible* (config-swappable BYOM); whether a deployment is genuinely independent is the operator's configuration, not an intrinsic guarantee.

> **Convergence worth noting.** The rate-limit admission math (`W × calls_per_worker_per_min ≤ provider_RPM`) and the concurrency math **independently converge on ~8–15 workers** as the useful band. (Source: `scalability.md` §4.1–§4.6.)

---

8. The literature-grounded upside — measured elsewhere, not here

To keep the multipliers honest, it is worth stating plainly where the strongest positive evidence actually comes from: independent research, not the platform's own benchmarks.

Finding	Value	Label	Whose number
Multi-agent vs single-agent uplift	**+90.2%** on an internal eval	VERIFIED	Anthropic's orchestrator-worker research system — at ~15× tokens, with token usage explaining ~80% of performance variance. NOT the platform's own benchmark. (`researchers.md` §3.)

Finding	Value	Label	Whose number
Context-editing token reduction	**84%** over a 100-turn eval	VERIFIED	Anthropic context-engineering research; context editing alone +29% lift, with a memory tool +39%. Validates the platform's context-discipline approach in principle. (`researchers.md` §5.)
Prompt-caching cost / latency cut	up to **~90% / ~85%**	VERIFIED	On long stable prefixes; 80–95% production hit rates. (`researchers.md` §7.)
Inference price decline	**~50×/yr** (median; range 9×–900×)	VERIFIED	Epoch AI — the reason cost-tier-routing advantages depreciate. (`researchers.md` §7.)

> **The honest reading.** The multi-agent upside is **real and measured — elsewhere.** The platform adopts the same pattern, so its expected upside is "plausibly large on breadth-heavy tasks, at a ~15× token premium" — but that is **NOT a number the platform has independently benchmarked** in the public sources. For the platform specifically, the uplift figure is **UNVERIFIED**; what is verified is that its scaling *choices* ride documented curves. (Source: `scalability.md` §2.4.)

---

9. One-line honest summary

Dimension	Headline figure	Strongest honesty caveat
Token volume	~10.53B tokens / 19,333 msgs / 556 files / 19 days	VERIFIED raw counts.
Cost-to-serve	**~$26,920** API-equivalent	NOT cash (subscription); rate-dependent; ~93% is cheap cache-read; the older "~$10K" is stale.
Capability modules	165 (7 core)	Self-asserted, reconciler-guarded count; inventory ≠ all-validated.
Build iterations	up to ~260 indexed	Internal log; upper-bound index, not all reproducible from the public pack.
Acceptance tests	14/14 sites · eval original-5 100% / full-suite ~79% gate-cleared · 26/28 smoke · 8/8 hooks	Mechanisms inspectable; specific runs largely internal.
Public claim register	**4 VERIFIED / 8 PARTIAL / 0 UNVERIFIED**	The 4 VERIFIED were actually run; the 8 are honestly down-labelled.
Public reproducibility harness	**8/8 PASS** (`verify_all.py`, exit 0)	Stdlib-only, network-free, runnable on a fresh clone with no keys; eight public claims, not a full safety certification.
Small-model fine-tune	**−20pp → +13.3pp** (still sub-threshold)	A capability-in-progress; the scaled projection is UNVERIFIED.
Caching saving	~42% off a cached turn (up to ~90% input-only)	Scoped: whole-turn vs input-only; break-even at ~2+ hits.
Multi-agent premium	~15× tokens	Caching ~halves the fan-out but it is still ~14× a single cached turn.
Cost-tier routing	~3.0× on routine-heavy mix	Depreciating as inference prices fall ~50×/yr; quote with date + mix.

Dimension	Headline figure	Strongest honesty caveat
Concurrency cap	~8–10 effective workers	10 workers ≈ 5.3×, not 10× (Amdahl); cap is over-determined and structural.

---

10. What an auditor would tell a buyer about these numbers

1. **The token/cost ledger is the most solid evidence** — machine-generated, internally consistent, two-snapshot history. Trust the **counts** as VERIFIED; treat the **dollar figure** as a configurable cost-to-serve (not cash), and quote the **current ~$26.9K**, not the stale ~$10K. (Source: `metrics.md` §8.1.)
2. **The acceptance-test tallies (14/14 sites; eval original-five at 100% / full-suite ~79% gate-cleared; 26/28 smoke; 8/8 hooks) are real and the mechanisms are inspectable**, but most specific runs live in internal logs. For full external reproducibility, the bundled tests must be run on a clean checkout — and the eight public gate/queue/retention/runner properties are *already* executable today via `scripts/verify_all.py` (§5). (Source: `metrics.md` §8.2.)
3. **The fine-tune track is honest about being unfinished.** Do not let −20pp → +13.3pp be read as a shipped capability, and never quote the ~5,900-pair projection on its own. (Source: `metrics.md` §8.3.)
4. **Every multiplier depreciates or is scoped.** Caching saves the input bill but not output; the ~15× fan-out premium is not cancelled by caching; the ~3× routing advantage erodes as inference prices fall ~50×/yr; concurrency saturates near ~8–10 workers. None of these is a permanent defensibility, and the audit does not present any of them as one. (Source: `scalability.md` §5.)
5. **The disclosure discipline itself is a positive signal.** A claim register that reads 4/8/0 with the 4 actually run, a smoke-test reported against its real larger denominator (26/28) rather than the flattering older subset, a "firing-but-blind" hook surfaced rather than hidden, and a −20pp regression left in the record — these are the marks of materials that down-label themselves rather than over-claim. That is worth more than any single multiplier. (Source: `metrics.md` §8.4; `_DIGEST.md` §3.4.)
6. **Weight the asymmetry.** This throughput is a solo operator with AI assistance over ~19 recorded days, not a funded team. That makes the volume notable and the un-finished edges expected. Competitor maturity (large star counts, shipped paid go-to-market, open-model production lines) is a roadmap to learn from — the platform's gap is **distribution and third-party trust proof, not engineering.** (Source: `metrics.md` §8.5; `_DIGEST.md` §8.1–§8.2.)

Forbidden framings (none used in this chapter, by design)

This audit makes no claim that the platform "beats frontier" systems, is "fully sovereign," or approaches "AGI," and uses no unbacked superlative. The platform CONSUMES open-source patterns one-way into its own architecture — it consumes them, it does not ship a harness. None of the source research claims otherwise, and this chapter keeps it that way. (Source: `_DIGEST.md` §8.8.)

Limits, Honesty Ledger & Reproducibility

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

Why this document exists

An audit is only as credible as its weakest unstated assumption. Every preceding chapter labels its claims, but those labels are scattered across six documents and eight research files. This chapter pulls them into one place and does three things an outside reviewer should demand:

1. **One consolidated honesty ledger** — every major claim in the audit, with its label (VERIFIED / PARTIAL / UNVERIFIED) and, for each PARTIAL or UNVERIFIED claim, the **specific unlock path** that would raise its label.
2. **A reproducibility boundary** — what an outside reviewer can confirm *right now* from the published materials versus what requires access not yet granted (a clean checkout, an internal log, a live API key, a not-yet-run experiment).
3. **The known limits, stated plainly** — small-local-model immaturity, the solo-builder asymmetry, and the go-to-market / third-party-trust-proof gap.

The intent is not to soften the audit. It is the opposite: a reviewer who can see exactly where the evidence stops is far more likely to trust the evidence that exists. The platform's own materials already practice this — they down-label their own claims where the proof was not run (metrics.md §4) — and this chapter holds the audit itself to the same standard.

How to read the labels

The labels are used exactly as the source research defines them (metrics.md §0):

Label	Meaning
VERIFIED	The figure or claim was read directly from a machine-generated ledger, an inspectable artifact, or public code/registry data a reviewer can re-read or re-run. It is the raw datum, not an estimate.
PARTIAL	The claim is real but scoped — it depends on a configured rate, holds only for a stated subset, rests on a stated assumption, or lives in an internal artifact not bundled with the public materials.
UNVERIFIED	Described, planned, or projected; not independently confirmable from the materials examined.

Two caveats sit underneath every cost figure in the whole audit and are not repeated at each line below (metrics.md §0, §1):

- **Cost is API-equivalent cost-to-serve, NOT cash.** The platform was operated largely under a flat-rate subscription. Dollar figures are token counts × published per-token list rates — "what this volume would cost a buyer on metered API access." That equivalence is the *point* (it is the cost-basis a bring-your-own-model customer would pay), but it is not money that left a bank account.
- **The asymmetry is weighted once, plainly.** This is the output of **one person with AI assistance over ~19 recorded days** (_DIGEST.md §8), not a funded team. Figures are evidence of throughput for a solo+AI operation, not parity with a staffed competitor. Competitor maturity is a roadmap to learn from, not a deficit to spin.

1. The consolidated honesty ledger

Every major claim the audit makes, grouped by chapter, with its label and the evidence source. This is the single table a skeptical reviewer should start from.

1.1 The structural thesis (the spine)

#	Claim	Label	Source
T1	The agent harness and the open `SKILL.md` capability format are now **commodities**; defensibility lives in the layers commodity harnesses lack by default.	**VERIFIED** (across three independent source files)	_DIGEST.md §1; openclaw.md §4; gpt-competitors.md §2.2
T2	The open `SKILL.md` standard is Anthropic-authored (published 2025-12-18); Microsoft (VS Code) and OpenAI (ChatGPT + Codex CLI) adopted it within ~48h; by ~2026-03 roughly 32 tools read the same format.	**VERIFIED**	openclaw.md §4; agentskills.io
T3	The SUR's defensible edge is **structural, not architectural** — gate layer, least-privilege execution, sealed/audited boundaries, fail-closed quality guard, disciplined consumption.	**VERIFIED structurally** (layers present and inspectable)	_DIGEST.md §1, §9; consumption.md §6

1.2 The shipped public artifact (capabilities)

The audited public repo is **Apache-2.0**: a React front end over a **pure-stdlib Python** HTTP server on `127.0.0.1:8911`, same-origin, bring-your-own-model (_DIGEST.md §2).

#	Capability	Label	Source
C1	BYOM model router (OpenAI-compatible `/chat/completions`; one small ~216-line stdlib file + config; one disclosed system line, no hidden prompt).	**VERIFIED**	_DIGEST.md §2 row 1
C2	Context-pressure policy (measures pressure, verifies survival pins, checkpoints).	**VERIFIED**	_DIGEST.md §2 row 2
C3	A **distinct** context-discipline sandbox-execute primitive.	**UNVERIFIED in repo**	_DIGEST.md §2 row 2, §8
C4	Allow-listed action runner (closed `ACTIONS` dict; unknown name returns the allowed list, executes nothing; not a raw shell).	**VERIFIED**	_DIGEST.md §2 row 3
C5	A general MCP tool layer.	**UNVERIFIED in repo**	_DIGEST.md §2 row 3, §8

#	Capability	Label	Source
C6	Multi-agent orchestration **surface** (agent grid, org chart, process-table overlay, per-agent channels).	**VERIFIED** (surface)	_DIGEST.md §2 row 4
C7	A concurrent multi-agent **execution engine** (the orchestration *surface* is present; the engine is not).	**UNVERIFIED in repo**	_DIGEST.md §2 row 4, §8
C8	Human-in-the-loop gate layer (autonomy dial L0–L5 × 4–20 risk → act/report/queue; **even at L5 a high-risk action still queues** to an append-only `gate-queue.jsonl`).	**VERIFIED**	_DIGEST.md §2 row 5
C9	Mechanical safety hooks (allow-list + hard gate + byte-exact revert loop; contributor rules forbid weakening them).	**VERIFIED**	_DIGEST.md §2 row 6
C10	File-based memory (curated surface reads only declared files, never globs; a missing file surfaces honestly as "missing").	**VERIFIED**	_DIGEST.md §2 row 7
C11	Audit surface + token meter (append-only audits; token tracker prices counts at configured rates; cost = API-equivalent, not cash).	**VERIFIED**	_DIGEST.md §2 row 8
C12	Quality-gated retention loop (KEEP only if no metric degrades, else byte-exact sha256-verified auto-revert; fail-closed).	**VERIFIED**	_DIGEST.md §2 row 9
C13	A broader **failure-pattern regression memory** (beyond the retention loop).	**UNVERIFIED in repo**	_DIGEST.md §2 row 9, §8
C14	Local server + reliability (stdlib-only, same-origin, every route degrades to a clean "unavailable" at HTTP 200, fail-closed health).	**VERIFIED**	_DIGEST.md §2 row 10

> **The honesty rail for this whole audit.** Four wider-platform capabilities — **C3, C5, C7, C13** — are **not present in the published repo**. They are catalogued as roadmap/integration claims, never as shipped. The published artifact is best characterised as a **verifiable, gate-first BYOM agent-OS shell**, and on that scope it delivers what it claims. The public shell should not be over-read as performing autonomous fan-out-and-merge. (_DIGEST.md §2 honesty rail, §8)

1.3 The numbers (metrics & evidence)

#	Figure	Label	Source
M1	~10.53B total tokens / 19,333 messages / 556 transcript files	**VERIFIED** (raw counts)	metrics.md §1

#	Figure	Label	Source
	/ 19 calendar days (2026-05-16 → 2026-06-03).		
M2	Cache-read tokens = 10.07B ≈ **93%** of all volume, priced at the cheapest tier (~10× discount vs fresh input) — why blended cost is low.	**VERIFIED**	metrics.md §1, §1b
M3	API-equivalent cost-to-serve = **$26,920.27**; blended ≈ $0.00256 / 1,000 tokens.	**PARTIAL** (rate-dependent; cost-to-serve, not cash)	metrics.md §1, §1b
M4	The widely-quoted "~$10K" is the **stale** 2026-05-24 snapshot (8,004 msgs / $10,047.89); cite the current ~$26.9K.	**VERIFIED** (both snapshots in the history log)	metrics.md §1c
M5	165 capability modules (7 architectural-core).	**PARTIAL** (self-asserted, reconciler-guarded; not enumerated on disk this pass)	metrics.md §2
M6	Up to ~260 indexed build-and-verify iterations.	**PARTIAL** (internal log; upper-bound index)	metrics.md §3
M7	14/14 multi-site generation acceptance (spec valid + 4/4 pages + working contact form + export-cleanliness scan + live HTTP 200).	**PARTIAL** (internal log; mechanism inspectable)	metrics.md §4
M8	Quality-evaluation harness: the original five categories pass at 100%; the full expanded suite clears its ≥70% gate at ~79% (failures concentrated in newer framing/discipline categories, not the original five).	**PARTIAL** (internal run; harness re-runnable)	metrics.md §4
M9	26/28 aggregate health smoke-test on the current expanded suite (the older "19/19" denominator is a strict subset; honest denominator + failing checks shown).	**PARTIAL**	metrics.md §4
M10	8/8 mechanical safety hooks present + parsing + running — with one "firing-but-blind" hook found and fixed in the same pass.	**PARTIAL** (internal self-audit)	metrics.md §4
M11	12 tracked product claims: **4 VERIFIED / 8 PARTIAL / 0 UNVERIFIED** — the 4 VERIFIED were the ones actually run on the collection date.	**VERIFIED** (read from the register)	metrics.md §4; gpt-competitors.md §1.2
M12	Public reproducibility harness (`verify_all.py`): 8/8 PASS, exit 0 — stdlib-only, network-free, runnable on a fresh clone with	**VERIFIED** (executed on the public repo)	scripts/verify_all.py; metrics.md §5

#	Figure	Label	Source
	no keys; verifies the gate hard-floor, fail-closed scoring, append-only queue, honest BYOM no-config, retention KEEP/REVERT, allow-listed runner, HTTP-200 routes, gitignored secrets.		
M13	Small-model fine-tune: 50 pairs → ≈ −20pp (regression, FAIL); ~150 pairs → +13.3pp (first positive, below the +15pp bar).	**PARTIAL**	metrics.md §6
M14	Fine-tune projection at ~5,900 pairs → predicted to clear the bar.	**UNVERIFIED** (not yet run)	metrics.md §6

1.4 Comparison & science

#	Claim	Label	Source
X1	OpenClaw security events — CVE-2026-25253 (CVSS 8.8 one-click RCE), command-injection / indirect-prompt-injection CVEs, a companion-service breach exposing ~1.5M auth tokens, a "shadow superuser" demonstration.	**VERIFIED** (events)	openclaw.md; gpt-competitors.md §2
X2	A staged human-in-the-loop defense layer raised sandbox-escape defense from a ~17% baseline into the 19%–92% range (Claude ~91.5%) — the academic remedy that *is* the SUR's claimed differentiator.	**PARTIAL** (exact single-paper percentages)	openclaw.md; researchers.md
X3	OpenClaw scale ~247K stars (2026-03-02 snapshot), reported ~355K by mid-2026; Hermes ~175K stars in <4 months; ECC 205,179 stars / 31,492 forks (2026-06-03 live API).	**VERIFIED** snapshots; "most-starred / beat React" framing **PARTIAL** (date-dependent)	openclaw.md; ecc.md; gpt-competitors.md §2
X4	ECC / SUR / OpenClaw all share the same solo or near-solo bus-factor; the real contrast is governance maturity (ECC ships Sponsors, a $19/seat/mo paid tier, ~1,000-test CI, an 80% coverage gate).	**VERIFIED**	ecc.md
X5	There is **no on-record GPT (or external-AI) critique or endorsement** of the SUR; GPT/OpenAI appear only as a request-shape compatibility target and an optional backend; the external-model	**VERIFIED**	gpt-competitors.md Part 1

#	Claim	Label	Source
	connector is a mock-driven shell with **no successful real call logged** ($0.00, zero tokens).		
S1	The SUR implements the named, public LLM-as-OS / "Software 3.0" paradigm (Karpathy) — it does not invent one.	**VERIFIED**	researchers.md; Karpathy
S2	Multi-agent FOR: Anthropic's orchestrator-worker system outperformed single-agent by 90.2% on its internal eval at ~15× the tokens, token usage explaining ~80% of performance variance.	**VERIFIED**	researchers.md; Anthropic
S3	Multi-agent AGAINST (steel-man): Cognition's "Don't Build Multi-Agents" — dispersed decisions across agents that don't share full context compound conflicting errors. The SUR is defensible **only insofar as it shares full traces + synthesizes** — an implementation property the audit must keep examining.	**VERIFIED** (the science); SUR's trace-sharing property is **PARTIAL**	researchers.md; Cognition; _DIGEST.md §8 item 7
S4	Context engineering: context editing alone = +29% lift; with a memory tool = +39%; in a 100-turn eval context editing cut tokens 84%. Degradation is universal across all major models.	**VERIFIED**	researchers.md
S5	Long-horizon reliability: per-step error rate rises as a task progresses and compounds; reliability is domain-stratified — the empirical backbone for the gate + per-step-verification argument.	**VERIFIED** (a ~12-hour-horizon figure is PARTIAL)	researchers.md; arXiv 2509.09677
S6	Inference economics: prompt caching cuts cost up to ~90% / latency up to ~85% on long stable prefixes; inference prices fall ~50×/yr (median, Epoch AI) — so the cheap-vs-frontier routing gap is a **depreciating** asset.	**VERIFIED**	researchers.md

1.5 Scalability math (all `$` = API-equivalent at stated rates)

#	Result	Label	Source
P1	Per-turn cost: uncached representative turn (20K in / 2K out) = $0.090; cached (16K stable prefix / 4K delta, h=0.90) ≈ $0.0523 → ~42% off the whole turn, ~63% off the input bill. The "~90%" headline is the input-only, high-hit, large-prefix case.	**VERIFIED** arithmetic; splits are **stated assumptions**	scalability.md; _DIGEST.md §6
P2	The 15× multi-agent premium is **not cancelled** by caching: a shared-prefix 15-worker cached fan-out ≈ $0.755 vs $1.35 naive — caching ~halves it but it is still ~14× a single cached turn.	**VERIFIED** + **PARTIAL** (shared-prefix assumption)	scalability.md
P3	Cost-tier routing ≈ ~3.0× cheaper on a routine-heavy mix (12 of 15 workers routine) — a **depreciating** advantage as prices fall ~50×/yr.	**VERIFIED** arithmetic (quote with date + tier mix + erosion note)	scalability.md
P4	Concurrency saturates (Amdahl): at f=0.90, 10 workers ≈ 5.3×, 25 workers ≈ 6.9×. The observed ~8–10 effective-worker cap is over-determined by the synthesis tax, the shared-context failure mode, and context degradation — structural, not a tuning artifact.	**VERIFIED**	scalability.md; _DIGEST.md §6
P5	Reliability math: auto-restart buys ~2 nines; retry+backoff handles transient errors only (long-horizon errors compound → need checkpoint+gate); a circuit breaker cuts outage waste ~15×; caching at h=0.8 gives ~5× rate-limit headroom; multi-provider BYOM removes the model SPOF **only if independently configured**.	**VERIFIED** formulas; **stated** inputs	scalability.md

1.6 Consumption posture

#	Claim	Label	Source
K1	The platform's posture is **disciplined, mostly-one-way consumption**: consume patterns from permissively-licensed projects, re-implement inside its own owned framework, keep the method private. Independence is visible — different language	**VERIFIED structurally**	consumption.md §6, §8; _DIGEST.md §7

#	Claim	Label	Source
	(stdlib Python vs vendors' TS/JS), segregated reference copies, running code that does not import the references.		
K2	A four-question intake gate (license check → value÷overhead ≥ 3× → framework-conflict check → paid-service-drift check) + a six-step pattern ending in a real test and human review.	**VERIFIED** (gate applied per-item in the ledger)	consumption.md; _DIGEST.md §7
K3	Consumed sources are permissive (SWE-agent, agentevals — both MIT; OpenClaw/Hermes pattern-only, no vendor code), with one source-available exception correctly scoped to pattern-only.	**VERIFIED**	consumption.md §7, §8
K4	The corrected headline: "consume freely; publish a verifiable shell + interfaces; never publish the method" — asymmetric, not literally zero-outbound.	**VERIFIED**	_DIGEST.md §7; consumption.md §8
K5	"No vendor code" is verified **structurally**, not by a token-level clean-room diff.	**PARTIAL**	consumption.md §7, §8
K6	A public NOTICE / attributions file is owed before redistribution; one source's license is fair-use rather than classic-OSI (a lawyer's eye recommended before redistributing its reference copy).	**UNVERIFIED** (recommend adding before publish) / **PARTIAL**	consumption.md §7
K7	Per-item smoke-tests exist (4/4, 5/5, 8/8 across edit-safety + eval items) but no single aggregate regression run of the consumed layer is captured.	**PARTIAL**	consumption.md §7

---

2. The PARTIAL → VERIFIED unlock paths

A PARTIAL label is not a dead end; it is a claim with a specified next step. This is the section a reviewer who wants to *raise* the audit's confidence should act on. Each row states the minimum action that would move the label.

Claim	Current label	Unlock path — the exact action that raises the label
M3 / M4 cost-to-serve $26.9K	PARTIAL	Re-check the configured per-token list rates (metrics.md §1a) against the providers' current published pricing. The **token counts are already**

Claim	Current label	Unlock path — the exact action that raises the label
		VERIFIED; only the dollar conversion is rate-dependent. Recomputing at confirmed current rates makes the figure VERIFIED-as-of-date.
M5 165 modules	PARTIAL	Enumerate the manifest files on a clean checkout (one directory-count command). The count becomes VERIFIED; note that count ≠ validation (see §4.2).
M6 ~260 iterations	PARTIAL	Publish the iteration index with per-entry self-test outcomes so each can be re-read. Until then it stays an upper-bound index, not a per-entry guarantee.
M7 14/14 sites	PARTIAL	Run the bundled deterministic acceptance generator on a clean checkout. The **mechanism is already inspectable**; a clean-room re-run moves the *result* to VERIFIED.
M8 eval (original-5 100% / full ~79%)	PARTIAL	Re-run the bundled quality-evaluation harness (deterministic graders, no human-in-the-loop). The harness and its grader self-test are inspectable; a fresh run moves the run to VERIFIED. Report the suite version with the score — the older "20/20" is the smaller 5-category suite.
M9 26/28 smoke	PARTIAL	Run the one-command health check on a clean checkout and report the snapshot against its real (28-check) denominator.
M10 8/8 hooks	PARTIAL	Re-run the hook self-audit on a clean checkout; the "firing-but-blind" fix is the kind of self-catch a re-run confirms.
M12 reproducibility harness	VERIFIED	Already runnable: clone the repo and run `scripts/verify_all.py` on a fresh checkout (stdlib-only, no keys); expect `PASS (8/8)`, exit 0. This is the audit's strongest freshly-publishable proof and needs no further unlock.
M13 fine-tune deltas	PARTIAL	Publish the training data, eval split, and run logs so the −20pp and +13.3pp deltas are independently reproducible against the open ~0.5B base.
M14 ~5,900-pair projection	**UNVERIFIED**	**Run the experiment.** This is the only honest way to move it — it is currently a projection, not a measured result, and must never be quoted as if it were one.
C3 / C5 / C7 / C13 roadmap capabilities	UNVERIFIED in repo	Ship them in the public repo (or a separately auditable component) with the same inspectable shape as the verified capabilities. Until then they are roadmap claims, not shipped facts.
K5 "no vendor code"	PARTIAL	Run a token-level clean-room diff between the owned stdlib

Claim	Current label	Unlock path — the exact action that raises the label
		implementations and the segregated reference copies; publish the diff summary. Structural independence is already shown; the diff makes it conclusive.
K6 NOTICE / attributions	UNVERIFIED / PARTIAL	Add a public NOTICE/attributions file naming the consumed sources and their licenses; obtain a license review for the one fair-use / source-available source before redistributing its reference copy.
K7 aggregate consumption regression	PARTIAL	Add a single bundled regression run that executes all per-item consumption smoke-tests together, so "the consumed patterns stay healthy" is one re-runnable command.
S3 SUR shares full traces	PARTIAL	Demonstrate, on the public shell, that multi-agent work shares full traces and synthesizes (rather than blind-concatenating) — the property that, per Cognition's steel-man, separates a defensible multi-agent design from a failure-prone one. (Bound by C7: the concurrent execution engine is not in the public repo.)
X2 sandbox-escape defense percentages	PARTIAL	Cite the specific academic paper(s) and quote their exact reported intervals; the *events* and the *direction* are VERIFIED, the exact single-paper percentages are not yet pinned.
Citations (S-rows, X-rows)	VERIFIED with internal pointer	Follow the public URL now surfaced in `docs/REFERENCES.md`; each external claim is checkable against its primary (Karpathy talk, Anthropic/Cognition posts, METR/Epoch/arXiv). No internal-pointer-only citation remains for an external claim.

> **The pattern across every unlock path:** almost all of them are a **clean-checkout re-run** or a **publish-the-artifact** step — not new engineering. The exception is **M14**, which genuinely requires running an experiment that has not been run. That distribution is itself a finding: the gap between PARTIAL and VERIFIED here is mostly *packaging and disclosure*, not capability.

---

3. Reproducibility — what an outside reviewer can do now vs not

3.1 Reproducible now, from the published materials

These require nothing beyond the public Apache-2.0 repo and public registry/API data. A reviewer can confirm them today.

- **Inspect the shipped shell directly.** The BYOM router (~216-line stdlib file + config, one disclosed system line), the allow-listed action runner (closed `ACTIONS` dict; unknown name → returns the allowed list, executes nothing), the file-based memory (declared-files-only, never globs), the audit surface + token meter, and the

local stdlib server (same-origin, every route degrades to a clean "unavailable" at HTTP 200) are all **present and readable in the repo**. (_DIGEST.md §2)

- **Run the public reproducibility harness.** The published shell ships a stdlib-only, network-free `scripts/verify_all.py`; a reviewer clones the repo and runs it on a fresh checkout with no API keys to get `PASS (8/8)`, exit 0 — verifying the gate hard-floor, fail-closed scoring, the append-only queue, honest BYOM no-config, retention KEEP/REVERT, the allow-listed runner, HTTP-200 routes, and gitignored secrets. This is the audit's strongest **executed** proof against published code, with no AI judgment in the loop. (scripts/verify_all.py; metrics.md §5)
- **Verify the gate model bites.** The autonomy dial (L0–L5) × risk score (4–20) routing, and the property that **even at L5 a high-risk action still queues** to the append-only `gate-queue.jsonl`, are inspectable in the published code — and executed directly by `verify_all.py` checks V001/V002. (_DIGEST.md §2 row 5)
- **Read the safety hooks.** The allow-list + hard gate + byte-exact revert loop, and the contributor rule forbidding their weakening, are in the repo. (_DIGEST.md §2 row 6)
- **Re-read the public claim register.** The 4 VERIFIED / 8 PARTIAL / 0 UNVERIFIED tally is read directly from the pack's own register and verdict. (metrics.md §4)
- **Re-derive the blended cost rate.** $26,920.27 ÷ 10,533M tokens ≈ $0.00256 / 1,000 tokens is division of two figures present in the ledger. The **token counts are VERIFIED**; only the rate behind the dollar figure needs a current-pricing check. (metrics.md §1, §1b)
- **Confirm the competitor facts.** OpenClaw/Hermes/ECC star counts and the ECC live GitHub API numbers (205,179 stars / 31,492 forks at 2026-06-03), the `SKILL.md` standard's provenance and cross-vendor adoption, and the OpenClaw CVE/breach events are all from public sources a reviewer can re-pull. (openclaw.md; ecc.md; gpt-competitors.md)
- **Check the consumption independence structurally.** Different implementation language from the vendors, segregated reference-copy directories, and running code that does not import those references are observable in the repo layout. (consumption.md §6, §8)
- **Re-run every scalability calculation.** The per-turn cost, the 15× fan-out premium, the Amdahl saturation curve, and the reliability formulas are arithmetic on stated assumptions — re-computable by hand. (scalability.md)

3.2 Reproducible only with a clean checkout + a run (not in the public pack yet)

The *mechanisms* are inspectable; the specific *results* live in internal logs. A reviewer with a clean checkout can reproduce these, but they cannot be confirmed from the bundled materials alone:

- The **14/14** site-generation acceptance run, the quality-eval suite (original-five categories at 100% / full-suite ~79% gate-cleared), the **26/28** smoke-test snapshot, and the **8/8** hook self-audit. (Run the bundled tests on a fresh checkout — metrics.md §4, §8 item 2.) *(Note: the eight public gate/queue/retention/runner properties from this set are already executable today via `scripts/verify_all.py` — §3.1.)*
- The **per-item consumption smoke-tests** as a single aggregate regression. (Add one bundled regression command — consumption.md §7.)

3.3 Not reproducible from public materials at all (yet)

These require access not granted, an artifact not published, or an experiment not run:

- The **~260 build-iteration log** and the **165-module inventory** as individually-validated items — internal logs; the public repo ships a subset, not the full private module set. (metrics.md §2, §3)
- The **fine-tune deltas** (−20pp, +13.3pp) until the training data + eval split + run logs are published; and the **~5,900-pair projection (M14)**, which has **not been run** and cannot be confirmed by any amount of re-reading. (metrics.md §6)
- A **successful real external-model (e.g. GPT) call** — the connector is a mock-driven shell with no successful live call logged; until a live call is captured, the cross-provider path is interface-only. (gpt-competitors.md Part 1)
- The four **roadmap capabilities (C3/C5/C7/C13)** — not in the public repo, so not reproducible as shipped behaviour. (_DIGEST.md §8)

- A **token-level clean-room diff (K5)** — an owed artifact, not yet published.

> **The reproducibility verdict.** The *architecture and the gate/audit/memory/consumption disciplines* are reproducible **now**. The *performance and acceptance tallies* are reproducible **with a clean checkout**. The *fine-tune wins, the autonomous multi-agent execution, and the live cross-provider call* are **not yet reproducible** — and the audit labels them accordingly. An honest buyer's checklist: trust the shell as readable today; require a clean-checkout test run before trusting the tallies; treat the fine-tune and roadmap items as in-progress.

---

4. The honest known limits

Three limits matter most for anyone deciding what the platform is. None is concealed; each is stated with its evidence and its honest framing.

4.1 Small-local-model maturity — a capability-in-progress, not a shipped win

The fine-tune track targets specializing a small open **~0.5B-parameter** instruct model against a **pre-registered** pass bar (format-compliance ≥ +20pp OR category-accuracy ≥ +15pp) (metrics.md §6). The measured record is:

- **50 training pairs → ≈ −20pp.** The first attempt made the small model *worse* — a regression, honestly left in the record. Loss converged high (~3.05), consistent with undertraining (3 epochs / 50 pairs / 0.5B / CPU). **PARTIAL.**
- **~150 pairs → +13.3pp category accuracy.** The first end-to-end *positive* result — it proves the pipeline works (train → adapter → chat-template → local conversion) but **does not clear** the pre-registered bar. Pipeline-validated, target-not-yet-met. **PARTIAL.**
- **~5,900-pair projection → predicted to clear.** A plan, **not a measured result. UNVERIFIED.**

The credibility signal *is* the −20pp and the sub-threshold +13.3pp left in the record. A vendor spinning this would quote only the projection (metrics.md §6). The honest reading: small-local-model specialization here is a **capability-in-progress**, never to be presented as shipped.

This sits inside the broader, independently-grounded reality that small-model maturity is hard everywhere: long-horizon per-step error compounds (S5), and the model layer is the part of the stack the platform most clearly does **not** lead. The sovereignty / run-your-own-model path is validated as a *direction* by peers who ship open weights (Nous/Hermes), but that is roadmap context, not a present SUR capability. (researchers.md; gpt-competitors.md §2.1, §2.3)

A second, narrower limit in the same family: the public reproducibility harness (`scripts/verify_all.py` , 8/8 PASS — §5 / M12) is the audit's strongest **executed** proof, but a PASS means *those eight public properties hold on the published code*, **not** "the system is safe" in the absolute. It checks the gate hard-floor, fail-closed scoring, the append-only queue, honest BYOM no-config, retention KEEP/REVERT, the allow-listed runner, HTTP-200 routes, and gitignored secrets — it is a verifiable floor, not a safety certification.

4.2 The solo-builder asymmetry — throughput is notable, edges are expected

The entire platform is the output of **one person with AI assistance over ~19 recorded calendar days** (_DIGEST.md §8 item 1; metrics.md §0). This asymmetry is weighted honestly in both directions:

- **In the platform's favour:** the throughput (~10.53B tokens of build-and-verify work, the breadth of inspectable disciplines) is genuinely notable for a solo+AI operation, and the disclosure discipline (down-labelling its own claims) is auditor-grade. (metrics.md §1, §4)
- **Against it: counts are an inventory, not a validation.** The ~165 modules is a self-asserted, reconciler-guarded figure; a meaningful fraction carry an "encoded but not yet independently validated" status. Acceptance tallies are real and their mechanisms inspectable, but most specific runs live in internal logs

(metrics.md §2, §4, §8). Unfinished edges are *expected* from a 19-day solo build and should be read as such, not spun as completeness.
- **Bus-factor is a shared category trait, not a unique weakness.** ECC carries the same near-solo asymmetry (`affaan-m` = 1,426 commits vs the #2 human's 47, ~30:1) (ecc.md). So solo-build is **not a differentiator either way** — what differs is governance maturity, which the next limit addresses.

The disciplined rule throughout this audit: **competitor maturity is a roadmap to learn from, not a deficit to spin** (consumption.md §7; _DIGEST.md §8 item 1).

4.3 The go-to-market / third-party-trust-proof gap — behind on reach, not on engineering

The clearest place the platform is **behind** is not the code; it is distribution and external validation (_DIGEST.md §8 item 2):

- **Reach / ecosystem.** Incumbents sit at **175K–355K stars** with large skill marketplaces (OpenClaw ~247K–355K; Hermes ~175K; ECC 205,179) (openclaw.md; ecc.md; gpt-competitors.md §2). The SUR has no comparable public footprint.
- **Shipped paid go-to-market.** ECC already ships a **$19/seat/mo** tier, GitHub Sponsors, a ~1,000-test CI suite, and an 80% coverage gate (ecc.md). The SUR's commercialization is not yet a shipped, externally-purchasable product.
- **Third-party trust proof.** There is **no on-record external-AI critique or endorsement** of the SUR, and **no successful live external-model call** is logged (gpt-competitors.md Part 1). What the SUR *does* offer a third party today is the **public reproducibility harness** (`scripts/verify_all.py`, 8/8 PASS, exit 0): anyone can clone the open repo and execute the gate/queue/retention/runner properties for themselves, with no AI judgment and no keys (§5 / M12). That is verifiable, but it is self-published code a reviewer runs — not an independent third-party endorsement, which the SUR does not have.
- **Open-model-production line.** The platform has no shipped open-weight model; peers in the category do (gpt-competitors.md §2.3).

The honest conclusion the source research converges on: **the gap is distribution and third-party trust-proof, not engineering** (_DIGEST.md §8 item 2). The remedy is reach, independent verification, and a shipped paid offering — exactly the unlock paths in §2 (publish artifacts, enable clean-checkout reruns) plus a go-to-market motion the competitor landscape already demonstrates is achievable.

---

5. What this ledger lets a reviewer conclude

Pulling the labels together gives a precise, defensible verdict — and nothing beyond it.

- **The shipped public artifact delivers what it claims on its actual scope:** a verifiable, gate-first, BYOM agentic operating-system shell. The gate layer, least-privilege runner, inspectable memory, audit surface, fail-closed retention, and stdlib reliability are **VERIFIED and readable today**. (_DIGEST.md §2, §9)
- **The defensible edge is structural, not architectural** — and that is a *stronger* claim than a harness-uniqueness claim would be, because in this category no harness uniqueness is available to anyone (T1–T3). The most load-bearing finding is the human-in-the-loop gate that still queues high-risk actions at maximum autonomy — precisely the remedy the category's worst documented security failures call for (X1, X2).
- **The numbers sit inside the public envelope.** Caching saves real money but does not cancel the ~15× fan-out premium (P1, P2, S2); concurrency saturates near ~8–10 effective workers (P4); the cheap-routing edge is depreciating (P3, S6). No figure exceeds what the independent literature supports.
- **The unfinished parts are labelled, not hidden:** four roadmap capabilities not in the repo (C3/C5/C7/C13), a fine-tune track that is genuinely in-progress (M13/M14), and a go-to-market gap that is real (§4.3). The single UNVERIFIED-and-unrunnable item is the fine-tune projection (M14) — and it is flagged as such.
- **Most of the distance between PARTIAL and VERIFIED is disclosure, not engineering** (§2). That is the most actionable conclusion in this chapter: the credibility of this audit can be materially raised by clean-

checkout reruns and published artifacts, with one genuine experiment (M14) outstanding.

No part of this audit asserts that the platform "beats frontier," is "fully sovereign," or approaches "AGI." None of those is claimed in any source, and the labels above are the reason a reader can trust that restraint: where the evidence was not run, it is marked — by the platform's own materials first, and by this audit second. That self-down-labelling is the single most reassuring property in the entire body of evidence.

---

6. Strongest counterarguments — pre-empted, not hidden

This section exists because a hostile reader should encounter the strongest objections *with* their honest responses, not discover them independently and conclude the audit was hiding them. Each attack below is mounted at full force; the response is given only where one genuinely exists. Where a point requires a concession, it is conceded.

6.1 "Consume-not-harness is just rebranding"

The attack: The SUR uses the same open `SKILL.md` format as its competitors, took patterns from the same ecosystem tools they build on, and re-implemented those patterns in its own language — which every derivative tool also does. The word "consume" is a narrative choice, not an observable engineering distinction.

Honest response (partial concession): The attack correctly identifies that "consume" carries more implication than the evidence directly supports. The falsifiable claim behind the terminology is narrower: the SUR's **running code carries no imported dependency on any consumed reference source.** That is a real property — it means no supply-chain coupling, no GPL contamination risk, no runtime reliance on a vendor's release cadence — and it is structurally verifiable (different implementation language, segregated reference directories, no import linkage; consumption.md §3). The narrative framing is defensively weak. The engineering property is real.

Concession: "Consume" as a differentiating brand should be read as shorthand for that specific falsifiable property, not as a claim of broader architectural distinctiveness at the harness level, which the audit explicitly concedes is commodity.

6.2 "The defensibility is overstated"

The attack: Every described architectural component — the HITL gate, the retention loop, the allow-listed action runner — is simple enough that a team reading this audit could reproduce any of them in a day. The audit describes how to copy the SUR's differentiators while claiming those differentiators are defensible.

Honest response (qualified concession): The attack is correct that no individual component is a permanent, copy-proof advantage, and this audit explicitly does not claim otherwise. The honest claim throughout is a **current-state structural advantage**, not a permanent one: the VERIFIED shipped capabilities constitute an advantage specifically over the documented failure modes of the most-adopted category incumbents (CVE-2026-25253, the ~1.5M-token breach, the "shadow superuser" demonstration — openclaw.md; gpt-competitors.md §2). A team that reads this audit and then builds the equivalent of `autonomy.py` and `verify_queue.py` would be closing the exact gap the CVE record evidences — and the existence of that gap today, among the most-adopted tools, is what the current-state claim rests on.

Concession: The docs use "current-state structural advantage" rather than any permanent-advantage framing throughout; informal reads of the word "defensible" should be understood in the bounded sense above. The four UNVERIFIED roadmap capabilities (C3/C5/C7/C13) contribute nothing to this claim and should not be read as if they do.

6.3 "The metrics are cherry-picked"

The attack: The strongest acceptance tallies are PARTIAL because the specific runs are internal. The only independently verifiable quantitative figures are the token counts (machine-generated) and the fine-tune deltas — and the fine-tune deltas show a regression (−20pp) and a sub-threshold result (+13.3pp). The headline numbers are internal; the verifiable numbers are the failures.

Honest response: The distribution of labels in the metrics chapter is not cherry-picking — it is the evidence structure. The −20pp regression, the corrected eval/smoke figures, the stale-vs-current cost correction, and the 8 PARTIAL claims are all present and labeled. A cherry-picked metrics chapter would quote only the clean tallies and omit the regression; this audit calls the −20pp "the credibility signal" (metrics.md §6).

Presentation fix applied: The Executive Summary (doc 00) now carries the PARTIAL labels co-located with every quoted acceptance tally ("14/14 (PARTIAL — internal log)", the eval shown as original-five 100% / full-suite ~79% gate-cleared, "26/28 (PARTIAL)") rather than quoting them bare. The labels that lived downstream in doc 07 are now carried upstream to where the figures first appear.

6.4 "Single-operator means this is not real engineering"

The attack: No CI, no peer review, no coverage gate, no external test suite, no paying customers, no third-party security audit. Four capabilities unshipped. A fine-tune track with a net regression. A smoke test that fails some checks. This is a prototype, not a production engineering artifact.

Honest response: Every item in that list is already stated in this audit at the locations cross-referenced throughout; no part of it is hidden. The accurate framing of the asymmetry: **this is engineering at prototype/MVP scale with verification debt outstanding**, not "not real engineering." The stdlib-only Python server, the inspectable gate logic, the byte-exact retention loop — these are verifiable engineering artifacts with provably correct properties independent of team size. "Even at L5 a high-risk action still queues" is either true or false and is checkable by reading the gate function in `autonomy.py`. Team size does not change that property.

The gap is specific and documented: ECC ships ~1,000 tests, an 80% coverage gate, and a $19/seat/mo paid tier (ecc.md). The SUR does not. That gap is distribution and verification infrastructure, not a category gap in engineering correctness on the shipped components. Both can be true simultaneously.

Unlock: Publish the bundled tests; enable clean-checkout reruns. Most of the distance from "prototype" to "verified" is disclosure, not new engineering (§2 of this document).

6.5 "An audit written with the subject's own AI is not independent"

The attack: The primary build tool (Claude Opus 4.8) also authored this audit. The same model that built the SUR evaluated it. The multi-model cross-check is three other AI models, not independent humans. "Independent" in any professional audit sense requires no prior involvement, no stake, and human accountability — none of which applies here. This is structurally a self-review.

Concession (significant): This attack is **substantially correct on the terminology.** The audit does not use "independent" in the professional-audit sense. The README abstract and the executive-summary header have been changed to "structured technical audit built with AI assistance, applying a labeled-evidence standard (VERIFIED / PARTIAL / UNVERIFIED) to primary sources," and a methodology note states the AI-authorship limitation up front.

What survives the attack: The VERIFIED claims in this audit rest on **publicly readable code and public registry data**, not on the AI's assessment of its own work. A reviewer who has never seen Claude can read `autonomy.py`, `verify_queue.py`, `retention.py`, and the public OpenClaw CVE advisories, and confirm every section-A VERIFIED claim (`09-claims-register.md` §A) without trusting a single AI judgment. That is the honest meaning of "independently verifiable" here — the evidence is public-source, not AI-attested. The AI's role was to read the public sources and structure the labeling; the labels themselves can be re-derived by any reviewer from the same sources.

The multi-model cross-check is not a substitute for human peer review. It is a de-correlation mechanism that catches within-family blind spots — useful, but it does not constitute an independent human audit. What the audit offers a reviewer *instead* of an internal cross-check record is the **public reproducibility harness** (`scripts/verify_all.py`, 8/8 PASS, exit 0): the load-bearing safety properties are executable on the published code by anyone, with no model judgment in the loop (§5 / M12).

The honest bounded claim: This is a structured, labeled, multi-model-cross-checked review of a shipped codebase, where the evidence for VERIFIED claims is independently readable public code. It is not a Big Four audit, a NIST assessment, or an ISO certification. It is more transparent — with labeled evidence grades, explicit down-labeling, and a stated reproducibility boundary — than most product documentation in this category. It is not, and does not claim to be, professionally independent.

6.6 Summary verdict table

Attack	Concession required?	Where it is handled
"Consume-not-harness is rebranding"	PARTIAL — tighten terminology	§6.1 above; consumption.md §3, §5
"Defensibility is overstated"	YES — "current-state structural advantage", not a permanent copy-proof advantage	§6.2 above; doc 00 "A note on 'defensibility'"
"Metrics are cherry-picked"	NO on substance; YES on presentation	§6.3 above; doc 00 PARTIAL labels co-located
"Single-operator = not real engineering"	YES on scale; NO on category	§6.4 above; §4.2 of this document
"Audit written with own AI is not independent"	YES — language concession	§6.5 above; README + doc 00 header + methodology note

---

All figures carry VERIFIED / PARTIAL / UNVERIFIED labels per the source research. Cost figures are API-equivalent cost-to-serve, not cash. Every multiplier states its scope. This document is sealed for outward publication and asserts no unbacked superlatives. The platform consumes open-source patterns one-way into an owned framework; it does not ship a harness and is not described as one.

Claims & Evidence Register — Make the Audit Independently Checkable

Independent audit by Haris Mekic — built with Claude Opus 4.8 (lead author and engineer), with Claude Sonnet, OpenAI GPT, and Google Gemini used for cross-checks — 2026-06-03

Purpose

This appendix exists so a skeptical outside reviewer never has to take the audit on faith. It collects the audit's load-bearing claims into one table, grades each **VERIFIED / PARTIAL / UNVERIFIED**, and — the part that matters — gives each one a **reproduction step**: the exact thing a reviewer can run or read to confirm it. Where a claim is PARTIAL or UNVERIFIED, the register states the **specific** evidence that is missing and the action that would close it. Cost figures are API-equivalent cost-to-serve, not cash; no superlative is asserted.

The register's own rule: a claim is only **VERIFIED** here if it was read directly from published code/data **or executed**. Inspection of a mechanism without a run is **VERIFIED (mechanism)**; a specific result that lives in an internal log is **PARTIAL** until a clean-checkout run reproduces it.

> **A note on independence.** This audit was built by the same AI model (Claude Opus 4.8) that built the platform under review, with Sonnet, GPT, and Gemini for cross-checks. The claims labelled **VERIFIED** in section A below rest on **publicly readable code and public registry data that any reviewer can inspect directly** — they do not rest on the AI's assessment of its own work. This is a structured, labelled, multi-model technical review with a public-source evidence base; it is **not** a professionally-independent auditor engagement, and section A is the part designed to be checked without trusting any AI judgment. (See `06-methodology.md` for the full methodology and `08-limits-and-reproducibility.md` §6 for the direct rebuttal to the "an audit written with the subject's own AI is not independent" objection.)

A. Reproducible right now (public repo only)

These need nothing but the published Apache-2.0 repository.

#	Claim	Grade	Reproduction step (run/read this)
R-1	**The autonomy gate bites: at maximum autonomy (L5) a high-risk action still queues; an invalid score fails closed to queue.**	**VERIFIED (executed)**	`cd server && python autonomy.py --set demo 5 && python autonomy.py --decide demo 18` → expect `"verdict":"queue"`, band `refuse`. `--decide demo 5` → `act`. `--decide demo 999` → `queue` (fail-closed). Exit code is 2 for queue, 0 for act/report.
R-2	**The action runner is allow-listed, not a raw shell: an unknown action returns the allowed list and executes nothing.**	**VERIFIED**	Read `server/servari_server.py` `_run_action`: a closed `ACTIONS` dict; an unknown name returns `{ok:false, "refused…", allowed:[…]}`.
R-3	**Memory is file-based and the surface reads only declared files (never globs); a missing declared**	**VERIFIED (mechanism)**	Read `server/memory_surface.py`; remove a declared file and confirm the surface reports `updated:"missing"`.

#	Claim	Grade	Reproduction step (run/read this)
	file surfaces honestly as "missing".		
R-4	**BYOM router is one small stdlib file with a single disclosed system line and no hidden prompt; missing config degrades to an honest message, never a fabricated reply.**	**PARTIAL**	Read `server/chat_byom.py` (≈216 lines): `SYSTEM_LINE` is the entire system prompt; `is_configured()` gates an honest `ok=False` message. The "key transmitted only to the chosen provider" property is consistent with the single `urllib` call but not exhaustively traced.
R-5	**Quality-gated retention loop: KEEP only if no metric degrades, else byte-exact sha256-verified revert; fail-closed; a run can be decided once.**	**VERIFIED (mechanism)**	Read `server/retention.py` `baseline()` / `decide()` invariants; run `baseline → edit a target to degrade a metric → decide` on a clean checkout and confirm the byte-exact revert.
R-6	**Local server is pure stdlib, same-origin, every route degrades to a clean "unavailable" at HTTP 200; health is fail-closed.**	**VERIFIED**	Start the server; request a route whose backing data is absent → HTTP 200 with an "unavailable" payload, not a crash. Read `server/health.py` for the fail-closed degrade.
R-7	**Append-only audit boundary: the gate queue and retention audit are append-only and never rewritten.**	**VERIFIED (mechanism)**	Read `server/verify_queue.py` (`enqueue` / `decide` / `history`): pending and decision are separate append lines to `gate-queue.jsonl` .
R-8	**Consumption independence is structural: owned code is stdlib Python; vendor references are segregated; running code does not import the references.**	**VERIFIED structurally**	Inspect the repo layout; grep the owned scripts for imports of the reference directories (expect none). A token-level clean-room diff would upgrade this to conclusive (see C-2).
R-9	**Token ledger blended rate is internally consistent.**	**VERIFIED (derived)**	$26,920.27 ÷ 10,533M tokens ≈ $0.00256 per 1,000 tokens — division of two ledger figures. Counts are ground truth; the dollar figure is rate-dependent cost-to-serve, not cash.
R-10	**Competitor public facts (star counts, the open skill standard's provenance, the documented security events) are from public sources.**	**VERIFIED (snapshots)**	Re-pull the GitHub APIs and the cited CVE/breach reports; the skill-standard provenance and cross-vendor adoption are independently confirmable. Exact "largest / beat React" framings are date-dependent (PARTIAL). See `docs/REFERENCES.md` .

#	Claim	Grade	Reproduction step (run/read this)
R-11	**All scalability arithmetic is re-computable by hand on stated assumptions.**	**VERIFIED**	Re-derive the per-turn cost, the ~15× fan-out premium, the Amdahl saturation curve, the own-server break-even, and the reliability formulas from the inputs stated in `04-scalability-science.md`.

B. Reproducible with a clean checkout + a run

The *mechanisms* are inspectable; the specific *results* live in internal logs.

#	Claim	Grade	Unlock step
B-1	14/14 multi-site generation acceptance (spec valid + 4/4 pages + working contact form + export-cleanliness scan + live HTTP 200).	PARTIAL	Run the bundled deterministic generator on a clean checkout.
B-2	Quality-evaluation harness: the original five categories pass at 100%; the full expanded suite clears its ≥70% gate at ~79% (the failures are concentrated in newer, harder framing/discipline categories — see `07-metrics.md` §4).	PARTIAL	Re-run the bundled harness; its grader self-test is inspectable. Report the suite version with the score — the older "20/20" is for a smaller 5-category suite.
B-3	Aggregate health smoke-test: 26 of 28 checks pass on the current expanded suite (the older "19/19" denominator is a strict subset).	PARTIAL	Run the one-command health check; report the snapshot and the denominator.
B-4	8/8 mechanical safety hooks present + parsing + running; one "firing-but-blind" hook found and fixed in the same pass.	PARTIAL	Re-run the hook self-audit on a clean checkout.
B-5	Product-boundary interview: probes generated; the scorer flags a planted leak (gate bites).	PARTIAL	Bundle the protocol generator + scorer with a fixture transcript; reproduce the planted-leak flag. Note: leak scoring is literal-term, not paraphrase-aware.
B-6	Per-item consumption smoke-tests (4/4, 5/5, 8/8) pass.	PARTIAL	Add and run a single aggregate regression that executes all per-item smoke-tests together.

C. Not reproducible from public materials yet

Access not granted, artifact not published, or experiment not run.

#	Claim	Grade	What would move it
C-1	Small-model fine-tune deltas (−20pp → +13.3pp, still sub-threshold).	PARTIAL	Publish training data + eval split + run logs against the open ~0.5B base.
C-2	"No vendor code copied" at the line level.	PARTIAL	Publish a token-level clean-room diff of owned scripts vs reference copies.
C-3	~5,900-pair fine-tune projection clears the bar.	**UNVERIFIED**	**Run the experiment.** It is a projection, not a result — never quote it alone.
C-4	A successful real external-model (e.g. GPT) round-trip.	UNVERIFIED	Capture one live call; today the connector is a mock-driven shell with no successful live call logged ($0.00, zero tokens).
C-5	The four roadmap capabilities (distinct sandbox-execute primitive · general MCP layer · concurrent execution engine · failure-pattern regression memory).	UNVERIFIED in repo	Ship them in a publicly auditable component with the same inspectable shape as the verified capabilities.
C-6	Pattern-source attributions in the public NOTICE.	PARTIAL	A NOTICE exists for runtime dependencies; add the consumed-pattern attributions (SWE-agent / agentevals, both MIT; the named harness-ecosystem patterns consumed pattern-only). One edit closes it.

The pattern across the register

Almost every distance between **PARTIAL** and **VERIFIED** here is a **clean-checkout re-run** or a **publish-the-artifact** step — not new engineering. The single exception is the ~5,900-pair fine-tune projection (C-3), which genuinely requires running an experiment that has not been run. That distribution is itself the finding: the credibility ceiling of this audit is raised mostly by *disclosure and reproduction*, not by building anything new. The one claim a reader should personally run first is **R-1** — the gate that still queues a high-risk action at maximum autonomy — because it is the audit's central thesis and it takes two commands to confirm.

Cost figures are API-equivalent cost-to-serve, not cash. No claim of "beating frontier," "full sovereignty," or "AGI" appears in this register or anywhere in the audit, by design.

References

*Consolidated external literature for this independent audit. Every URL below was verified reachable and on-topic as of 2026-06-03. Labels follow the audit-wide convention — **VERIFIED** (the source states the claim directly), **PARTIAL** (substance supported, a figure/wording is paraphrased by a secondary outlet or revised since), **UNVERIFIED** (repeated but not confirmed against a primary here). Cost/inference figures are API-equivalent cost-to-serve, never cash.*

---

1. The paradigm — LLM-as-Operating-System / Software 3.0

- Andrej Karpathy, *Software Is Changing (Again)* — Y Combinator AI Startup School, 2025-06-17. Primary host: https://www.ycombinator.com/library/MW-andrej-karpathy-software-is-changing-again — **VERIFIED**
- Video: https://www.youtube.com/watch?v=LCEmiRjPEtQ — **VERIFIED**
- Author confirmation (announcing the talk): https://x.com/karpathy/status/1935518272667217925 — **VERIFIED**
- Talk notes (LLM-OS analogy, 1960s-mainframe framing, Llama→Linux ecosystem split): http://ikyle.me/blog/2025/andrej-karpathy-software-is-changing-again — **VERIFIED** (faithful summary)
- Software 3.0 explainer: https://travis.media/blog/software-3-0-ai-changing-programming-karpathy/ — **VERIFIED** (summary)

2. AI safety / alignment context — AI 2027

- AI 2027 (project site), AI Futures Project — https://ai-2027.com — **VERIFIED**
- About / authors (Kokotajlo, Lifland, Larsen, Dean): https://ai-2027.com/about — **VERIFIED**
- Timeline revision toward the early 2030s (secondary): https://www.hungarianconservative.com/articles/tech/ai-futures-project-ai-2027-apocalypse-timeline-superintelligence-daniel-kokotajlo/ — **PARTIAL**

3. Multi-agent orchestration — the case FOR and AGAINST

- Anthropic, *How we built our multi-agent research system* (90.2% upside, ~15× tokens, ~80% variance): https://www.anthropic.com/engineering/multi-agent-research-system — **VERIFIED**
- Cognition, *Don't Build Multi-Agents* (the steel-man against parallel sub-agents): https://cognition.ai/blog/dont-build-multi-agents — **VERIFIED**
- Cognition-vs-Anthropic debate, same-day aggregation (2025-06-13): https://news.smol.ai/issues/25-06-13-cognition-vs-anthropic — **VERIFIED** (third-party)
- *The Orchestration of Multi-Agent Systems* (academic corroboration of gating conditions): https://arxiv.org/abs/2601.13671 — **VERIFIED**

4. Context engineering & long-horizon reliability

- Anthropic, *Effective context engineering for AI agents* (+29% / +39%; 84% token cut): https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents — **VERIFIED**
- Anthropic, *Effective harnesses for long-running agents*: https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents — **VERIFIED**
- METR, *Measuring AI Ability to Complete Long Tasks* (~7-mo doubling): https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/ ; paper https://arxiv.org/abs/2503.14499 — **VERIFIED**
- *The Illusion of Diminishing Returns: Measuring Long-Horizon Execution in LLMs* (per-step error rises): https://arxiv.org/abs/2509.09677 — **VERIFIED**

- *Beyond pass@1: A Reliability Science Framework for Long-Horizon LLM Agents* (domain-stratified decay): https://arxiv.org/abs/2603.29231 — **VERIFIED**

5. Inference economics

- Epoch AI, *LLM inference prices have fallen rapidly but unequally across tasks* (9×–900×/yr, median ~50×/yr): https://epoch.ai/data-insights/llm-inference-price-trends — **VERIFIED**
- Prompt caching / batching economics (caching ~90% cost / ~85% latency on long stable prefixes): https://www.prompthub.us/blog/prompt-caching-with-openai-anthropic-and-google-models — **VERIFIED** (secondary; reputable)

6. The open-source agentic-OS ecosystem (comparables)

- OpenClaw (MIT; local-first Markdown memory; ~377K stars as of 2026-06-03): https://github.com/openclaw/openclaw ; site https://openclaw.ai/ — **VERIFIED**
- OpenClaw — independent organizational-impact commentary (NVIDIA): https://blogs.nvidia.com/blog/what-openclaw-agents-mean-for-every-organization/ — **VERIFIED** (third-party)
- Nous Research (Hermes open-weights; primary org): https://nousresearch.com — **VERIFIED**
- Hermes 4.3 release (first model post-trained on the Psyche network): https://nousresearch.com/introducing-hermes-4-3 — **VERIFIED**
- Nous Research / Psyche decentralized-training deep-dive (independent): https://oakresearch.io/en/analyses/innovations/nous-research-psyche-open-source-decentralized-ai-revolution — **VERIFIED** (third-party); the ~$50M Paradigm round / ~$1B valuation figure is **PARTIAL** (secondary): https://www.gate.com/news/detail/nous-research-deep-dive-a-decentralized-ai-lab-that-paradigm-invested-in-at-20290685

7. The open SKILL.md standard (shared, cross-vendor layer)

- Canonical specification: https://agentskills.io/specification — **VERIFIED**
- Overview: https://agentskills.io/home — **VERIFIED**
- Origin — Anthropic, *Equipping agents for the real world with Agent Skills*: https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills — **VERIFIED**
- Public repo: https://github.com/anthropics/skills — **VERIFIED**

8. Pattern sources consumed (pattern-only, no vendor code)

- SWE-agent (MIT) — the agent-computer-interface (ACI) edit-safety / syntax-gate pattern: https://github.com/SWE-agent/SWE-agent — **VERIFIED** (real public MIT project)
- agentevals (langchain-ai, MIT) — the trajectory-evaluation pattern: https://github.com/langchain-ai/agentevals — **VERIFIED** (real public MIT project)

Note on arXiv identifiers: arXiv IDs use the `YYMM.NNNNN` convention (e.g. `2503.*` = Mar 2025, `2601.*` = Jan 2026). The `/abs/` form is the stable landing page. If any single ID fails to resolve at read-time, treat that row as **PARTIAL** pending re-confirmation; the substance is independently corroborated by the Anthropic / Cognition / METR / Epoch primaries above.